

Manual de MikroTik

RouterOS do básico ao avançado, com foco na operação de provedor de internet (fibra e rádio)

Provedor de Fibra Óptica

2026-07-06

Índice

Apresentação	11
Para quem é este manual	11
Como este manual funciona	11
Mapa de pré-requisitos	11
As três fases	13
Convenções usadas no livro	13
I Volume 1 — Primeiros passos	14
1 O que é MikroTik e RouterOS	15
1.1 A empresa e o sistema	15
1.1.1 Um pouco de história	16
1.1.2 RouterOS ou SwOS?	16
1.2 Por que os provedores escolhem MikroTik?	16
1.3 Onde o MikroTik se encaixa na rede de um provedor	17
1.4 O CHR: seu laboratório gratuito	18
1.5 Laboratório	18
1.5.1 Ambiente	18
1.5.2 Comandos passo a passo	19
1.5.3 Verificação	20
1.5.4 Desafios	20
1.6 Resumo	21
Bibliografia do capítulo	22
2 Linhas de hardware MikroTik	23
2.1 O catálogo visto de longe	23
2.2 Decifrando o nome dos produtos	23
2.3 As famílias, papel por papel	24
2.3.1 CCR — Cloud Core Router: o núcleo	25
2.3.2 CRS — Cloud Router Switch: o switch do POP	25
2.3.3 hEX, hAP e L009/RB5009 — roteadores de mesa	25
2.3.4 SXT, LHG, DISC, Wire — a família de rádio	26
2.3.5 CHR — o “hardware” que é software	26
2.4 As especificações que realmente importam	26
2.5 Laboratório	27
2.5.1 Ambiente	27
2.5.2 Comandos passo a passo	27

2.5.3	Verificação	28
2.5.4	Desafios	28
2.6	Resumo	29
	Bibliografia do capítulo	29
3	Licenças RouterOS e RouterBOOT	30
3.1	Licença: o que você compra junto com a caixa	30
3.1.1	A licença do CHR: outro modelo	32
3.2	RouterBOOT: o que acorda o RouterOS	32
3.2.1	As proteções do RouterBOOT	33
3.3	Laboratório	34
3.3.1	Ambiente	34
3.3.2	Comandos passo a passo	34
3.3.3	Verificação	35
3.3.4	Desafios	35
3.4	Resumo	36
	Bibliografia do capítulo	36
4	Formas de acesso ao RouterOS	37
4.1	Muitas portas para a mesma casa	37
4.1.1	WinBox — a ferramenta símbolo	37
4.1.2	WebFig — o WinBox no navegador	38
4.1.3	SSH — a porta da linha de comando	39
4.1.4	Console serial — o cabo da UTI	39
4.1.5	MAC-Telnet — terminal sem IP	39
4.2	Safe Mode: o cinto de segurança	40
4.3	Os serviços em <code>/ip service</code>	40
4.4	Laboratório	41
4.4.1	Ambiente	41
4.4.2	Comandos passo a passo	41
4.4.3	Verificação	42
4.4.4	Desafios	42
4.5	Resumo	43
	Bibliografia do capítulo	43
5	Montando o laboratório com CHR	45
5.1	Um laboratório para o livro inteiro	45
5.1.1	Os três tipos de rede do VirtualBox	46
5.2	GNS3 e EVE-NG: quando crescer	47
5.3	Laboratório	47
5.3.1	Ambiente	47
5.3.2	Comandos passo a passo	47
5.3.3	Verificação	49
5.3.4	Desafios	49
5.4	Resumo	50
	Bibliografia do capítulo	50

6	Primeiro acesso, usuários e segurança inicial	52
6.1	O que vem de fábrica	52
6.2	Usuários e grupos: aposentando o <code>admin</code> compartilhado	53
6.3	Fechando as portas: serviços e acesso	54
6.4	Relógio certo: segurança, não estética	54
6.5	Laboratório	55
6.5.1	Ambiente	55
6.5.2	Comandos passo a passo	55
6.5.3	Verificação	57
6.5.4	Desafios	57
6.6	Resumo	58
	Bibliografia do capítulo	58
7	Backup, export e reset	59
7.1	Duas fotografias diferentes da mesma configuração	59
7.2	Backup binário: gerar, baixar, restaurar	60
7.3	Export: a configuração como texto	61
7.4	Reset: recomeçar sem sustos	61
7.5	Laboratório	62
7.5.1	Ambiente	62
7.5.2	Comandos passo a passo	62
7.5.3	Verificação	64
7.5.4	Desafios	64
7.6	Resumo	65
	Bibliografia do capítulo	66
8	Netinstall	67
8.1	Quando o reset não basta	67
8.2	Como funciona por baixo	68
8.3	Laboratório	69
8.3.1	Ambiente	69
8.3.2	Comandos passo a passo	70
8.3.3	Verificação	70
8.3.4	Desafios	70
8.4	Resumo	71
	Bibliografia do capítulo	72
II	Volume 2 — CLI e configuração essencial	73
9	Anatomia do terminal	74
9.1	A árvore e os verbos	74
9.2	Itens, números e o perigo escondido	75
9.3	TAB e ?: o manual que mora no equipamento	76
9.4	<code>print</code> afiado e <code>export</code> como lupa	77

9.5	Laboratório	78
9.5.1	Ambiente	78
9.5.2	Comandos passo a passo	78
9.5.3	Verificação	79
9.5.4	Desafios	80
9.6	Resumo	80
	Bibliografia do capítulo	81
10	Endereçamento IP e rota padrão	82
10.1	O endereço e a máscara, sem mistério	82
10.2	/ip address: o que você dá e o que o RouterOS deduz	83
10.3	Rotas estáticas: ensinando o caminho	84
10.3.1	distance: o desempate entre rotas	84
10.3.2	A rota padrão e o check-gateway	85
10.4	Laboratório	85
10.4.1	Ambiente	85
10.4.2	Comandos passo a passo	86
10.4.3	Verificação	88
10.4.4	Desafios	88
10.5	Resumo	89
	Bibliografia do capítulo	89
11	DHCP client e server	90
11.1	O empréstimo de endereços	90
11.2	DHCP client: a WAN que se configura sozinha	91
11.3	DHCP server: a LAN que se serve sozinha	92
11.3.1	Leases: o livro de registros	92
11.4	Laboratório	93
11.4.1	Ambiente	93
11.4.2	Comandos passo a passo	93
11.4.3	Verificação	95
11.4.4	Desafios	95
11.5	Resumo	96
	Bibliografia do capítulo	96
12	DNS no RouterOS	97
12.1	O catálogo telefônico da internet	97
12.2	Configurando o resolvidor	98
12.2.1	allow-remote-requests: servidor, e para quem?	98
12.3	Entradas estáticas: o DNS interno do provedor	99
12.4	Laboratório	99
12.4.1	Ambiente	99
12.4.2	Comandos passo a passo	100
12.4.3	Verificação	101
12.4.4	Desafios	101
12.5	Resumo	102

Bibliografia do capítulo	103
13 NAT masquerade	104
13.1 O problema: cartas sem remetente válido	104
13.2 Como a máscara funciona	104
13.3 A regra no RouterOS	105
13.3.1 masquerade ou src-nat?	106
13.3.2 O que o NAT não é	106
13.4 Laboratório	107
13.4.1 Ambiente	107
13.4.2 Comandos passo a passo	107
13.4.3 Verificação	108
13.4.4 Desafios	108
13.5 Resumo	109
Bibliografia do capítulo	110
14 Projeto: roteador de borda	111
14.1 A ordem de serviço	111
14.2 Laboratório	112
14.2.1 Ambiente	112
14.2.2 Comandos passo a passo	112
14.2.3 Verificação	115
14.2.4 Desafios	115
14.3 Resumo	116
Bibliografia do capítulo	117
III Volume 3 — Firewall	118
15 Connection tracking	119
15.1 A memória que muda tudo	119
15.2 Os quatro estados	120
15.3 Lendo a tabela	121
15.4 O custo da memória	121
15.5 Laboratório	122
15.5.1 Ambiente	122
15.5.2 Comandos passo a passo	122
15.5.3 Verificação	123
15.5.4 Desafios	124
15.6 Resumo	125
Bibliografia do capítulo	125
16 Chains e a lógica do firewall	126
16.1 Três tribunais, uma pergunta	126
16.2 A lógica: de cima para baixo, primeira que casa decide	127
16.2.1 As actions	127

16.2.2	Chains próprias: organização por <code>jump</code>	128
16.3	O esqueleto que você vai repetir para sempre	128
16.4	Laboratório	129
16.4.1	Ambiente	129
16.4.2	Comandos passo a passo	129
16.4.3	Verificação	131
16.4.4	Desafios	131
16.5	Resumo	132
	Bibliografia do capítulo	132
17	Protegendo o roteador	133
17.1	Default deny: a inversão que muda tudo	133
17.2	Interface lists: regras que sobrevivem à topologia	133
17.3	O inventário: o que o input precisa aceitar	134
17.3.1	A política de ICMP	135
17.4	A sequência completa	135
17.5	Laboratório	136
17.5.1	Ambiente	136
17.5.2	Comandos passo a passo	136
17.5.3	Verificação	137
17.5.4	Desafios	138
17.6	Resumo	139
	Bibliografia do capítulo	139
18	Forward e address-lists	140
18.1	O forward de um provedor não é o de uma empresa	140
18.2	Address lists: separar política de dados	141
18.3	Listas dinâmicas: o firewall que aprende	141
18.4	Laboratório	142
18.4.1	Ambiente	142
18.4.2	Comandos passo a passo	142
18.4.3	Verificação	144
18.4.4	Desafios	144
18.5	Resumo	145
	Bibliografia do capítulo	146
19	dstnat e port forwarding	147
19.1	O espelho do masquerade	147
19.2	Firewall e dstnat: aceitando com precisão	148
19.3	Hairpin NAT: o defeito clássico	148
19.4	<code>redirect</code> : dstnat para o próprio roteador	149
19.5	Laboratório	150
19.5.1	Ambiente	150
19.5.2	Comandos passo a passo	150
19.5.3	Verificação	151
19.5.4	Desafios	151

19.6	Resumo	152
	Bibliografia do capítulo	153
20	Mangle e RAW	154
20.1	A linha de montagem completa	154
20.2	Mangle: o almoxarifado de etiquetas	155
20.3	RAW: agir antes de existir	155
20.4	Laboratório	156
20.4.1	Ambiente	156
20.4.2	Comandos passo a passo	157
20.4.3	Verificação	158
20.4.4	Desafios	158
20.5	Resumo	159
	Bibliografia do capítulo	159
21	Hardening completo	161
21.1	Defesa em camadas: o mapa do volume	161
21.2	As peças finais	162
21.3	O checklist consolidado	163
21.4	Laboratório	164
21.4.1	Ambiente	164
21.4.2	Comandos passo a passo	164
21.4.3	Verificação	166
21.4.4	Desafios	166
21.5	Resumo	167
	Bibliografia do capítulo	167
IV	Volume 4 — Bridge, switching e VLANs	169
22	Bridge e hardware offload	170
22.1	Camada 2: o andar de baixo	170
22.2	Bridge no RouterOS	171
22.3	Hardware offload: quem carrega o piano?	171
22.4	Laboratório	173
22.4.1	Ambiente	173
22.4.2	Comandos passo a passo	173
22.4.3	Verificação	174
22.4.4	Desafios	175
22.5	Resumo	176
	Bibliografia do capítulo	176
23	VLANs e 802.1Q	177
23.1	O problema: um cabo, várias redes	177
23.2	VLANs no RouterOS: a interface <code>vlan</code>	178

23.3	Laboratório	179
23.3.1	Ambiente	179
23.3.2	Comandos passo a passo	179
23.3.3	Verificação	181
23.3.4	Desafios	181
23.4	Resumo	182
	Bibliografia do capítulo	182
24	Bridge VLAN filtering	184
24.1	Do consentimento à lei	184
24.1.1	Peça 1: a tabela de VLANs da bridge	185
24.1.2	Peça 2: o PVID — a etiqueta de entrada	185
24.1.3	Peça 3: frame-types — o que a porta admite	185
24.2	A gerência e a pegadinha do tagged=bridge	186
24.3	Laboratório	187
24.3.1	Ambiente	187
24.3.2	Comandos passo a passo	187
24.3.3	Verificação	189
24.3.4	Desafios	189
24.4	Resumo	190
	Bibliografia do capítulo	190
25	Switch chip nos CRS	191
25.1	O porão do switch	191
25.2	Dois mundos, uma decisão	192
25.3	O que mais mora no chip	192
25.4	Laboratório	193
25.4.1	Ambiente	193
25.4.2	Comandos passo a passo	194
25.4.3	Verificação	194
25.4.4	Desafios	195
25.5	Resumo	196
	Bibliografia do capítulo	196
26	Segmentação da rede do provedor	197
26.1	A ordem de serviço	197
26.2	Os três princípios	198
26.3	O firewall enxergando segmentos	198
26.4	Laboratório	200
26.4.1	Ambiente	200
26.4.2	Comandos passo a passo	200
26.4.3	Verificação	201
26.4.4	Desafios	201
26.5	Resumo	202
	Bibliografia do capítulo	203

Apêndices	204
Referências	204

Apresentação

Bem-vindo ao **Manual de MikroTik**, um livro aberto da série *Manuais de Ciências*. Este manual leva você do zero absoluto — “o que é um roteador MikroTik?” — até a operação do backbone de um provedor de internet real, com fibra óptica e enlaces de rádio em área rural.

Para quem é este manual

Para quem está começando em redes e quer chegar ao nível de **operar a rede de um provedor (ISP)**: técnicos de campo que querem migrar para o NOC, donos de pequenos provedores, estudantes e curiosos. Não pressupomos conhecimento prévio de RouterOS; cada conceito é explicado antes de ser usado, e a ordem dos capítulos respeita os pré-requisitos entre eles.

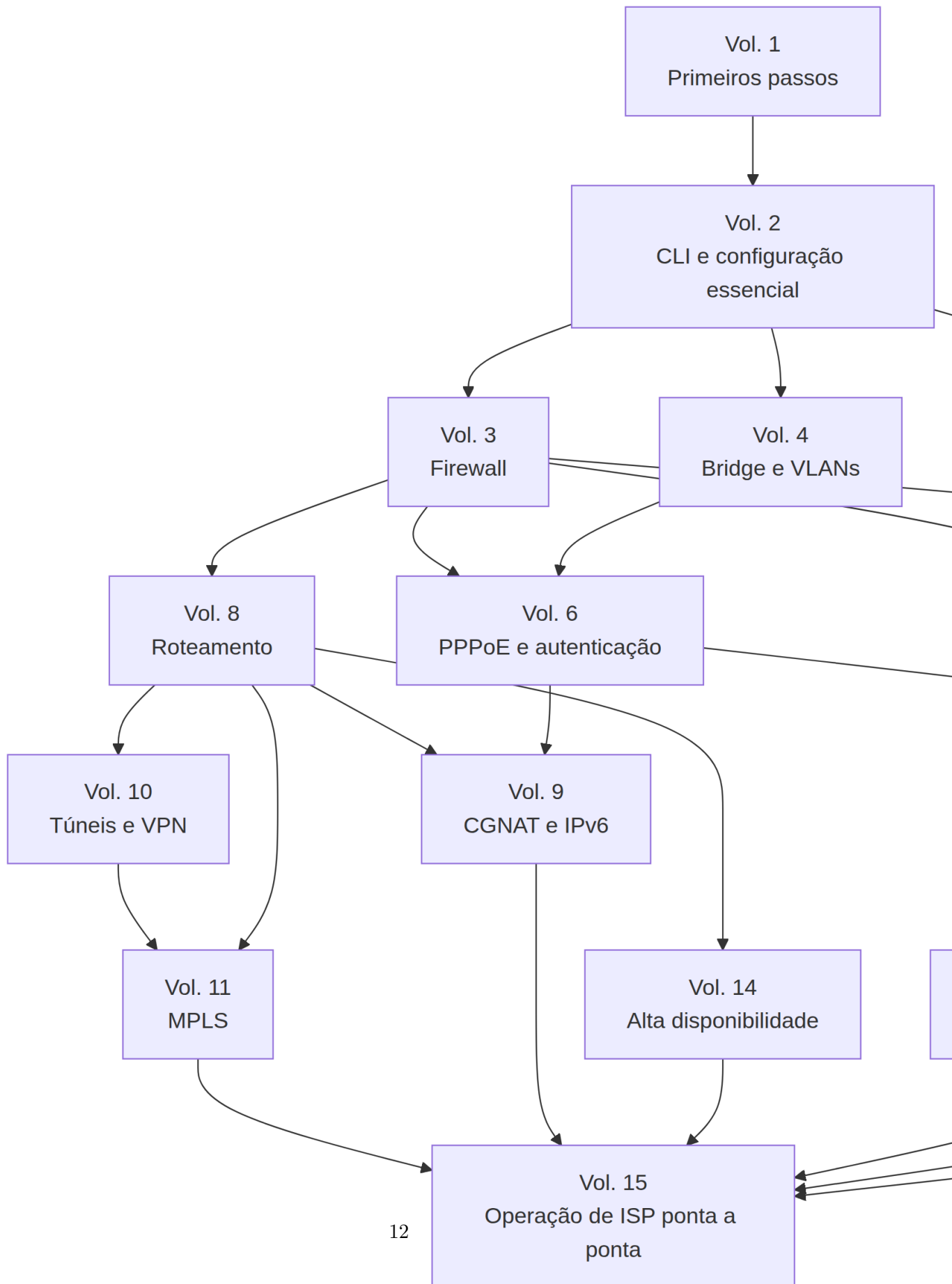
Se você quiser reforçar a teoria de redes por trás dos capítulos (modelo OSI, endereçamento IP, fibra óptica, rádio), o [Manual de Redes de Computadores](#) da mesma série é o companheiro ideal deste livro.

Como este manual funciona

- **Progressão gradual**: os volumes formam três fases — fundamentos, provedor na prática e operação avançada. Cada capítulo declara o que você precisa ter visto antes.
- **Laboratório em todo capítulo**: você pratica *enquanto* estuda, usando o **MikroTik CHR** (RouterOS em máquina virtual) — sem precisar comprar hardware. O Capítulo 5 ensina a montar o ambiente uma única vez; os demais capítulos apenas indicam a topologia necessária.
- **Comandos reais**: todos os blocos de código são RouterOS de verdade, prontos para copiar e colar no terminal, sempre explicados linha a linha.
- **Desafios com solução**: cada laboratório termina com exercícios; as soluções ficam escondidas em caixas colapsáveis para você tentar antes de olhar.

Mapa de pré-requisitos

O grafo abaixo mostra a dependência entre os volumes: siga as setas para saber o que estudar antes de cada assunto.



Grafo de pré-requisitos entre os volumes do manual.

As três fases

Fase 1 — Fundamentos e operação básica (Volumes 1–4): o que é o RouterOS, como acessá-lo e operá-lo com segurança; a linha de comando; endereçamento, DHCP, DNS e NAT; firewall completo; bridges, switching e VLANs. Ao final desta fase você configura, do zero, um roteador de borda seguro e segmentado.

Fase 2 — Provedor na prática (Volumes 5–9): os pilares do dia a dia de um ISP — enlaces de rádio PtP/PtMP, PPPoE com RADIUS, planos de velocidade com QoS, roteamento OSPF/BGP, CGNAT e IPv6.

Fase 3 — Avançado e operação (Volumes 10–15): túneis e VPN entre POPs, MPLS no backbone, automação com scripts, monitoramento, alta disponibilidade e, fechando o livro, o desenho ponta a ponta da rede de um provedor real de fibra e rádio.

Convenções usadas no livro

- Blocos de código `routeros` contêm comandos prontos para colar no terminal. O prompt `[admin@MikroTik] >` só aparece quando ilustramos uma sessão interativa com a saída do equipamento.
- Caixas de aviso () antecedem **todo** comando destrutivo (`reset`, `Netinstall`, remoção de regras).
- Referências bibliográficas seguem o padrão ABNT e estão listadas ao final de cada capítulo.

Bons estudos — e boas configurações!

Parte I

Volume 1 — Primeiros passos

1 O que é MikroTik e RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é a MikroTik (a empresa) e o que é o RouterOS (o sistema);
- Diferenciar RouterOS de SwOS e entender por que essa distinção importa;
- Reconhecer o que torna o MikroTik tão popular entre provedores de internet;
- Localizar, num diagrama de rede de provedor (fibra + rádio), os pontos onde equipamentos MikroTik costumam trabalhar;
- Dar o primeiro passo prático: baixar o CHR (RouterOS em máquina virtual) e fazer o primeiro login.

1.1 A empresa e o sistema

MikroTik é uma empresa da Letônia, fundada em 1996 na capital Riga, que faz duas coisas: um sistema operacional de rede chamado **RouterOS** e o hardware que roda esse sistema, a linha **RouterBOARD**. Essa dupla — software poderoso e hardware barato — transformou a MikroTik numa das marcas mais usadas por provedores de internet do mundo, especialmente em países como o Brasil, onde milhares de ISPs regionais construíram suas redes sobre ela.

Aqui vale uma analogia. Pense num computador comum: ele tem um hardware (a máquina física) e um sistema operacional (Windows, Linux) que faz o hardware ser útil. Com o MikroTik é igual:

- O **RouterOS** é o sistema operacional. É ele que roteia pacotes, filtra tráfego no firewall, autentica assinantes por PPPoE, fala OSPF e BGP com outros roteadores — tudo o que este manual ensina é, no fundo, “como operar o RouterOS”.
- A **RouterBOARD** é a família de hardware: roteadores de mesa, roteadores de rack com dezenas de núcleos, switches, antenas e rádios. Todos saem de fábrica com o RouterOS instalado.

O detalhe importante — e que explica boa parte da fama da marca — é que **o RouterOS é o mesmo em todos os equipamentos**. O sistema que roda num roteadorzinho de R\$ 300 é o mesmo que roda no roteador de rack que segura o BGP de um provedor com dezenas de milhares de clientes. Muda o desempenho do hardware; não muda a linguagem, os menus

nem os conceitos. É por isso que tudo o que você aprender neste manual num laboratório virtual gratuito vale, sem tradução, para qualquer equipamento físico da marca.

Analogia para guardar

RouterOS está para a RouterBOARD assim como o Android está para um celular: o mesmo sistema, em aparelhos de tamanhos e preços muito diferentes. Aprenda o sistema uma vez e você sabe operar todos.

1.1.1 Um pouco de história

A MikroTik nasceu vendendo software: no fim dos anos 1990, o RouterOS era instalado em PCs comuns para transformá-los em roteadores — uma alternativa muito mais barata que os grandes roteadores comerciais da época. Em 2002 a empresa passou a fabricar o próprio hardware (a RouterBOARD), e o conjunto decolou entre provedores pequenos e médios: preço de equipamento doméstico, recursos de equipamento de operadora.

Duas versões do sistema importam para você hoje:

- **RouterOS v6** — dominou as redes por mais de uma década; você ainda vai encontrá-lo em muito equipamento antigo em campo.
- **RouterOS v7** — a geração atual, com motor de roteamento reescrito (OSPF/BGP), WireGuard, suporte a hardware novo e melhorias contínuas. **Este manual ensina RouterOS v7**; quando uma diferença do v6 for relevante na vida real do provedor, ela será apontada explicitamente.

1.1.2 RouterOS ou SwOS?

Alguns equipamentos MikroTik — em especial os switches da linha CRS — podem inicializar com um segundo sistema, o **SwOS**: um firmware minimalista que só faz switching (camada 2) e é configurado por uma página web simples. Ele existe porque, quando um switch só precisa comutar quadros, um sistema enxuto dedicado a isso é mais simples de operar.

A regra prática deste manual: **usamos RouterOS em tudo**. O RouterOS também faz switching (e o faz em hardware, como você verá no Volume 4), e manter um único sistema em todo o parque simplifica a operação, os backups e a automação. Saiba que o SwOS existe para não se confundir ao encontrá-lo, mas nosso caminho é o RouterOS.

1.2 Por que os provedores escolhem MikroTik?

Quatro razões aparecem em praticamente toda rede de ISP construída com a marca:

1. **Custo**: um roteador capaz de concentrar centenas de assinantes PPPoE custa uma fração do equivalente das marcas tradicionais de operadora.

2. **Completeness:** firewall com estado, NAT e CGNAT, PPPoE com RADIUS, QoS, OSPF, BGP, MPLS, VPNs, scripts — tudo incluso no sistema, sem licenças adicionais por recurso.
3. **Versatilidade:** o mesmo sistema opera na borda (BGP com a operadora), no concentrador de assinantes, no switch do POP, no rádio da torre e na casa do cliente.
4. **Comunidade:** fóruns, documentação aberta (MikroTik, 2026a), treinamentos e uma enorme base de conhecimento acumulada por provedores do mundo todo.

Há também um ponto de atenção honesto: todo esse poder vem com **muita responsabilidade de configuração**. O RouterOS entrega as ferramentas; quem decide se a rede será segura e estável é quem a configura. É exatamente essa a lacuna que este manual fecha — inclusive dedicando um volume inteiro a firewall e proteção (Volume 3).

1.3 Onde o MikroTik se encaixa na rede de um provedor

Para entender o que você vai construir ao longo do livro, veja o caminho que os dados percorrem num provedor típico do interior, que atende clientes por fibra óptica na cidade e por rádio na zona rural (Figura 1.1):

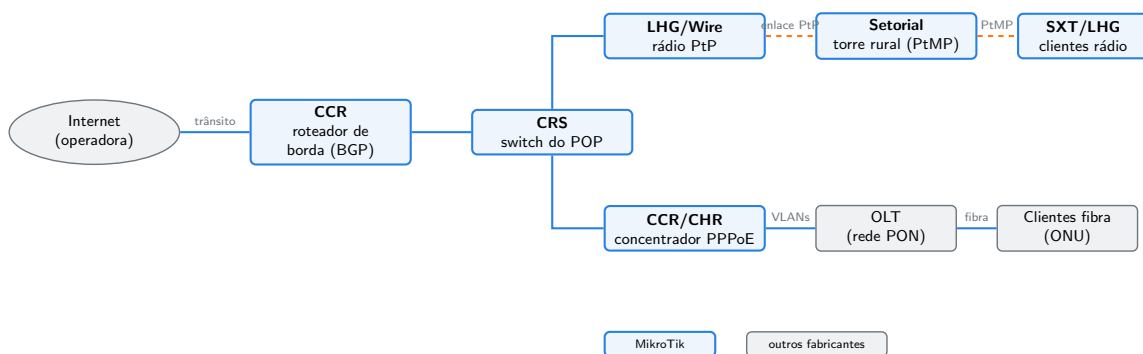


Figura 1.1: Onde os equipamentos MikroTik (em azul) trabalham na rede de um provedor típico de fibra e rádio.

Acompanhe o diagrama da esquerda para a direita:

- **Roteador de borda:** um CCR conversa via **BGP** com a operadora que vende trânsito ao provedor (Volume 8). É a porta de entrada e saída de toda a rede.
- **Switch do POP:** um CRS distribui o tráfego dentro do ponto de presença, separando os serviços em **VLANs** (Volume 4).
- **Concentrador PPPoE (BNG):** um CCR — ou um CHR virtualizado — autentica cada assinante, entrega IP e aplica o plano de velocidade contratado (Volumes 6 e 7).
- **Ramo fibra:** a OLT e as ONUs dos clientes são, em geral, de outros fabricantes — mas quem roteia, autentica e controla a banda desses clientes continua sendo o MikroTik (Volume 15 detalha essa integração).

- **Ramo rádio:** enlaces **PtP** levam a internet até torres na zona rural; lá, antenas setoriais atendem em **PtMP** os rádios na casa de cada cliente — tudo MikroTik, das linhas LHG, SXT e afins (Volume 5).

Guarde esta imagem: os quinze volumes deste manual são, essencialmente, um tour guiado por esse diagrama, do centro para as pontas.

! O conceito mais importante do capítulo

Um único sistema — o RouterOS — opera todos os papéis azuis do diagrama. Aprender RouterOS não é aprender “um roteador”: é aprender a operar **a rede inteira do provedor**.

1.4 O CHR: seu laboratório gratuito

Como praticar tudo isso sem gastar um centavo em equipamento? A resposta é o **CHR** — **Cloud Hosted Router** (MikroTik, 2026b): uma versão do RouterOS empacotada como **máquina virtual**, feita pela própria MikroTik para rodar em VirtualBox, VMware, Proxmox, GNS3, EVE-NG e afins.

O CHR é um RouterOS completo: mesma linha de comando, mesmo firewall, mesmo OSPF e BGP. Sem licença, ele funciona para sempre com uma única limitação — cada interface fica limitada a **1 Mbps** de velocidade, o que não atrapalha em nada o estudo. É nele que rodam todos os laboratórios deste livro.

Neste capítulo você fará o primeiro contato: baixar o CHR, dar o boot e entrar no sistema. A montagem completa do ambiente de estudos — com várias máquinas virtuais interligadas em topologias — é o assunto do Capítulo 5, que você montará uma única vez e reutilizará no livro inteiro.

1.5 Laboratório

1.5.1 Ambiente

Neste primeiro laboratório você precisa apenas de um computador com o **VirtualBox** instalado (gratuito, disponível para Windows, Linux e macOS em [virtualbox.org](https://www.virtualbox.org)). Nada de topologias ainda: uma única máquina virtual basta.

1. Acesse mikrotik.com/download e, na seção **Cloud Hosted Router**, baixe a imagem **VDI** da versão estável mais recente (7.x).
2. No VirtualBox, crie uma máquina virtual nova: tipo **Linux**, versão **Other Linux (64-bit)**, com **256 MB** de RAM.
3. Na etapa de disco, escolha **usar um disco existente** e aponte para o arquivo **.vdi** baixado (não crie disco novo).

4. Inicie a máquina virtual.

Ao final do boot aparece o prompt de login do RouterOS na janela da VM.

1.5.2 Comandos passo a passo

Faça login com o usuário padrão de fábrica — usuário `admin`, senha em branco (só pressionar Enter):

```
MikroTik Login: admin
Password:
```

Nas versões atuais, o RouterOS pede imediatamente uma senha nova — escolha uma e anote. Em seguida ele mostra a licença e o banner com o logotipo, e você cai no prompt do terminal:

```
[admin@MikroTik] >
```

Esse prompt diz duas coisas: quem você é (`admin`) e o nome do equipamento (`MikroTik`, a identidade de fábrica). Vamos fazer o RouterOS se apresentar. O comando abaixo mostra o “RG” do sistema — versão, tempo ligado, memória, CPU e placa:

```
/system resource print
```

A saída esperada é parecida com esta (os números variam conforme sua máquina):

```
[admin@MikroTik] > /system resource print
    uptime: 4m12s
    version: 7.15.3 (stable)
    build-time: 2024-07-24 10:39:01
    free-memory: 190.1MiB
    total-memory: 224.0MiB
        cpu: QEMU
        cpu-count: 1
        cpu-load: 1%
    free-hdd-space: 43.1MiB
    total-hdd-space: 61.4MiB
    board-name: CHR
        platform: MikroTik
```

Repare em `version: 7.x` (estamos no RouterOS v7) e `board-name: CHR` (é a versão virtual). Agora veja as interfaces de rede que a VM enxerga:

```
/interface print
```

```
[admin@MikroTik] > /interface print
Flags: R - RUNNING
Columns: NAME, TYPE, ACTUAL-MTU, MAC-ADDRESS
#  NAME      TYPE      ACTUAL-MTU  MAC-ADDRESS
0 R ether1   ether      1500        08:00:27:6A:2B:1C
```

Uma única `ether1`: é a placa de rede virtual do VirtualBox. Nos próximos capítulos adicionaremos mais. Por fim, dê um nome ao seu roteador — boa prática que você levará para a vida de provedor, onde cada equipamento do parque precisa de identidade única:

```
/system identity set name=lab-r1
```

O prompt muda na hora, confirmando a alteração:

```
[admin@lab-r1] >
```

1.5.3 Verificação

Confirme os três resultados do laboratório:

1. `/system resource print` mostra `version` começando com 7. e `board-name: CHR`;
2. `/interface print` lista ao menos `ether1` com a flag `R` (running);
3. O prompt exibe `[admin@lab-r1]` — a identidade foi gravada.

Se a VM não deu boot, verifique se você **apontou o disco existente** (o `.vdi` baixado) em vez de criar um disco vazio — esse é o erro mais comum.

1.5.4 Desafios

1. Descubra, usando apenas a tecla `?` no terminal, o que faz o comando `/system clock print` — e execute-o. O horário está certo?
2. O RouterOS organiza os comandos em menus, como pastas. Entre no menu `/system` (digite `/system` e Enter) e liste o que há dentro dele com `?`. Depois volte à raiz com `/`.
3. Sem consultar a internet: qual comando você usaria para trocar a identidade do roteador para `chr-teste`? Execute-o e desfaça em seguida, voltando para `lab-r1`.
4. Desligue o roteador **pelo terminal** (não feche a janela da VM no X!). Dica: explore o menu `/system` do desafio 2.

💡 Solução dos desafios

1. Digite `/system clock print ?` (ou apenas execute o comando). Ele mostra data, hora e fuso configurados:

```
/system clock print
```

No CHR recém-instalado o fuso costuma vir errado (`gmt` ou detectado pela VM); você aprenderá a ajustá-lo no capítulo de primeiro acesso.

2. Sessão esperada:

```
[admin@lab-r1] > /system  
[admin@lab-r1] /system> ?
```

O `?` lista submenus como `clock`, `identity`, `resource`, `reboot`, `shutdown`... Para voltar à raiz:

```
[admin@lab-r1] /system> /  
[admin@lab-r1] >
```

3. O mesmo comando do corpo do capítulo, com outro valor:

```
/system identity set name=chr-teste  
/system identity set name=lab-r1
```

4. O comando é `/system shutdown`. Ele pede confirmação — responda `y`:

```
[admin@lab-r1] > /system shutdown  
Shutdown, yes? [y/N]:
```

Desligar pelo sistema (em vez de fechar a VM “no cabo”) evita corromper o armazenamento — nos equipamentos físicos de produção, vale a mesma regra.

1.6 Resumo

- **MikroTik** é a empresa (Letônia, 1996); **RouterOS** é o sistema operacional de rede; **RouterBOARD** é a linha de hardware que o roda.
- O RouterOS é **o mesmo** do equipamento mais barato ao mais caro — o que você aprende no laboratório vale para todo o parque.
- **SwOS** é um firmware alternativo minimalista para switches; neste manual, operamos tudo com RouterOS.
- Provedores escolhem MikroTik por **custo, completude, versatilidade e comunidade** — e assumem a responsabilidade de configurar bem.

- Na rede de um ISP, o MikroTik aparece da **borda BGP** ao **rádio do cliente**, passando por switch, concentrador PPPoE e enlaces PtP/PtMP (Figura 1.1).
- O **CHR** é o RouterOS em máquina virtual, gratuito (limitado a 1 Mbps por interface) — o laboratório perfeito, usado no livro inteiro.
- Você já fez o primeiro login, leu `/system resource print` e batizou seu roteador com `/system identity set`.

No próximo capítulo, conheceremos as linhas de hardware — hEX, hAP, CRS, CCR, LHG/SXT — e como escolher o equipamento certo para cada papel do diagrama.

Bibliografia do capítulo

Para aprofundar o que vimos aqui: a documentação oficial do RouterOS (MikroTik, 2026a) e a página do CHR (MikroTik, 2026b) são as fontes primárias que acompanharão todo o livro; Discher (2016) e Burgess (2011) são bons livros introdutórios em inglês; e Tanenbaum; Feamster; Wetherall (2021) cobre a teoria de redes por trás de tudo. As referências completas, em ABNT, estão no apêndice [Referências](#).

2 Linhas de hardware MikroTik

Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Decifrar o nome de um produto MikroTik (ex.: CCR2004-1G-12S+2XS) e saber, só pelo código, quantas e quais portas ele tem;
- Reconhecer as grandes famílias de hardware — hEX, hAP, RB/L009/RB5009, CRS, CCR, SXT/LHG/DISC — e o papel de cada uma na rede de um provedor;
- Escolher equipamento pelo **papel** que ele exercerá (borda, concentrador, switch de POP, rádio de torre, CPE do cliente), e não pelo preço;
- Avaliar as especificações que realmente importam: CPU (núcleos *versus* desempenho por núcleo), portas SFP/SFP+, PoE e uso externo;
- Consultar o catálogo e as fichas técnicas oficiais da MikroTik com olhar crítico.

2.1 O catálogo visto de longe

No Capítulo 1 você aprendeu que o RouterOS é o mesmo em todos os equipamentos da marca. A consequência prática é libertadora: **escolher hardware MikroTik é escolher desempenho e portas, nunca “recursos”**. O firewall, o PPPoE, o OSPF e o BGP estão em todos; o que muda é quanta CPU e quantas interfaces existem para executá-los.

Por isso, este capítulo não é um catálogo decorado — é um mapa. A pergunta certa nunca é “qual MikroTik é o melhor?”, e sim “**qual papel este equipamento vai exercer na minha rede?**”. Cada papel tem uma família de produtos desenhada para ele.

Analogia para guardar

Escolher equipamento por papel é como montar um time de futebol: não existe “o melhor jogador”, existe o melhor goleiro, o melhor zagueiro, o melhor atacante. Um CCR no papel de rádio de cliente é um goleiro de centroavante — caro e mal aproveitado.

2.2 Decifrando o nome dos produtos

Os nomes MikroTik parecem sopa de letrinhas, mas seguem um código consistente. A parte final do nome descreve as **portas**, e cada letra tem significado (Tabela 2.1):

Tabela 2.1: Sufixos de porta na nomenclatura MikroTik

Código	Significado	Velocidade típica
5, 8, 12...	quantidade de portas do tipo que segue	—
G	Ethernet Gigabit (RJ45)	1 Gbps
P	porta com PoE de saída (alimenta outro equipamento)	—
S	gaiola SFP (fibra)	1 Gbps
S+	gaiola SFP+ (fibra)	10 Gbps
XS	gaiola SFP28 (fibra)	25 Gbps
Q+	gaiola QSFP+ (fibra)	40 Gbps

Com a tabela, o nome assustador CCR2004-1G-12S+2XS vira uma frase: é um **C**loud **C**ore **R**outer da geração 2004, com **1** porta **G**igabit, **12** gaiolas **SFP+** (10 Gbps) e **2** gaiolas **SFP28** (25 Gbps). Um roteador nascido para viver num rack cheio de fibra.

Vale o mesmo para os menores: o hEX S é um hEX com uma gaiola **SFP**; o hAP ax² é um hAP com WiFi padrão **ax** de segunda geração. Quando encontrar um nome novo, decifre as portas primeiro — elas dizem para que rede o produto foi feito.

2.3 As famílias, papel por papel

A Figura 2.1 posiciona as famílias no mesmo diagrama de provedor que você conheceu no capítulo anterior — agora com os nomes das linhas de produto em cada posição:

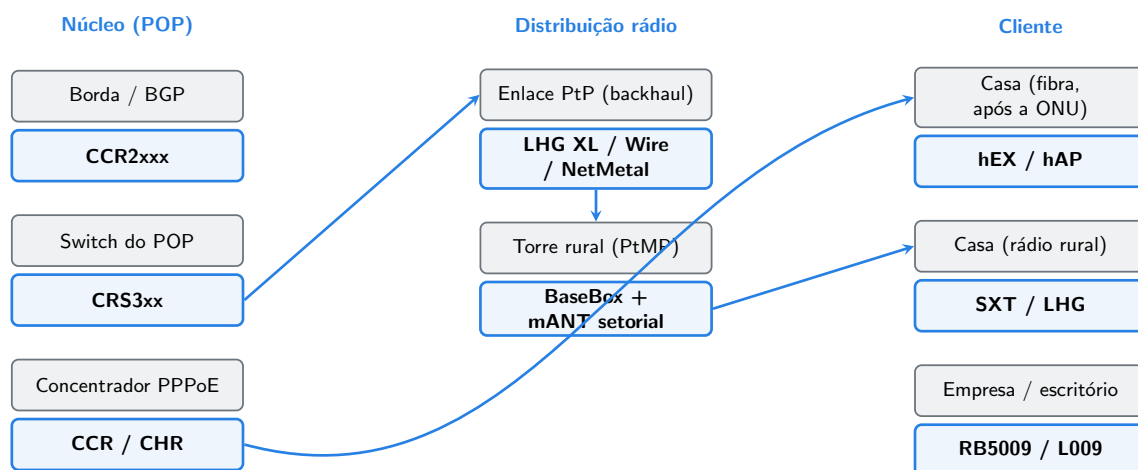


Figura 2.1: As famílias de produto MikroTik posicionadas pelo papel que exercem na rede de um provedor de fibra e rádio.

Vamos percorrer as famílias na mesma ordem — do núcleo para o cliente.

2.3.1 CCR — Cloud Core Router: o núcleo

A linha **CCR** é a série de roteadores de rack da MikroTik: muitos núcleos de CPU, muita porta de fibra, fontes redundantes nos maiores modelos. É o hardware para os papéis mais pesados da rede:

- **Borda BGP** — receber a tabela cheia da internet de uma ou mais operadoras exige memória e CPU; um CCR2004 ou CCR2116 é a escolha natural.
- **Concentrador PPPoE (BNG)** — autenticar milhares de sessões e aplicar filas de velocidade é trabalho de CPU; os CCR de geração 2 (CCR2004, CCR2116, CCR2216) usam núcleos **ARM64** rápidos, muito superiores aos antigos.

Uma nota histórica que ainda importa em campo: os CCR de primeira geração (CCR1009, CCR1016, CCR1036) usam a arquitetura **TILE** — dezenas de núcleos individualmente lentos. Eles brilham quando o tráfego se espalha por muitos núcleos, mas sofrem em tarefas que rodam num núcleo só. Você aprenderá no Volume 6 que uma sessão PPPoE é exatamente esse caso — e por que um CCR2004 com 4 núcleos modernos supera um CCR1036 com 36 núcleos antigos em muitos cenários de provedor.

2.3.2 CRS — Cloud Router Switch: o switch do POP

A linha **CRS** tem cara de switch e alma dupla: roda RouterOS completo, mas carrega um **switch chip** — um circuito dedicado que comuta quadros em hardware, sem gastar CPU. É a família dos POPs: CRS3xx e CRS5xx oferecem combinações de portas Gigabit, SFP+ e até QSFP+ para agregar os enlaces do provedor.

Dois cuidados com essa linha:

- A CPU dos CRS é modesta. Eles são **switches** que sabem rotear em caso de necessidade, não roteadores. Colocar um CRS para concentrar PPPoE é receita de suporte lotado.
- Os modelos CSS (sem o “R”) rodam **apenas SwOS** — o firmware minimalista do Capítulo 1. Confira a letra antes de comprar.

O Volume 4 é inteiro dedicado a extrair o máximo dos CRS: bridge, VLANs e switching em hardware.

2.3.3 hEX, hAP e L009/RB5009 — roteadores de mesa

São os “roteadores de caixinha”, cada um com sua vocação:

- **hEX** — roteador cabeado de 5 portas Gigabit. Barato e confiável, é o clássico atrás da ONU do cliente de fibra que contratou IP fixo ou precisa de um roteador de verdade. O hEX S acrescenta uma gaiola SFP e PoE de saída.
- **hAP** — o hEX com WiFi. A linha atual (hAP ax², hAP ax³) entrega WiFi 6 e é o CPE padrão que muitos provedores instalam na casa do assinante. No Volume 5 você o verá gerenciado em massa via CAPsMAN.

- **L009 e RB5009** — o degrau acima: mais CPU, porta de 2.5/10 Gbps, versões com PoE. Servem pequenas empresas, condomínios e até papéis de POP compacto (um RB5009 aguenta um POP de bairro inteiro).

2.3.4 SXT, LHG, DISC, Wire — a família de rádio

O ramo rural do provedor vive desta família — antenas com o rádio (e o RouterOS) embutidos, prontas para poste e torre:

- **SXT** — antena compacta, abertura de feixe mais larga; o rádio de cliente para distâncias curtas e médias.
- **LHG** — *Light Head Grid*: uma grade parabólica leve com ganho alto; o rádio de cliente para longas distâncias e a opção econômica para enlaces PtP de backhaul (versões LHG XL têm ganho ainda maior).
- **DISC** — parábola sólida fechada, imune a acúmulo de detritos, papel semelhante ao LHG.
- **Wire (Wireless Wire)** — enlaces em **60 GHz**: multi-gigabit em distâncias curtas, perfeitos para atravessar uma rua ou ligar prédios.
- **BaseBox/NetMetal + mANT** — rádios em caixa metálica sem antena integrada, que você combina com uma antena externa — tipicamente uma **setorial mANT** na torre para atender dezenas de clientes em PtMP.

O Volume 5 detalha como projetar esses enlaces; por ora, guarde o padrão: **setorial na torre, SXT/LHG na casa do cliente, LHG XL/Wire no backhaul**.

2.3.5 CHR — o “hardware” que é software

Por fim, lembre que o **CHR** (Capítulo 1) é um membro pleno do catálogo: um RouterOS que roda em máquina virtual sobre um servidor x86. Em produção, provedores usam CHR como concentrador PPPoE, como roteador de borda e como caixa de ferramentas de rede — com a vantagem de escalar CPU e memória comprando servidor, não roteador. Todos os laboratórios deste livro rodam em CHR justamente porque ele é um RouterOS de verdade.

2.4 As especificações que realmente importam

Ficha técnica de roteador tem dezenas de números. Na prática do provedor, cinco decidem a compra:

1. **Desempenho por núcleo da CPU** — tarefas como uma sessão PPPoE ou uma fila de QoS rodam num único núcleo. Quatro núcleos rápidos valem mais que trinta e seis lentos para esses papéis.

2. **Testes de throughput no cenário certo** — a MikroTik publica números em três condições: *fast path* (encaminhamento otimizado, sem firewall), *bridging/routing* simples e **com 25 regras de firewall**. O último é o único próximo da sua realidade. Desconfie de qualquer número de capa.
3. **Portas de fibra (S/S+/XS)** — provedor cresce em fibra. Um equipamento de núcleo sem SFP+ nasce obsoleto.
4. **PoE** — nas pontas, alimentar o rádio pelo próprio cabo de rede (PoE de saída no equipamento de baixo, ou injetor) elimina fonte na torre — menos ponto de falha.
5. **Uso externo e temperatura** — equipamentos de torre (SXT, LHG, BaseBox) são selados para intempérie; equipamentos de mesa, não. Parece óbvio, até você encontrar um hEX dentro de uma marmita de PVC no alto de um poste.

! O conceito mais importante do capítulo

Todo equipamento MikroTik roda o mesmo RouterOS — portanto, **compra-se capacidade, não recurso**. Defina primeiro o papel (borda, concentrador, switch, rádio, CPE), depois dimensione CPU e portas para esse papel, usando os números de teste **com firewall**, não os de capa.

2.5 Laboratório

2.5.1 Ambiente

O mesmo do capítulo anterior: a VM única do CHR no VirtualBox (Capítulo 1). Você também vai precisar de um navegador para consultar mikrotik.com — parte deste laboratório é aprender a ler o catálogo oficial.

2.5.2 Comandos passo a passo

Primeiro, vamos ver o que o RouterOS informa sobre o “hardware” em que está rodando. No CHR:

```
/system resource print
```

Observe três campos na saída: `board-name` (no CHR, CHR; num equipamento físico, o modelo — ex.: hEX S), `cpu-count` e `architecture-name` (x86_64 no CHR; arm64, mipsbe ou tile nos físicos):

```
[admin@lab-r1] > /system resource print
      uptime: 12m4s
      version: 7.15.3 (stable)
architecture-name: x86_64
```

```
board-name: CHR
cpu-count: 1
```

Num equipamento físico existe um menu a mais, que o CHR não tem por ser virtual — ele mostra modelo, número de série e a versão do RouterBOOT (o bootloader, assunto do próximo capítulo):

```
/system routerboard print
```

No CHR esse comando responde `not routerboard` — comportamento esperado, e uma forma rápida de descobrir se você está numa máquina virtual ou física.

Agora a parte de catálogo. Abra mikrotik.com/products no navegador e localize a ficha do **CCR2004-1G-12S+2XS**. Na aba de especificações, encontre a tabela **Test results** e compare as linhas de throughput em *fast path* e com *25 ip filter rules*. Anote os dois números — a diferença entre eles é o “preço” do firewall, e você reencontrará esse conceito no Volume 3.

2.5.3 Verificação

1. `/system resource print` no seu CHR mostra `architecture-name: x86_64` e `board-name: CHR`;
2. `/system routerboard print` responde `not routerboard` (você está numa VM);
3. Você anotou os dois números de throughput do CCR2004 (com e sem firewall) e sabe explicar por que são diferentes.

2.5.4 Desafios

1. Decifre, sem consultar o site: quantas portas, e de que tipo, tem um CRS326-24G-2S+? E um CCR2116-12G-4S+?
2. Um provedor precisa de um switch para um POP novo: 20 clientes de fibra dedicados em portas Gigabit e 2 uplinks de 10 Gbps para o núcleo. Que família e que modelo do desafio 1 atendem?
3. Um cliente rural mora a 12 km da torre, com visada limpa. SXT ou LHG? Por quê?
4. Sua borda BGP com a operadora receberá tabela cheia e crescerá para dois trânsitos. Por que um CCR1036 (36 núcleos TILE) pode ser pior que um CCR2004 (4 núcleos ARM64) nesse papel, mesmo tendo nove vezes mais núcleos?

Solução dos desafios

1. CRS326-24G-2S+: Cloud Router Switch com **24** portas **Gigabit** RJ45 e **2** gaiolas **SFP+** (10 Gbps). CCR2116-12G-4S+: Cloud Core Router com **12** portas **Gigabit** e **4** gaiolas **SFP+**.
2. Papel de switch de POP → família **CRS**. O CRS326-24G-2S+ do desafio 1 serve

como uma luva: 24 portas Gigabit para os clientes (sobram 4) e 2 uplinks SFP+ de 10 Gbps para o núcleo.

3. LHG. A 12 km, o que decide é o ganho da antena: a grade parabólica do LHG concentra o feixe e entrega sinal utilizável onde o SXT (feixe mais aberto, ganho menor) já não alcança com qualidade. O SXT é para distâncias curtas/médias; acima de ~8–10 km, LHG (ou DISC).

4. Processar atualizações de BGP e recalculando a tabela de rotas são tarefas que rodam em **um núcleo** — o que importa é o desempenho por núcleo, e cada núcleo ARM64 do CCR2004 é muito mais rápido que um núcleo TILE de 2012. Os 36 núcleos do CCR1036 só ajudariam se a carga se espalhasse por eles, o que não é o caso do BGP.

2.6 Resumo

- O nome do produto descreve as portas: **G** (Gigabit), **S** (SFP), **S+** (SFP+ 10G), **XS** (25G), **P** (PoE de saída) — decifre o nome antes de ler a ficha (Tabela 2.1).
- Famílias por papel (Figura 2.1): **CCR** no núcleo (borda BGP, concentrador), **CRS** como switch de POP, **hEX/hAP** na casa do cliente, **RB5009/L009** em empresas, **SXT/LHG/DISC/Wire** no rádio, **BaseBox + mANT setorial** na torre.
- **CSS CRS**: modelos CSS rodam apenas SwOS.
- CCR geração 1 (TILE) tem muitos núcleos lentos; geração 2 (ARM64) tem poucos núcleos rápidos — para PPPoE e BGP, vence o desempenho **por núcleo**.
- Avalie throughput sempre no teste **com regras de firewall**, nunca no número de capa (*fast path*).
- O **CHR** é membro pleno do catálogo: RouterOS em VM, usado em produção como concentrador e borda — e como nosso laboratório.

No próximo capítulo, veremos o que dá o “direito de rodar” ao RouterOS — os níveis de licença — e o RouterBOOT, o bootloader que você acabou de procurar com `/system routerboard print`.

Bibliografia do capítulo

O catálogo oficial (MikroTik, 2026c) e as fichas técnicas de cada produto são a fonte primária deste capítulo — em especial as tabelas *Test results*, cuja leitura crítica discutimos. A documentação do RouterOS (MikroTik, 2026a) descreve os menus `/system resource` e `/system routerboard`. Sobre arquiteturas de hardware de rede em geral, Tanenbaum; Feamster; Wetherall (2021) é a referência teórica. As referências completas, em ABNT, estão no apêndice [Referências](#).

3 Licenças RouterOS e RouterBOOT

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar como funciona o licenciamento do RouterOS: níveis 0–6, o que cada um libera e por que a licença acompanha o equipamento para sempre;
- Diferenciar as licenças de equipamento físico das licenças do CHR (**free**, **p1**, **p10**, **p-unlimited**) e da modalidade de teste de 60 dias;
- Consultar a licença de um equipamento com `/system license print`;
- Explicar o que é o RouterBOOT, o que ele faz no boot e por que mantê-lo atualizado junto com o RouterOS;
- Conhecer as proteções do RouterBOOT (senha, **protected-routerboot**) e os riscos de usá-las sem critério.

3.1 Licença: o que você compra junto com a caixa

Uma dúvida comum de quem chega ao MikroTik vindo de outras marcas: “quanto custa a assinatura do sistema?”. A resposta surpreende: **nada**. O modelo da MikroTik não tem mensalidade, contrato de suporte obrigatório nem licença por recurso. Cada equipamento RouterBOARD sai de fábrica com uma **licença perpétua** de um certo **nível**, embutida no preço — e essa licença:

- **nunca expira**;
- inclui **todas as atualizações** do RouterOS dentro da mesma major version (e, na prática, a MikroTik liberou até hoje as trocas de major — v5→v6→v7 — sem custo);
- pertence ao **equipamento**, não a você: se o roteador for vendido, a licença vai junto.

O que os níveis diferenciam não é “recurso” — firewall, BGP, MPLS e scripts existem em todos —, mas **capacidade de alguns serviços**, principalmente quantos túneis e sessões o equipamento pode servir e para que uso o wireless está liberado (1).

Tabela 3.1: O que muda entre os níveis de licença mais comuns (*como servidor; como cliente, ilimitado)

	Nível 3 (CPE)	Nível 4 (WISP)	Nível 5 (WISP AP)	Nível 6 (Controller)
Wireless cliente (station)	sim	sim	sim	sim
Wireless AP (ponto de acesso)	—	sim	sim	sim
Túneis PP-PoE/PPTP/L2TP/OVPN ativos	1*	200	500	ilimitado
Túneis EoIP	1*	200	500	ilimitado
Sessões hotspot ativas	1	200	500	ilimitado
Interfaces de usuário (vlan, queues...)	ilimitado	ilimitado	ilimitado	ilimitado

Os níveis que faltam na tabela são triviais: **nível 0** é o modo de demonstração de 24 horas de uso (o que você obtém instalando o RouterOS x86 sem licença); **nível 1** é a demo registrada, quase inútil hoje; **nível 2** não existe mais (era das versões antigas).

Na prática do provedor, você raramente escolhe nível — ele vem casado com o hardware:

- Equipamentos de **cliente** (SXT, LHG, hEX, hAP) vêm com **nível 3 ou 4**;
- Equipamentos de **torre e POP** (BaseBox, NetMetal, RB4011/5009) vêm com **nível 4 ou 5**;
- Equipamentos de **núcleo** (CCR) vêm com **nível 6**.

Repare como os limites casam com os papéis do 2: o nível 3 não sobe AP wireless (é licença de *station*, de rádio de cliente); o nível 6 não limita túneis (é licença de concentrador PPPoE). A tabela de níveis é o catálogo de papéis visto por outro ângulo.



Analogia para guardar

A licença do RouterOS é como a carta de alforria gravada no chassi de um carro: paga uma vez, vale para sempre e vai junto na venda. Os níveis são como categorias da CNH — todas dirigem, mas cada uma autoriza um porte de veículo (aqui: uma quantidade de sessões e um papel wireless).

3.1.1 A licença do CHR: outro modelo

O CHR usa um licenciamento próprio, por **velocidade de interface** — e é por isso que ele é o laboratório perfeito:

Tabela 3.2: Licenças do CHR

Licença CHR	Limite por interface	Custo
free	1 Mbps	gratuito, para sempre
p1	1 Gbps	pago (perpétuo)
p10	10 Gbps	pago (perpétuo)
p-unlimited	ilimitado	pago (perpétuo)

Além dessas, todo CHR novo pode ativar um **trial de 60 dias** de qualquer nível pago (`/system license renew`, com uma conta gratuita do site da MikroTik) — útil para testar desempenho real antes de comprar. Para os laboratórios deste livro, o **free** com seu 1 Mbps por interface é mais que suficiente: estudamos comportamento, não velocidade.

Consultar a licença é um comando só, e a saída é diferente em cada mundo. No CHR:

```
/system license print
```

```
[admin@lab-r1] > /system license print
system-id: hK3x-Wm9P
level: free
```

Num equipamento físico apareceriam **software-id**, **nlevel** (o nível 3–6) e **features**. O **system-id/software-id** identifica a instalação — é ele que você informa à MikroTik ao comprar upgrade de licença.

3.2 RouterBOOT: o que acorda o RouterOS

Todo computador tem um programa minúsculo que roda antes do sistema operacional — no PC, o firmware UEFI/BIOS; nas RouterBOARD, o **RouterBOOT**. Quando você liga um MikroTik físico, é o RouterBOOT que:

1. inicializa CPU e memória;
2. decide **de onde dar o boot** — do armazenamento interno (normal) ou da rede (modo de recuperação usado pelo Netinstall, Capítulo 8);
3. carrega o RouterOS e lhe entrega o controle.

O RouterBOOT tem firmware próprio, com versão independente do RouterOS — e aqui mora uma tarefa de manutenção que muitos esquecem. Ao atualizar o RouterOS, a versão nova do bootloader vem junta no pacote, mas **só é gravada quando você manda** e reinicia. O fluxo completo:

```
/system routerboard print
```

```
[admin@MikroTik] > /system routerboard print
routerboard: yes
board-name: hEX S
model: RB760iGS
serial-number: HEA08XXXXXX
firmware-type: mt7621L
factory-firmware: 6.46.3
current-firmware: 6.49.10
upgrade-firmware: 7.15.3
```

Quando `current-firmware` está atrás de `upgrade-firmware`, aplique:

```
/system routerboard upgrade
/system reboot
```

Manter RouterOS e RouterBOOT na mesma geração evita uma classe inteira de problemas — de suporte a hardware a correções de segurança do próprio boot. No CHR não existe nada disso (`not routerboard`): quem inicializa a VM é o “BIOS” do hipervisor.

3.2.1 As proteções do RouterBOOT

Por padrão, qualquer pessoa com acesso físico a uma RouterBOARD manda nela: basta segurar o botão de reset ao ligar para limpar a configuração, ou inicializar pela rede e reinstalar o sistema pelo Netinstall. Para equipamento em poste, torre ou armário compartilhado, o RouterBOOT oferece camadas de defesa em `/system routerboard settings`:

- `boot-device=flash-boot-once-etherboot` mantém o boot normal mas ainda permite recuperação; já `boot-protocol` e afins controlam o boot por rede;
- **senha do bootloader** (`/system routerboard settings set boot-protocol=... protected-routerboot=...`) — as opções variam por modelo;
- **protected-routerboot=enabled** — o modo forte: desabilita o botão de reset e o boot por rede. O equipamento só reinstala/reseta com uma intervenção muito mais trabalhosa.

Aviso

`protected-routerboot=enabled` corta justamente os mecanismos de **recuperação**: sem botão de reset e sem Netinstall, uma senha esquecida transforma o equipamento num peso de papel até você conseguir desativar a proteção — o que exige acesso admin ao RouterOS. Só ative em equipamento exposto a acesso físico de terceiros, com a senha do admin guardada em gerenciador de senhas do provedor, e **documente** quais equipamentos do parque têm a proteção ligada.

O conceito mais importante do capítulo

No MikroTik, o custo do software é **zero ao longo da vida** do equipamento: licença perpétua, atualizações incluídas. O que os níveis limitam é capacidade de sessões/túneis e o papel wireless — mais um motivo para escolher hardware pelo papel, como no 2. E abaixo do RouterOS vive o RouterBOOT: atualize-o junto com o sistema e proteja-o com critério.

3.3 Laboratório

3.3.1 Ambiente

A VM única do CHR no VirtualBox, como nos capítulos anteriores. (A partir do Capítulo 5 montaremos topologias maiores.)

3.3.2 Comandos passo a passo

Veja a licença do seu CHR:

```
/system license print
```

Saída esperada:

```
[admin@lab-r1] > /system license print
    system-id: hK3x-Wm9P
    level: free
```

`level: free` confirma a licença gratuita — cada interface limitada a 1 Mbps, sem prazo de validade. Agora confirme que o CHR não tem RouterBOOT:

```
/system routerboard print
```

```
[admin@lab-r1] > /system routerboard print
routerboard: no
```

Por fim, explore o menu de histórico do sistema, que registra mudanças de configuração — ele nos ajudará a auditar o que os laboratórios alteram:

```
/system history print
```

A saída lista as últimas ações (como o `set name=lab-r1` do primeiro capítulo), com autor e horário.

3.3.3 Verificação

1. `/system license print` mostra `level: free`;
2. `/system routerboard print` responde `routerboard: no` — logo, nada de RouterBOOT para gerenciar no laboratório;
3. `/system history print` registra as alterações que você fez até aqui.

3.3.4 Desafios

1. Um provedor compra um **hAP ax²** (nível 4) para a casa de um assinante e um **CCR2116** (nível 6) para concentrador PPPoE. Algum dos dois precisará de upgrade de licença para exercer seu papel? Justifique com a
 - 1.
2. No site da MikroTik, cada produto informa o nível de licença na ficha técnica. Verifique o nível do **SXT SA5** e explique por que ele **não** poderia ser usado como AP de uma torre PtMP com essa licença.
3. Seu parque tem 40 rádios em torres de difícil acesso. Monte (em texto) a política de proteção do RouterBOOT: o que ativar, o que documentar e qual o plano B se a senha do admin de um rádio se perder.
4. Sem executar ainda: qual seria o efeito de `/system license renew` num CHR novo? Quando isso seria útil na vida real?



Solução dos desafios

1. Nenhum dos dois. O hAP ax² atenderá **uma** casa: o nível 4 permite AP wireless e até 200 túneis — décadas de folga para um CPE. O CCR2116 como concentrador precisará de **milhares** de sessões PPPoE, e o nível 6 é justamente o ilimitado. A MikroTik casa o nível com o papel de fábrica.
2. O SXT SA5 vem com **nível 3** — licença de *station*: pode conectar-se a um AP, mas não pode **ser** AP (a linha “wireless AP” da 1 é “—” no nível 3). Para a torre PtMP usa-se um equipamento nível 4+ (ex.: BaseBox com mANT setorial), e os SXT ficam

nos clientes. (Existe a variante “SXT SA5 ap”, vendida com nível 4, exatamente para isso.)

3. Política razoável: (a) ativar `protected-routerboot=enabled` apenas nos rádios fisicamente expostos; (b) senha de admin única por equipamento, guardada no gerenciador de senhas do provedor; (c) registrar em inventário quais equipamentos têm a proteção; (d) plano B: com a senha do admin perdida e proteção ativa, resta o procedimento de desbloqueio de fábrica — na prática, RMA. Por isso a proteção só entra onde o risco de acesso físico supera o risco de perda de senha.

4. `/system license renew` ativa o **trial de 60 dias** de um nível pago (`p1/p10/p-unlimited`), pedindo login de uma conta MikroTik. Útil para testar um CHR de produção — por exemplo, medir se um servidor aguenta o papel de concentrador PPPoE a 10 Gbps — antes de pagar a licença perpétua.

3.4 Resumo

- Licença RouterOS: **perpétua**, sem mensalidade, com atualizações incluídas; pertence ao equipamento.
- Níveis 3–6 diferenciam **papel wireless** (station/AP) e **quantidade de túneis/sessões** (200/500/ilimitado) — não recursos (1).
- CHR licencia por **velocidade de interface**: **free** (1 Mbps) para laboratório, `p1/p10/p-unlimited` para produção, trial de 60 dias (Tabela 3.2).
- **RouterBOOT** é o bootloader das RouterBOARD: atualize-o junto com o RouterOS (`/system routerboard upgrade + reboot`).
- Proteções de boot (`protected-routerboot`) defendem equipamento exposto — ao custo de eliminar os caminhos de recuperação. Use com critério e documentação.

No próximo capítulo, deixamos a caixa e vamos às portas de entrada: WinBox, WebFig, SSH, serial e MAC-Telnet — todas as formas de acessar um RouterOS, e quando usar cada uma.

Bibliografia do capítulo

A página oficial de licenciamento do RouterOS e a documentação do CHR (MikroTik, 2026b) detalham níveis e preços atuais; a documentação do RouterOS (MikroTik, 2026a) cobre os menus `/system license` e `/system routerboard` (incluindo o `protected-routerboot`). O catálogo (MikroTik, 2026c) informa o nível de licença de cada produto na ficha técnica. As referências completas, em ABNT, estão no apêndice [Referências](#).

4 Formas de acesso ao RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Listar as formas de acesso ao RouterOS — WinBox, WebFig, SSH, Telnet, console serial e MAC-Telnet — e escolher a adequada a cada situação;
- Explicar a diferença entre acesso **por IP** (camada 3) e acesso **por MAC** (camada 2), e por que o segundo salva o dia quando o IP está errado;
- Usar o **Safe Mode**, o cinto de segurança que desfaz alterações quando você perde o próprio acesso;
- Acessar seu CHR de laboratório pelo WinBox, pelo WebFig e por SSH;
- Reconhecer os serviços de acesso em `/ip service` e entender por que eles serão restringidos no capítulo de segurança.

4.1 Muitas portas para a mesma casa

Tudo o que fizemos até aqui usou o **console da máquina virtual** — o equivalente a sentar na frente do equipamento com teclado e monitor. Na vida real de provedor, o roteador está num rack, num poste ou numa torre a 40 km, e você precisa entrar nele pela rede. O RouterOS oferece várias portas de entrada, e a escolha certa depende de duas perguntas:

1. **O equipamento tem IP acessível?** Se sim, qualquer forma serve; se não (equipamento novo, configuração quebrada), você precisa das formas que funcionam em **camada 2**, direto pelo cabo.
2. **Você quer interface gráfica ou linha de comando?** A gráfica é confortável para explorar; a linha de comando é precisa, documentável e automatizável — e é o padrão deste manual.

A Figura 4.1 organiza as opções nesses dois eixos:

Vamos conhecer uma a uma, começando pelas que você mais usará.

4.1.1 WinBox — a ferramenta símbolo

O **WinBox** é o aplicativo gráfico de gerência da MikroTik — tão associado à marca que muita gente “configura pelo WinBox” sem saber o nome do sistema que está configurando.

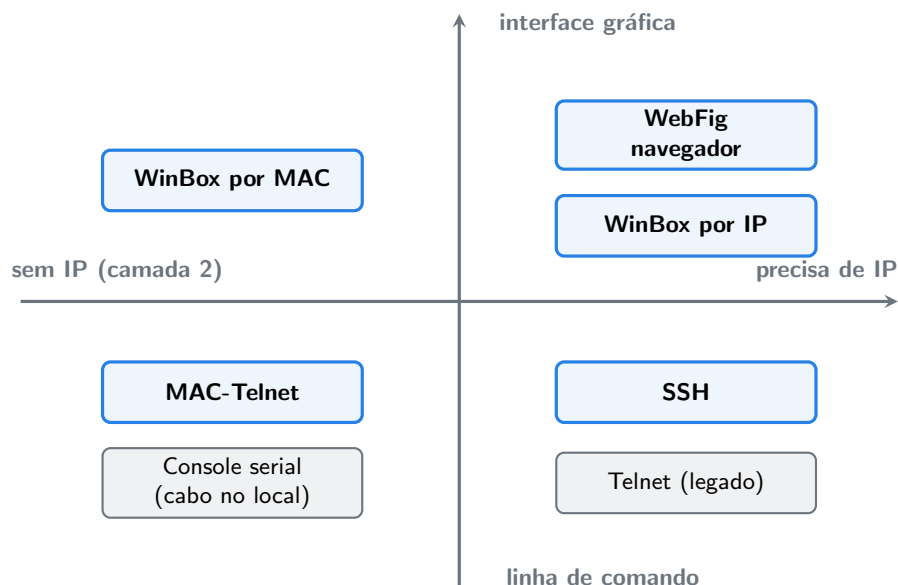


Figura 4.1: As portas de entrada do RouterOS, organizadas por tipo de interface e por dependência de IP.

Ele conversa com o roteador pela porta **8291/TCP** e hoje tem versão nativa para Windows, macOS e Linux.

Três características o tornam a ferramenta de campo por excelência:

- **Descoberta de vizinhos:** a tela inicial lista os equipamentos MikroTik alcançáveis no cabo (via protocolo de descoberta), mesmo os sem IP;
- **Conexão por MAC:** dê um clique no endereço MAC da lista e você entra num equipamento que não tem IP nenhum — mágica de camada 2 que veremos adiante;
- **Espelho da CLI:** cada janela do WinBox corresponde a um menu do terminal (IP → Addresses é `/ip address`). Aprender um é aprender o outro — e é por isso que este manual pode ensinar por linha de comando sem abandonar quem prefere clicar.

4.1.2 WebFig — o WinBox no navegador

O **WebFig** é a interface web embutida: aponte o navegador para o IP do roteador (porta **80/HTTP** ou **443/HTTPS**) e você tem praticamente as mesmas telas do WinBox, sem instalar nada. É a saída natural quando você está numa máquina emprestada ou num sistema onde não pode instalar aplicativos. Nos equipamentos com configuração de fábrica, é o WebFig que recebe o usuário no modo simplificado “Quick Set”.

4.1.3 SSH — a porta da linha de comando

O **SSH** (porta **22/TCP**) entrega o mesmo terminal que você usou no console da VM, com criptografia, a partir de qualquer sistema operacional:

```
ssh admin@192.168.88.1
```

É a forma preferida deste manual para o dia a dia, por três motivos: é universal (todo sistema tem cliente SSH), é **automatizável** (scripts, Ansible e afins entram por aqui — Volume 12) e registra bem o que foi feito (seu terminal guarda o histórico da sessão). O RouterOS também aceita **SFTP/SCP** na mesma porta, para copiar arquivos — algo que usaremos nos backups (3).

O primo velho do SSH, o **Telnet** (porta 23), também existe — sem criptografia alguma. Trate-o como peça de museu: útil apenas em redes de laboratório isoladas, e um dos primeiros serviços que desligaremos no capítulo de segurança.

4.1.4 Console serial — o cabo da UTI

Roteadores de rack (CCR, alguns CRS) trazem uma porta **serial** (RJ45 ou USB). Ligando um cabo console, você acessa o terminal **antes mesmo do RouterOS subir** — vê o boot, o RouterBOOT (Capítulo 3) e o sistema, mesmo com a rede completamente quebrada. É o acesso de UTI: usado raramente, mas insubstituível quando o equipamento não responde por nenhuma outra via e você está fisicamente diante dele.

4.1.5 MAC-Telnet — terminal sem IP

E se o equipamento não tem IP, você não tem o WinBox à mão, mas há **outro MikroTik** no mesmo cabo? O **MAC-Telnet** resolve: de um RouterOS para outro, abre-se um terminal usando só o endereço MAC, em camada 2:

```
/tool mac-telnet 48:A9:8A:12:34:56
```

O RouterOS descobre os vizinhos MikroTik (e seus MACs) sozinho:

```
/ip neighbor print
```

Esse par — descoberta de vizinhos + MAC-Telnet — é o kit de resgate do provedor: o técnico conecta o notebook (ou usa o roteador de cima da torre) e entra no rádio “perdido” sem depender de IP certo. A contrapartida de segurança é evidente: qualquer um no cabo pode tentar o mesmo. Como restringir isso é assunto do 5.

4.2 Safe Mode: o cinto de segurança

Antes do laboratório, a ferramenta mais importante do capítulo. Imagine: você está acessando um rádio de torre **pela própria rede que ele roteia** e desativa, sem querer, a interface errada. A conexão cai — e com ela, seu acesso e o dos clientes. Viagem de 40 km.

O **Safe Mode** existe para impedir exatamente isso. Ao ativá-lo (tecla **Ctrl+X** no terminal; botão *Safe Mode* no WinBox/WebFig), o RouterOS passa a tratar suas alterações como **provisórias**:

- Se você **sair do Safe Mode normalmente** (**Ctrl+X** de novo), as mudanças são confirmadas;
- Se a **conexão cair** com o Safe Mode ativo — inclusive porque sua mudança derrubou o próprio acesso —, o RouterOS **desfaz tudo** o que foi feito desde a ativação, e alguns segundos depois você entra de novo como se nada tivesse acontecido.

No terminal, o modo ativo aparece no prompt:

```
[admin@lab-r1] <SAFE>
```

! O conceito mais importante do capítulo

Toda alteração remota que possa afetar o seu próprio caminho até o equipamento — interfaces, IPs, rotas, firewall — deve ser feita **dentro do Safe Mode**. **Ctrl+X** antes, mexa, confirme com **Ctrl+X** depois. Esse hábito, sozinho, paga o livro: ele elimina a viagem até a torre.

4.3 Os serviços em /ip service

Cada forma de acesso por IP corresponde a um **serviço** que o RouterOS mantém ouvindo, listado em `/ip service`:

Tabela 4.1: Serviços de acesso do RouterOS

Serviço	Porta padrão	Forma de acesso
winbox	8291/TCP	WinBox
www	80/TCP	WebFig (HTTP)
www-ssl	443/TCP	WebFig (HTTPS)
ssh	22/TCP	SSH / SFTP
telnet	23/TCP	Telnet (legado)
ftp	21/TCP	FTP (legado)
api, api-ssl	8728, 8729/TCP	API de automação (Volume 12)

Guarde a existência dessa tabela: no 5 ela vira uma lista de decisões — o que desabilitar, o que restringir por endereço de origem (`allowed-address`) e o que mover de porta. Um roteador de provedor exposto à internet com todos esses serviços abertos é convite para invasão.

4.4 Laboratório

4.4.1 Ambiente

A VM do CHR no VirtualBox — agora com um ajuste de rede que permitirá ao **seu computador** conversar com ela. Com a VM **desligada** (`/system shutdown`):

1. No VirtualBox, abra as **configurações da VM** → **Rede**;
2. No **Adaptador 1**, troque o modo para **Placa de rede exclusiva de hospedeiro** (*host-only adapter*), selecionando a rede padrão (VirtualBox Host-Only Ethernet Adapter, tipicamente 192.168.56.0/24);
3. Inicie a VM.

Assim, o CHR (`ether1`) e o seu computador ficam ligados num “cabo virtual” — exatamente o cenário de um técnico com notebook conectado ao equipamento.

4.4.2 Comandos passo a passo

No **console da VM**, dê um endereço IP à `ether1` (a sintaxe de endereços é assunto do Volume 2 — por ora, digite e observe):

```
/ip address add address=192.168.56.10/24 interface=ether1
```

Confirme:

```
/ip address print
```

```
[admin@lab-r1] > /ip address print
Columns: ADDRESS, NETWORK, INTERFACE
# ADDRESS          NETWORK          INTERFACE
0 192.168.56.10/24  192.168.56.0    ether1
```

Acesso 1 — WebFig. No navegador do seu computador, abra `http://192.168.56.10`. Entre com `admin` e a senha que você definiu no primeiro boot. Explore: localize em **IP** → **Addresses** o endereço que você acabou de criar pelo terminal.

Acesso 2 — SSH. No terminal do seu computador (PowerShell, Terminal do macOS ou shell Linux — todos têm o cliente):

```
ssh admin@192.168.56.10
```

Aceite a chave do servidor (**yes**), informe a senha e observe: é o mesmo prompt `[admin@lab-r1] >` do console. Execute `/system resource print` para provar que tudo que você aprendeu vale aqui.

Acesso 3 — WinBox. Baixe o WinBox em mikrotik.com/download (seção *WinBox*). Ao abrir, a aba **Neighbors** deve listar seu CHR. Conecte duas vezes: primeiro clicando no **IP**, depois clicando no **endereço MAC** — esta segunda é uma conexão de camada 2, que funcionaria mesmo sem o endereço IP configurado.

Acesso 4 — Safe Mode (o exercício mais importante). Na sessão SSH, pressione **Ctrl+X**. O prompt ganha o marcador `<SAFE>`. Agora derrube o próprio acesso de propósito, removendo o endereço IP:

```
/ip address remove numbers=0
```

Sua sessão SSH congela e cai — você “se trancou para fora”. Espere cerca de 15 segundos e conecte de novo por SSH: funciona, porque o Safe Mode detectou a queda da sessão e **desfez a remoção**. Confirme com `/ip address print` que o endereço voltou.

4.4.3 Verificação

1. WebFig abre em `http://192.168.56.10` e mostra o endereço em IP → Addresses;
2. SSH conecta e entrega o prompt `[admin@lab-r1] >`;
3. O WinBox lista o CHR em Neighbors e conecta por IP e por MAC;
4. Após o teste do Safe Mode, `/ip address print` mostra o item 0 restaurado — a prova de que a reversão automática funcionou.

4.4.4 Desafios

1. Liste os serviços ativos do seu CHR com `/ip service print`. Quais das quatro conexões deste laboratório passaram por qual serviço?
2. Ainda em `/ip service`: descubra (com TAB e ?) como **desabilitar** o serviço `telnet` — e desabilite-o. Por que essa é uma boa ideia mesmo em laboratório?
3. Repita a remoção do endereço IP **sem** Safe Mode. O que muda? Como você recupera o acesso desta vez? (Dica: você tem o console da VM — e o WinBox por MAC.)
4. Em produção, um técnico precisa configurar um rádio novo, sem IP, no topo da torre, a partir do POP lá embaixo (há um MikroTik no POP, no mesmo cabo). Descreva o caminho de acesso completo, citando as ferramentas deste capítulo.

💡 Solução dos desafios

1. `/ip service print` lista a Tabela 4.1. WebFig usou `www` (porta 80), SSH usou `ssh` (22), o WinBox por IP usou `winbox` (8291) — e o WinBox **por MAC** não passou por nenhum serviço IP: é camada 2.

2.

```
/ip service disable telnet
/ip service print
```

O Telnet transmite tudo — inclusive a senha — em texto puro. Desabilitar o que não se usa reduz a superfície de ataque; o hábito vale até em laboratório, para virar reflexo.

3. Sem Safe Mode, a remoção é **definitiva**: o SSH cai e não volta, pois o endereço se foi de verdade. A recuperação: entrar pelo **console da VM** (ou WinBox pelo **MAC**) e recriar o endereço com `/ip address add address=192.168.56.10/24 interface=ether1`. É a experiência que ensina o valor do `Ctrl+X`.

4. No MikroTik do POP: `/ip neighbor print` para descobrir o MAC do rádio novo → `/tool mac-telnet <MAC>` para abrir o terminal em camada 2 → dentro da sessão, configurar IP de gerência no rádio → a partir daí, seguir por SSH/WinBox normalmente (e, sendo mudança arriscada em rádio de torre, tudo dentro do Safe Mode).

4.5 Resumo

- Formas de acesso por **IP**: WinBox (8291), WebFig (80/443), SSH (22) — e Telnet/FTP como legados a desativar (Tabela 4.1).
- Formas de acesso por **camada 2** (sem IP): WinBox por MAC e MAC-Telnet (entre RouterOS), com descoberta de vizinhos por `/ip neighbor print`; console serial como acesso físico de emergência (Figura 4.1).
- **WinBox espelha a CLI**: cada janela é um menu do terminal — aprender um é aprender os dois.
- **Safe Mode (Ctrl+X)**: alterações viram provisórias e são desfeitas se a sua sessão cair — obrigatório em qualquer mudança remota que possa derrubar o próprio acesso.
- Cada acesso por IP é um **serviço** em `/ip service`, que será restringido no capítulo de segurança inicial.

No próximo capítulo, montamos de uma vez o laboratório definitivo do livro: múltiplos CHRs em topologia, com acesso confortável do seu computador — o ambiente que reutilizaremos até a última página.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) descreve cada ferramenta nas seções *WinBox*, *WebFig* e *MAC server* (MAC-Telnet), além do menu `/ip service`. O download do WinBox

e do cliente para cada plataforma está em mikrotik.com/download. Discher (2016) dedica bons capítulos iniciais às interfaces de gerência. As referências completas, em ABNT, estão no apêndice [Referências](#).

5 Montando o laboratório com CHR

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Montar, no VirtualBox, o **laboratório padrão do livro**: três CHRs (**lab-r1**, **lab-r2**, **lab-r3**) com rede de gerência fixa e redes internas para as topologias;
- Explicar o papel de cada tipo de rede do VirtualBox no laboratório — *host-only* (gerência), *rede interna* (enlaces entre roteadores) e *NAT* (saída para a internet);
- Clonar VMs e usar **snapshots** para começar cada laboratório limpo;
- Acessar qualquer roteador do laboratório por SSH e WinBox a partir do seu computador, pela rede de gerência;
- Reconhecer quando vale migrar para GNS3 ou EVE-NG — e por que o VirtualBox basta para este livro.

5.1 Um laboratório para o livro inteiro

Este é o capítulo mais rentável do manual: o que você montar aqui será reutilizado em **todos** os laboratórios até a última página. A seção “Ambiente” dos próximos capítulos dirá apenas coisas como “2 CHRs em rede interna (Capítulo 5)” — e você já saberá exatamente o que ligar.

O desenho atende três exigências que acompanham qualquer laboratório de redes:

1. **Gerência estável**: você precisa alcançar os roteadores por SSH/WinBox do seu computador **sempre**, mesmo quando o exercício quebra a rede entre eles. A solução é uma rede de gerência separada, que os exercícios nunca tocam.
2. **Enlaces isolados**: os “cabos” entre roteadores devem ser redes que só existem entre eles, sem interferência do seu computador ou da sua rede doméstica.
3. **Reset rápido**: errar faz parte; recomeçar do zero não pode custar uma reinstalação. Snapshots resolvem.

A topologia-base fica assim (Figura 5.1):

O mapa fixo de interfaces e endereços — decore ou marque esta página:

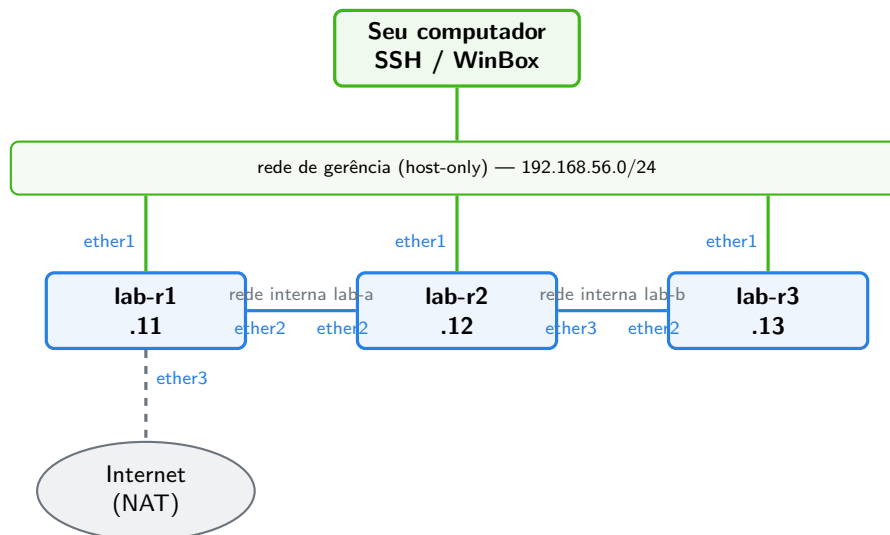


Figura 5.1: A topologia-base do laboratório: gerência host-only fixa (verde), enlaces por redes internas (azul) e saída opcional para a internet via NAT em **lab-r1** (tracejado).

Tabela 5.1: Mapa de interfaces do laboratório padrão

VM	ether1 (gerência)	ether2	ether3
lab-r1	host-only — 192.168.56.11	rede interna lab-a	NAT (internet)
lab-r2	host-only — 192.168.56.12	rede interna lab-a	rede interna lab-b
lab-r3	host-only — 192.168.56.13	rede interna lab-b	—

💡 Analogia para guardar

A rede de gerência é o **corredor de serviço** de um hotel: os hóspedes (o tráfego dos exercícios) circulam pelos corredores normais (**lab-a**, **lab-b**), enquanto a manutenção (você, por SSH) tem um corredor próprio que nunca fecha — mesmo com um quarto em obras.

5.1.1 Os três tipos de rede do VirtualBox

O VirtualBox oferece vários modos de rede; o laboratório usa três, cada um num papel:

- **Host-only** — cria uma rede virtual entre o seu computador e as VMs. Perfeita para gerência: o hospedeiro alcança todas as VMs, e nada disso vaza para sua rede doméstica.

- **Rede interna** (*internal network*) — um “switch virtual” nomeado que só conecta VMs que apontem para o mesmo nome (**lab-a**, **lab-b**). É o cabo entre roteadores: seu computador não participa.
- **NAT** — a VM sai para a internet através do seu computador, como um celular no modo roteador. Usamos apenas na **ether3** do **lab-r1**, nos capítulos que precisam de internet de verdade (atualizações, DNS, NTP).

5.2 GNS3 e EVE-NG: quando crescer

GNS3 e **EVE-NG** são plataformas de emulação de redes completas: desenho de topologia por arrastar-e-soltar, dezenas de nós, imagens de vários fabricantes. Ambas rodam o CHR muito bem, e você encontrará laboratórios MikroTik prontos para elas na internet.

Este livro, porém, padroniza no **VirtualBox** por três razões: é mais leve (roda em qualquer máquina de estudo), não exige imagens adicionais nem servidor dedicado, e três roteadores cobrem quase todos os exercícios — nos poucos capítulos que pedem um quarto nó (o estudo de caso do Volume 15, por exemplo), o padrão de clonagem que você aprenderá aqui resolve em minutos. Se você já domina GNS3/EVE-NG, use-os sem medo: os laboratórios descrevem **topologias**, não cliques — basta reproduzir as redes da Tabela 5.1.

5.3 Laboratório

Este capítulo é um grande laboratório: vamos montar o ambiente definitivo, do zero ao snapshot.

5.3.1 Ambiente

- VirtualBox instalado (o mesmo do Capítulo 1);
- A imagem **.vdi** do CHR baixada de mikrotik.com/download;
- A VM avulsa dos capítulos anteriores pode ser **removida** ao final — o laboratório novo a substitui.

5.3.2 Comandos passo a passo

Parte 1 — preparar a rede host-only. No VirtualBox, menu **Arquivo** → **Ferramentas** → **Gerenciador de Rede** (*Network Manager*): confirme que existe um adaptador host-only com IPv4 **192.168.56.1/24** (é o padrão; crie-o se não existir) e **desative o servidor DHCP** dele — os IPs do laboratório são fixos, e um DHCP intrometido só atrapalha.

Parte 2 — criar a VM-molde. Crie uma VM chamada **lab-r1**, como no primeiro capítulo (Linux / Other Linux 64-bit, 256 MB de RAM), usando uma **cópia** do **.vdi** do CHR como disco. Antes de ligar, em **Configurações** → **Rede**:

1. **Adaptador 1:** habilitado, modo *Placa exclusiva de hospedeiro* (host-only) — será a `ether1`;
2. **Adaptador 2:** habilitado, modo *Rede interna*, nome `lab-a` — será a `ether2`;
3. **Adaptador 3:** habilitado, modo *NAT* — será a `ether3`.

Ligue a VM, faça o primeiro login (`admin`, senha em branco, defina a nova senha) e configure identidade e IP de gerência:

```
/system identity set name=lab-r1
/ip address add address=192.168.56.11/24 interface=ether1 comment=gerencia
```

Do **seu computador**, confirme o acesso antes de continuar:

```
ssh admin@192.168.56.11
```

Parte 3 — clonar. Desligue a VM (`/system shutdown`). No VirtualBox, clique com o botão direito em `lab-r1` → **Clonar**: nome `lab-r2`, tipo **clone completo**, e marque a opção de **gerar novos endereços MAC** para todas as placas (fundamental — MACs duplicados criam defeitos enlouquecedores). Repita para criar `lab-r3`.

Ajuste as redes dos clones conforme a Tabela 5.1:

- `lab-r2`: Adaptador 2 → rede interna `lab-a`; Adaptador 3 → rede interna `lab-b`;
- `lab-r3`: Adaptador 2 → rede interna `lab-b`; Adaptador 3 → **desabilitado**.

Ligue `lab-r2`, e pelo **console** corrija identidade e IP (o clone veio com os dados do molde):

```
/system identity set name=lab-r2
/ip address set numbers=0 address=192.168.56.12/24
```

O mesmo em `lab-r3`:

```
/system identity set name=lab-r3
/ip address set numbers=0 address=192.168.56.13/24
```

Parte 4 — testar os enlaces internos. Com as três VMs ligadas, dê endereços provisórios ao enlace `lab-a` para prová-lo. Em `lab-r1`:

```
/ip address add address=10.0.12.1/30 interface=ether2 comment=teste-lab-a
```

Em `lab-r2`:

```
/ip address add address=10.0.12.2/30 interface=ether2 comment=teste-lab-a
```

E, ainda de `lab-r2`, pingue o vizinho:


```
/ping 10.0.12.1 count=4
```

```
[admin@lab-r2] > /ping 10.0.12.1 count=4
SEQ HOST      SIZE TTL TIME      STATUS
  0 10.0.12.1   56  64 1ms234us
  1 10.0.12.1   56  64 987us
  2 10.0.12.1   56  64 1ms102us
  3 10.0.12.1   56  64 995us
sent=4 received=4 packet-loss=0%
```

`packet-loss=0%` prova o enlace. Remova os endereços de teste nos dois roteadores — os capítulos seguintes sempre começam com os enlaces “crus”:

```
/ip address remove [find comment=teste-lab-a]
```

Parte 5 — snapshot. Desligue as três VMs (`/system shutdown`). No VirtualBox, selecione cada uma → **Snapshots** → **Criar** — nomeie `base-limpa`. Este é o estado oficial de partida: identidade + IP de gerência, e nada mais.

5.3.3 Verificação

O checklist de “laboratório pronto”:

1. As três VMs existem, com os adaptadores exatamente como na Tabela 5.1;
2. Do seu computador, `ssh admin@192.168.56.11`, `.12` e `.13` entram nos três roteadores (com as VMs ligadas), e o prompt de cada um mostra a identidade certa;
3. O WinBox lista os três em **Neighbors** (Capítulo 4);
4. O ping da Parte 4 fechou com `packet-loss=0%` e os endereços de teste foram removidos;
5. Cada VM tem o snapshot `base-limpa`.

A partir de agora, “restaurar o laboratório” = desligar as VMs e restaurar o snapshot `base-limpa` de cada uma. Faça isso sempre que um capítulo pedir ambiente limpo — ou quando um experimento sair do controle.

5.3.4 Desafios

1. Sem olhar a tabela: qual rede interna conecta `lab-r2` a `lab-r3`, e por quais interfaces de cada um?
2. De `lab-r3`, encontre os vizinhos MikroTik com `/ip neighbor print`. Quem aparece, e por quê `lab-r1` (não) aparece?
3. O enlace `lab-b` ainda não foi testado. Repita a Parte 4 nele — desta vez escolhendo você mesmo a sub-rede de teste — e desfaça ao final.

4. Habilite temporariamente o adaptador NAT (**ether3**) do **lab-r1** e, no roteador, rode `/ip dhcp-client add interface=ether3` seguido de `/ip dhcp-client print`. O que aconteceu? (É um gostinho do Volume 2 — desfaça com `/ip dhcp-client remove numbers=0` ao terminar.)

💡 Solução dos desafios

1. Rede interna **lab-b**, entre a **ether3** de **lab-r2** e a **ether2** de **lab-r3** (Tabela 5.1).
2. Aparecem **lab-r2** (vizinho direto pela **lab-b**) e — dependendo do modo do seu adaptador **host-only** — os demais pela rede de gerência, já que ela é um segmento comum a todos. Pela rede **interna**, **lab-r1** não é vizinho de **lab-r3**: eles não compartilham segmento (**lab-a** e **lab-b** são switches separados, e **lab-r2** está no meio).
3. Exemplo com **10.0.23.0/30**: em **lab-r2**, `/ip address add address=10.0.23.1/30 interface=ether3`; em **lab-r3**, `/ip address add address=10.0.23.2/30 interface=ether2`; ping de um para o outro; remoção com `/ip address remove [find comment=...]` (se você usou comentário — bom hábito) ou pelo número do item.
4. O DHCP client pediu endereço ao NAT do VirtualBox e recebeu algo como **10.0.2.15/24** com gateway **10.0.2.2** — o roteador ganhou uma “WAN” e já alcança a internet (`/ping 8.8.8.8` funciona). É exatamente assim que o capítulo de DHCP montará a WAN do roteador de borda.

5.4 Resumo

- O laboratório padrão: **3 CHR**s no VirtualBox, gerência **host-only** fixa (**192.168.56.11/.12/.13** na **ether1**), enlaces por **redes internas lab-a** e **lab-b**, NAT opcional na **ether3** de **lab-r1** (Figura 5.1, Tabela 5.1).
- Gerência separada dos exercícios = acesso que **nunca cai**, aconteça o que acontecer com a topologia.
- Clones completos com **MACs novos**; identidade e IP ajustados no console logo após clonar.
- Snapshot **base-limpa** em cada VM: restaurá-lo é o “reset de fábrica” do laboratório.
- GNS3/EVE-NG são upgrades válidos; os laboratórios do livro descrevem topologias e funcionam em qualquer plataforma.

No próximo capítulo, usamos o laboratório novo para o assunto que abre toda operação real: o primeiro acesso a um equipamento, a configuração de fábrica e a segurança inicial — usuários, senhas e serviços.

Bibliografia do capítulo

A documentação do CHR (MikroTik, 2026b) cobre instalação em VirtualBox, VMware, Proxmox e nuvens públicas; o manual do VirtualBox (em [virtualbox.org/manual](https://www.virtualbox.org/manual)) detalha

os modos de rede que usamos. Para laboratórios em GNS3/EVE-NG, a documentação de cada plataforma traz guias de importação do CHR. As referências completas, em ABNT, estão no apêndice [Referências](#).

6 Primeiro acesso, usuários e segurança inicial

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Descrever o que um equipamento MikroTik traz de fábrica — a *default configuration* — e decidir entre aproveitá-la ou partir do zero;
- Criar usuários com o **grupo** certo (**read**, **write**, **full**) e abolir o uso compartilhado do **admin**;
- Aplicar o checklist mínimo de segurança inicial: senha forte, serviços desnecessários desligados, acesso restrito por **allowed-address**, descoberta de vizinhos contida e MAC-server sob controle;
- Ajustar relógio e fuso (**/system clock**, NTP) — e explicar por que hora certa é item de **segurança**;
- Deixar qualquer equipamento novo em estado “apresentável” em cinco minutos, seguindo sempre a mesma sequência.

6.1 O que vem de fábrica

Ligue um hAP novo e ele já funciona: distribui IPs na LAN, faz NAT para a porta WAN, tem firewall básico e WiFi com senha na etiqueta. Essa é a **configuração padrão** (*default configuration*), pensada para o roteador doméstico — e ela é boa nesse papel.

No mundo do provedor, porém, a configuração de fábrica é quase sempre **ponto de partida errado**: um rádio de torre não precisa de servidor DHCP; um concentrador não deve ter NAT de fábrica; um switch de POP não quer firewall nas portas. Por isso o costume profissional é começar **limpo** e construir só o que o papel do equipamento pede. Há duas formas de chegar ao estado limpo:

- No primeiro boot de um equipamento físico, o RouterOS mostra a configuração padrão e oferece a tecla **r** para removê-la já no primeiro login;
- A qualquer momento: **/system reset-configuration no-defaults=yes** — que veremos, com os devidos avisos, no 3.

O **CHR nasce limpo** — sem default configuration —, e é por isso que nossos laboratórios nunca precisaram removê-la. O que o CHR *tem*, como todo RouterOS, é o usuário **admin** — e é nele que a segurança inicial começa.

6.2 Usuários e grupos: aposentando o admin compartilhado

Todo RouterOS nasce com o usuário `admin`, do grupo `full`. Numa operação com três técnicos, é tentador todo mundo usar “o admin” — até o dia em que uma configuração some às 3h da manhã e o `/system history print` (Capítulo 3) diz apenas: foi o `admin`. Quem?

A regra profissional: **uma pessoa, um usuário** — e cada usuário com o menor privilégio que cumpre sua função. Os privilégios vêm dos **grupos**, em `/user group`:

Tabela 6.1: Grupos padrão de usuários

Grupo	Pode	Uso típico
<code>read</code>	ver tudo, mudar nada	monitoramento, auditoria, NOC nível 1
<code>write</code>	configurar (exceto gerenciar usuários)	técnicos de campo
<code>full</code>	tudo, inclusive usuários e reboot	administradores

Os comandos essenciais:

```
/user add name=vitor group=full password=SenhaForte!2026 comment="Vitor - admin"
/user add name=noc group=read password=OutraSenha!2026 comment="Monitoramento"
/user print
```

Criado o seu usuário nominal e testado o login com ele, desabilite o `admin` de fábrica — todo scanner de internet conhece esse nome:

```
/user disable admin
```

Sobre as senhas, o essencial cabe em três linhas: longas (frases servem), únicas por equipamento ou por parque (gerenciador de senhas é ferramenta de provedor, não luxo) e **nunca** a mesma do Wi-Fi da sua casa. O RouterOS não impõe complexidade — a disciplina é sua.

Analogia para guardar

Usuário compartilhado é caixa de supermercado sem operador identificado: qualquer diferença no caixa, ninguém foi. Grupos são os cargos — o repositor (`read`) não abre a gaveta; o gerente (`full`) assina a sangria. O `/system history` só vira auditoria quando cada login tem dono.

6.3 Fechando as portas: serviços e acesso

No Capítulo 4 você viu a Tabela 4.1: cada forma de acesso por IP é um serviço em `/ip service`. Segurança inicial é decidir o destino de cada um. A política deste manual, que aplicaremos no laboratório:

1. **Desabilitar o que não se usa:** `telnet` e `ftp` sempre; `www` (HTTP puro) e `api/api-ssl` até que algum uso real os exija;
2. **Restringir o que fica:** os serviços sobreviventes (`ssh`, `winbox`, `www-ssl`) ganham `allowed-address` — uma lista de redes de onde aceitam conexão. Tipicamente, a rede de **gerência** do provedor;
3. **Registrar a decisão:** o `comment` nos itens críticos explica o porquê para o próximo técnico (que pode ser você, daqui a dois anos).

O mesmo raciocínio vale para os protocolos de **camada 2**, que não passam por `/ip service`:

- **Descoberta de vizinhos** (`/ip neighbor discovery-settings`): anuncia o equipamento no cabo. Ótimo na gerência, péssimo nas portas voltadas a clientes — um assinante curioso não precisa ver modelo e versão do seu concentrador;
- **MAC-server** (`/tool mac-server`): controla quem pode abrir MAC-Telnet e WinBox-por-MAC. O kit de resgate do técnico é também o kit de invasão de quem alcança o cabo — restrinja-o a uma lista de interfaces de confiança.

Ambos aceitam o mesmo remédio: uma **interface list** (uma lista nomeada de interfaces, ex.: **gerencia**) usada como filtro. Interface lists reaparecerão como protagonistas no firewall (Volume 3).

6.4 Relógio certo: segurança, não estética

`/system clock print` num equipamento recém-ligado mostra 1970 ou um fuso errado. Parece cosmético — até o dia em que você precisa cruzar o log do concentrador com a reclamação de um cliente (“caiu ontem 21h40”) ou atender um ofício judicial pedindo quem usava tal IP em tal **horário** (assunto sério no Brasil — Volume 9 detalha os requisitos legais de guarda de registros). Log com hora errada é log inútil.

A receita completa, que aplicaremos no laboratório: fuso correto (`America/Sao_Paulo...` ou o seu) e **NTP** — o protocolo que mantém o relógio sincronizado pela rede — apontando para servidores confiáveis; no Brasil, os do projeto `ntp.br`.

! O conceito mais importante do capítulo

Segurança inicial não é um produto que se instala; é uma **sequência que se repete** em todo equipamento que entra no parque: senha forte no usuário nominal → `admin` desabilitado → serviços não usados desligados → acesso restrito à gerência → descoberta

e MAC-server contidos → relógio certo com NTP. Cinco minutos por equipamento; anos de dores de cabeça evitadas.

6.5 Laboratório

6.5.1 Ambiente

O laboratório padrão do Capítulo 5, restaurado no snapshot **base-limpa**. Usaremos apenas **lab-r1** — com o adaptador NAT (**ether3**) habilitado, pois o NTP precisa de internet. Acesse por SSH: `ssh admin@192.168.56.11`.

6.5.2 Comandos passo a passo

Passo 1 — usuário nominal. Crie o seu usuário (troque nome e senha pelos seus), teste-o e desabilite o **admin**:

```
/user add name=vitor group=full password=Lab!Segura2026 comment="dono do lab"
```

Abra **outro** terminal e confirme o login novo antes de fechar o atual — regra de ouro ao mexer em usuários:

```
ssh vitor@192.168.56.11
```

Funcionando, desabilite o usuário de fábrica (na sessão nova):

```
/user disable admin  
/user print
```

```
[vitor@lab-r1] > /user print  
Columns: NAME, GROUP, LAST-LOGGED-IN  
# NAME      GROUP  LAST-LOGGED-IN  
0 ;;; system default user  
  X admin   full   2026-07-06 14:02:11  
1 ;;; dono do lab  
  vitor    full   2026-07-06 14:05:37
```

O X marca o **admin** desabilitado.

Passo 2 — serviços. Desligue o que não usamos e restrinja o resto à rede de gerência:

```
/ip service disable telnet,ftp,www,api,api-ssl
/ip service set ssh address=192.168.56.0/24
/ip service set winbox address=192.168.56.0/24
/ip service set www-ssl address=192.168.56.0/24
/ip service print
```

```
[vitor@lab-r1] > /ip service print
Columns: NAME, PORT, ADDRESS
#  NAME      PORT  ADDRESS
0  X telnet   23
1  X ftp      21
2  X www      80
3  ssh       22   192.168.56.0/24
4  winbox    8291  192.168.56.0/24
5  X api      8728
6  www-ssl   443   192.168.56.0/24
7  X api-ssl  8729
```

Passo 3 — camada 2 contida. Crie a interface list **gerencia**, ponha a **ether1** nela e aponte a descoberta de vizinhos e o MAC-server para a lista:

```
/interface list add name=gerencia comment="interfaces de gerencia do lab"
/interface list member add list=gerencia interface=ether1
/ip neighbor discovery-settings set discover-interface-list=gerencia
/tool mac-server set allowed-interface-list=gerencia
/tool mac-server mac-winbox set allowed-interface-list=gerencia
```

Passo 4 — relógio e NTP. Ajuste o fuso e ative o cliente NTP com os servidores brasileiros (garanta que o DHCP client do desafio do capítulo anterior está ativo na **ether3**, ou crie-o: **/ip dhcp-client add interface=ether3**):

```
/system clock set time-zone-name=America/Sao_Paulo
/system ntp client set enabled=yes
/system ntp client servers add address=a.st1.ntp.br
/system ntp client servers add address=b.st1.ntp.br
```

Aguarde alguns segundos e verifique a sincronização:

```
/system ntp client print
```

```
[vitor@lab-r1] > /system ntp client print
      enabled: yes
      mode: unicast
synced-stratum: 1
synced-server: a.st1.ntp.br
```


synced-server preenchido = relógio sob controle. Confira o resultado final com `/system clock print`.

6.5.3 Verificação

1. `ssh vitor@192.168.56.11` entra; tentar `ssh admin@192.168.56.11` é recusado (usuário desabilitado);
2. `/ip service print` mostra `telnet`, `ftp`, `www`, `api` e `api-ssl` com `X`, e os demais restritos a `192.168.56.0/24`;
3. `/ip neighbor discovery-settings print` e `/tool mac-server print` apontam para a lista `gerencia`;
4. `/system clock print` mostra fuso e hora corretos, e `/system ntp client print` mostra um `synced-server`.

Feche com um snapshot novo em `lab-r1` — nomeie `seguranca-inicial`: vários capítulos seguintes partirão dele.

6.5.4 Desafios

1. Crie um usuário `estagiario` do grupo `read` e, logado com ele, tente `/system identity set name=hacker`. O que acontece? E `/user print`, funciona?
2. Nos grupos padrão não existe “técnico que configura mas não reinicia”. Explore `/user group print` e crie um grupo `campo` com as políticas do `write` **sem** a política `reboot`.
3. Sua sessão SSH vem do `192.168.56.1` (o hospedeiro). Aplique, **dentro do Safe Mode**, um `allowed-address` de propósito errado no serviço `ssh` (ex.: `10.99.99.0/24`), deixe a sessão cair e explique o que o Safe Mode fez por você (Capítulo 4).
4. Monte, em texto, o **checklist de entrada** do seu provedor fictício: os passos deste capítulo na ordem em que você os aplicaria num rádio novo antes de ele subir na torre — e um item extra que este capítulo ainda não cobriu, mas o próximo cobrirá (dica: o que você faz *antes* de mexer numa configuração que funciona?).

Solução dos desafios

1. O `set` falha com erro de permissão (`not enough permissions`): o grupo `read` não tem a política `write`. Já `/user print` **também falha** — ver e gerenciar usuários exige a política `sensitive`/política de usuários, que o `read` não tem. Menor privilégio funcionando.

2.

```
/user group add name=campo policy=local,ssh,read,write,winbox,web,!reboot,!policy,!sensi
```

O essencial: partir das políticas do `write` e negar `reboot` (a sintaxe `!politica` nega). Confira com `/user group print` e teste com um usuário de mentira.

3. Com `Ctrl+X` ativo, `/ip service set ssh address=10.99.99.0/24` derruba sua sessão (você não está nessa rede). O Safe Mode detecta a queda e **reverte** o `allowed-`

`address` — reconecte e confirme com `/ip service print`. Sem Safe Mode, seria console da VM para consertar.

4. Sequência razoável: (1) usuário nominal + senha do gerenciador; (2) `admin` desabilitado; (3) serviços: só `ssh+winbox`, com `allowed-address` da gerência; (4) `discovery` e `MAC-server` na lista `gerencia`; (5) fuso + NTP; (6) identidade com o padrão de nomes do parque (ex.: `torre-sul-ntp1`). O item que falta: **backup/export da configuração** antes e depois — assunto do próximo capítulo.

6.6 Resumo

- Equipamentos físicos trazem **default configuration** boa para uso doméstico; no provedor, parte-se limpo (tecla `r` no primeiro login ou reset `no-defaults=yes`). O CHR já nasce limpo.
- **Um usuário por pessoa**, no grupo de menor privilégio (Tabela 6.1); `admin` desabilitado após criar o usuário nominal — e teste o login novo **antes** de fechar a sessão antiga.
- Serviços: desabilite `telnet`, `ftp`, `www`, `api`; restrinja `ssh`, `winbox` e `www-ssl` com `allowed-address` da gerência.
- Camada 2 também se fecha: `discovery-settings` e `mac-server` apontando para uma **interface list** de gerência.
- Fuso + NTP (`ntp.br`) porque log sem hora certa não serve nem ao troubleshooting nem à Justiça.
- Tudo isso é uma sequência repetível de ~5 minutos — o “ritual de entrada” de equipamento novo no parque.

No próximo capítulo, a rede de segurança que faltou ao seu checklist: backup binário, export de configuração, restauração — e o reset, com os avisos que ele merece.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre os menus `/user`, `/ip service`, `/ip neighbor` e `/system ntp`, além da página *Securing your router*, que inspira o checklist deste capítulo. O projeto NTP.br (`ntp.br`) documenta os servidores públicos brasileiros de hora legal. Discher (2016) traz um capítulo dedicado a usuários e segurança de acesso. As referências completas, em ABNT, estão no apêndice [Referências](#).

7 Backup, export e reset

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar a diferença entre o **backup binário** (`/system backup`) e o **export de configuração** (`/export`) — e por que um provedor precisa dos dois;
- Gerar, baixar para fora do equipamento e restaurar cada tipo;
- Ler um export com olhos críticos: o que ele contém, o que omite e como usá-lo para documentar e comparar configurações;
- Resetar um equipamento com `/system reset-configuration`, escolhendo as opções certas (`no-defaults`, `keep-users`, `run-after-reset`);
- Definir a rotina de backup de um parque: quando gerar, onde guardar, como testar.

7.1 Duas fotografias diferentes da mesma configuração

Tudo o que você configura num RouterOS vive numa base de dados interna. O sistema oferece duas maneiras de fotografá-la, e a diferença entre elas é o coração deste capítulo:

- O **backup binário** é um *clone* da base: um arquivo `.backup` que captura tudo — endereços, usuários, senhas, certificados, tudo — num formato que só o RouterOS lê. Restaurá-lo devolve o equipamento exatamente àquele estado, **inclusive identidade e MACs registrados**.
- O **export** é a *receita*: um arquivo de texto `.rsc` com os comandos que reconstroem a configuração — os mesmos comandos que você digitaria. Você pode lê-lo, editá-lo, versioná-lo no Git, aplicar um pedaço em outro equipamento.

Tabela 7.1: Backup binário × export

	Backup binário	Export
Formato	binário (<code>.backup</code>)	texto com comandos (<code>.rsc</code>)
Contém senhas de usuários	sim	não
Contém certificados/chaves	sim	não
Legível/editável	não	sim
Restauração	total, no mesmo equipamento	total ou parcial, em qualquer equipamento

	Backup binário	Export
Uso típico	desastre: voltar o equipamento como era	documentação, migração, auditoria, versionamento

A conclusão prática vem antes da técnica: **um não substitui o outro**. O backup binário é o seguro contra desastre do equipamento X; o export é a documentação viva da configuração — que sobrevive até à morte do hardware, porque pode ser aplicada num substituto de modelo diferente.

💡 Analogia para guardar

O backup binário é a **imagem do disco**; o export é o **código-fonte**. Com a imagem você restaura a máquina idêntica; com o código-fonte você entende, adapta e reconstrói em qualquer máquina. Todo projeto sério guarda os dois.

7.2 Backup binário: gerar, baixar, restaurar

Gerar é um comando (o nome é opcional — sem ele, o RouterOS usa a identidade e a data):

```
/system backup save name=lab-r1-2026-07-06
```

O arquivo aparece no armazenamento interno, visível em `/file print`. E aqui entra a regra número um dos backups: **backup que mora só dentro do equipamento não é backup** — se o hardware morrer, ele morre junto. Baixe para fora. Como o SSH do RouterOS aceita SFTP/SCP (Capítulo 4), do seu computador:

```
scp vitor@192.168.56.11:lab-r1-2026-07-06.backup .
```

Restaurar exige o caminho inverso (subir o arquivo por SCP, se não estiver mais lá) e:

```
/system backup load name=lab-r1-2026-07-06
```

O equipamento pede confirmação e **reinicia** aplicando o estado salvo.

Dois cuidados de quem opera parque:

- O `.backup` carrega **senhas e chaves**: trate o arquivo como material sensível (armazenamento com acesso controlado — não no grupo de WhatsApp da equipe);
- Por padrão o backup é criptografado com a senha do usuário; aceite o hábito de conferir se o arquivo restaura **antes** de precisar dele (o laboratório fará isso).

7.3 Export: a configuração como texto

O `/export` imprime os comandos que reconstroem a configuração — do sistema inteiro ou do menu onde você está:

```
/export file=lab-r1-full
/ip service export file=lab-r1-servicos
```

O primeiro gera `lab-r1-full.rsc` com tudo; o segundo, só a seção `/ip service`. Abra um `.rsc` e você encontra exatamente o que digitaria:

```
# 2026-07-06 14:32:10 by RouterOS 7.15.3
# software id = ...
/ip service
set telnet disabled=yes
set ftp disabled=yes
set www disabled=yes
set ssh address=192.168.56.0/24
```

Três detalhes para ler exports como gente grande:

- Por padrão, o export omite o que está com valor de fábrica. A variante `/export verbose` mostra **tudo** — útil para caçar diferenças sutis;
- Desde o RouterOS v7, `/export show-sensitive` é quem exhibe segredos como chaves de PPP (sem essa opção eles saem mascarados); ainda assim, **senhas de usuários nunca saem** num export;
- O export é a base do “diff de configuração”: exporte antes e depois de uma mudança e compare os arquivos — você enxerga exatamente o que mudou. É o complemento perfeito do `/system history`.

Para **aplicar** um `.rsc` (inteiro ou um trecho), o comando é `/import`:

```
/import file-name=lab-r1-servicos.rsc
```

O import executa linha a linha, como se você digitasse — inclusive parando no meio se uma linha falhar. Por isso, exports de seções pequenas e bem escolhidas são mais úteis que um `.rsc` gigante na hora de migrar pedaços de configuração entre equipamentos.

7.4 Reset: recomeçar sem sustos

O comando que zera a configuração é:

```
/system reset-configuration
```

E as opções que o transformam de “botão de pânico” em ferramenta de precisão:

- **no-defaults=yes** — não aplica a configuração de fábrica após o reset: o equipamento volta **vazio**, o ponto de partida profissional (5);
- **keep-users=yes** — preserva os usuários e senhas;
- **skip-backup=yes** — por padrão o RouterOS salva um backup automático antes de resetar; esta opção pula isso (raramente uma boa ideia);
- **run-after-reset=arquivo.rsc** — a joia da coroa: aplica um **.rsc** logo após o reset. Reset + **run-after-reset** = **provisionamento**: o equipamento renasce já com a configuração-base do seu provedor.

 Aviso

`/system reset-configuration` **apaga toda a configuração** e reinicia o equipamento. Remotamente, isso significa perder o acesso — o equipamento volta ao padrão de fábrica (ou vazio, com **no-defaults=yes**, sem nenhum IP). Antes de resetar qualquer coisa: backup binário e export baixados para fora, e certeza de que você tem um caminho de acesso pós-reset (console físico, WinBox por MAC no cabo, ou o **run-after-reset** bem testado).

 O conceito mais importante do capítulo

A pergunta de operação não é “tenho backup?”, é “**já testei restaurar?**”. Backup não testado é esperança, não backup. A rotina completa: gerar os dois formatos → baixar para fora do equipamento → testar a restauração de tempos em tempos (o laboratório de hoje é o ensaio geral).

7.5 Laboratório

7.5.1 Ambiente

O laboratório padrão (Capítulo 5), usando **lab-r1** a partir do snapshot **seguranca-inicial** do capítulo anterior — assim temos uma configuração de verdade para salvar, destruir e ressuscitar. Acesso: `ssh vitor@192.168.56.11`.

7.5.2 Comandos passo a passo

Passo 1 — fotografar dos dois jeitos. Gere backup binário e export completo:

```
/system backup save name=lab-r1-cap07
/export file=lab-r1-cap07
/file print
```

```
[vitor@lab-r1] > /file print
Columns: NAME, TYPE, SIZE
# NAME                                TYPE      SIZE
0 lab-r1-cap07.backup                 backup    24.3KiB
1 lab-r1-cap07.rsc                    script     1.8KiB
```

Passo 2 — baixar para fora. No terminal do seu computador:

```
scp vitor@192.168.56.11:lab-r1-cap07.backup .
scp vitor@192.168.56.11:lab-r1-cap07.rsc .
```

Abra o `.rsc` num editor de texto e reconheça o capítulo anterior nele: os serviços desabilitados, o `allowed-address`, o NTP. Esta é sua configuração, agora legível e arquivável.

Passo 3 — o desastre. Hora de quebrar tudo de propósito. O reset vai derrubar seu SSH (esperado — é um reset!); a volta será pelo **console da VM**:

```
/system reset-configuration no-defaults=yes skip-backup=yes
```

Confirme com `y`. Após o reboot, entre pelo console da VM: o login voltou a ser **admin sem senha**, não há usuários, não há IP — o equipamento está virgem. Sinta o frio na barriga: em produção, este seria um rádio mudo numa torre.

Passo 4 — ressuscitar pelo backup binário. O arquivo `.backup` ainda está no armazenamento interno (só a *configuração* foi zerada — arquivos sobrevivem ao reset). Pelo console:

```
/system backup load name=lab-r1-cap07
```

Confirme; o roteador reinicia e... está tudo de volta: identidade `lab-r1`, seu usuário, o IP de gerência, os serviços restritos. Prove pelo seu computador:

```
ssh vitor@192.168.56.11
```

Passo 5 — ressuscitar pela receita. Agora o caminho do export. Repita o reset do Passo 3. Pelo console, o equipamento vazio não tem IP — então crie o mínimo para receber o arquivo:

```
/ip address add address=192.168.56.11/24 interface=ether1
```

Do seu computador, suba o `.rsc` (o usuário agora é `admin` sem senha — estado pós-reset):

```
scp lab-r1-cap07.rsc admin@192.168.56.11:
```

E pelo console, importe:

```
/import file-name=lab-r1-cap07.rsc
```

```
Script file loaded and executed successfully
```

Compare com o Passo 4: o import reconstruiu serviços, listas, NTP, identidade — mas **não** os usuários com suas senhas (não estavam no export) nem removeu o que já existia (o IP que você criou à mão). A receita reconstrói a casa; os segredos, você repõe pelo checklist do 5.

7.5.3 Verificação

1. Os dois arquivos (`.backup` e `.rsc`) existem **no seu computador**, não só no roteador;
2. Após o Passo 4, `ssh vitor@...` funciona e `/ip service print` mostra a configuração do capítulo 6 intacta — restauração binária validada;
3. Após o Passo 5, `/system identity print` mostra `lab-r1` e os serviços estão restritos, mas `/user print` **não** tem o usuário `vitor` — você explicou a diferença da Tabela 7.1 na prática;
4. Termine restaurando o snapshot `seguranca-inicial` da VM (ou refazendo o usuário nominal), para os próximos capítulos partirem do estado certo.

7.5.4 Desafios

1. Gere um `/export verbose file=teste-verbose` e compare o tamanho com o export normal (`/file print`). De onde vem a diferença, e quando o verbose é mais útil?
2. Exporte **apenas** a seção `/system ntp client`, delete a configuração de NTP (`/system ntp client servers remove numbers=0,1` e `set enabled=no`) e restaure-a usando só o arquivo exportado.
3. Escreva (em texto) a rotina de backup de um provedor com 200 equipamentos: o que gerar, com que frequência, para onde os arquivos vão, e como você *testaria* as restaurações sem derrubar produção. Que ferramenta do RouterOS poderia agendar a geração? (Palpite livre — a resposta técnica completa é o Volume 12.)
4. Explique por que `run-after-reset` + um `.rsc` bem-feito é a forma mais segura de resetar um equipamento **remoto** — e o que precisa obrigatoriamente estar dentro desse `.rsc` para você não perder o acesso.

💡 Solução dos desafios

1. O verbose sai várias vezes maior: ele imprime **todos** os parâmetros, inclusive os que estão no padrão de fábrica (que o export normal omite). É mais útil para auditoria e comparação fina entre dois equipamentos “teoricamente iguais” — o diff do verbose denuncia qualquer padrão alterado.

2.

```
/system ntp client export file=ntp-backup
/system ntp client servers remove numbers=0,1
/system ntp client set enabled=no
/import file-name=ntp-backup.rsc
/system ntp client print
```

O import reexecuta os comandos da seção e o NTP volta a sincronizar.

3. Rotina defensável: export **diário** + backup binário **semanal** (e ambos antes/depois de qualquer mudança); envio automático para fora (FTP/SFTP de um servidor de backups, e-mail para os pequenos); retenção de algumas gerações; teste de restauração **mensal em laboratório** — um CHR que importa os exports do parque e denuncia erros de sintaxe/seções faltantes. O agendador do RouterOS é o `/system scheduler`, e o Volume 12 monta exatamente essa automação.

4. Porque elimina a janela em que o equipamento fica “vazio e surdo”: o reset já renasce aplicando a configuração-base. O `.rsc` precisa conter, no mínimo: IP de gerência na interface certa (ou DHCP client), usuário nominal com senha (lembre: o export não carrega senhas — num `.rsc` de provisionamento escrito à mão você **pode e deve** incluir o `/user add ... password=...`), e serviços de acesso habilitados/restritos. Sem isso, o “reset seguro” vira viagem à torre.

7.6 Resumo

- **Backup binário** = clone total (com senhas e certificados), restaura o mesmo equipamento; **export** = receita em texto, legível e portátil, sem segredos (Tabela 7.1). Provedor guarda **os dois**.
- Backup que só existe dentro do equipamento não é backup: **baixe por SCP/SFTP** e guarde com controle de acesso.
- `/export` por seção + `/import` é a ferramenta de migração e de “diff” de configuração; `verbose` e `show-sensitive` são os modificadores a conhecer.
- Reset profissional: `no-defaults=yes` para nascer limpo, `keep-users` e `run-after-reset` para não nascer surdo — e **sempre** com backups baixados antes.
- Backup não testado é esperança: ensaie a restauração (você acabou de fazer isso duas vezes).

No próximo capítulo, o último recurso de quem opera hardware de verdade: o **Netinstall** — reinstalar o RouterOS do zero quando nem o reset resolve.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/system backup`, `/export`, `/import` e `/system reset-configuration` (páginas *Backup* e *Configuration Management*). Discher (2016) discute estratégias de backup em parques de provedor. As referências completas, em ABNT, estão no apêndice [Referências](#).

8 Netinstall

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que o Netinstall faz e em que ele difere do reset do capítulo anterior — reinstalação do **sistema**, não só da configuração;
- Reconhecer as situações em que ele é o único caminho: equipamento que não dá boot, senha perdida, suspeita de comprometimento, downgrade;
- Descrever o mecanismo: RouterBOOT em modo *etherboot* + o aplicativo Netinstall respondendo na rede;
- Executar o procedimento completo num equipamento físico, incluindo os preparos que evitam as armadilhas clássicas (cabo direto, IP fixo, firewall do notebook);
- Usar os recursos extras: *keep old configuration*, pacotes por arquitetura, provisionamento com configuração inicial.

8.1 Quando o reset não basta

O 3 encerrou com o reset: apaga-se a configuração, o **sistema** permanece. Mas há defeitos que moram no sistema — e para eles existe o **Netinstall**, a reinstalação completa do RouterOS pela rede. Casos clássicos do provedor:

- o equipamento **não completa o boot** (atualização interrompida por queda de energia na torre, armazenamento corrompido);
- a **senha se perdeu** e não há backup — sem ela não há reset remoto, e o Netinstall zera tudo, sistema e senha juntos;
- suspeita de **comprometimento**: roteador invadido não se “limpa” com reset — a única higiene confiável é reinstalar o sistema de fora;
- **downgrade** controlado ou troca de *release channel* que o upgrade normal não resolve.

A analogia direta: o reset é a *restauração de fábrica* do celular; o Netinstall é *reinstalar o sistema operacional com um cabo no computador*. Um mexe nos dados; o outro regrava a própria plataforma.

O CHR fica de fora deste capítulo por natureza: sendo uma máquina virtual, “reinstalar o sistema” é simplesmente trocar o disco `.vdi` por um novo (Capítulo 5). O Netinstall é ferramenta do mundo **físico**, das RouterBOARD — e por isso este é o único capítulo do

livro cujo laboratório pede um equipamento de verdade (qualquer RouterBOARD barata serve, e um hEX usado é um ótimo investimento de estudo).

8.2 Como funciona por baixo

O truque está no RouterBOOT (Capítulo 3). Além de dar boot pelo armazenamento interno, ele sabe dar boot **pela rede** — o modo *etherboot*: o equipamento liga, envia pedidos de boot pela porta Ethernet e espera que alguém na rede lhe entregue um sistema.

Esse “alguém” é o aplicativo **Netinstall**, da MikroTik, rodando no seu computador, com dois papéis: servir o boot de rede e, em seguida, regravar o RouterOS escolhido no armazenamento do equipamento (Figura 8.1).

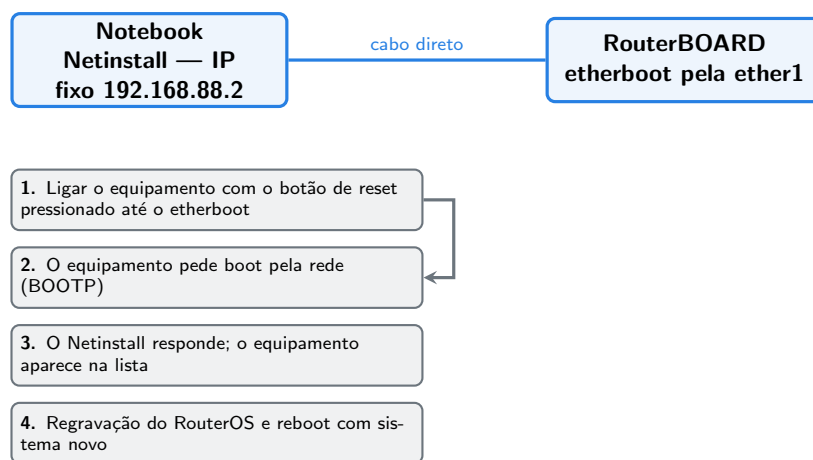


Figura 8.1: O fluxo do Netinstall: notebook e equipamento sozinhos num cabo, boot de rede e gravação do sistema.

Desse mecanismo derivam as regras práticas que evitam 90% das frustrações:

1. **Cabo direto, rede exclusiva.** O boot de rede fala com quem responder primeiro. Notebook e equipamento sozinhos (cabo ponta a ponta) — nunca através da rede do provedor, onde um DHCP alheio atropela o processo. Em muitos modelos, o etherboot só funciona pela **ether1/boot port**.
2. **IP fixo no notebook.** O Netinstall precisa de um endereço estável — o consagrado é 192.168.88.2/24. No aplicativo, configura-se o *Client IP* (ex.: 192.168.88.3) que será oferecido ao equipamento.
3. **Firewall do notebook fora do caminho.** O Windows adora bloquear BOOTP/TFTP silenciosamente; desative o firewall na interface do cabo durante o procedimento.
4. **Pacote da arquitetura certa.** O RouterOS é distribuído por arquitetura (*arm, arm64, mipsbe, tile...*). Baixe o *main package* correspondente ao seu modelo (a ficha do produto informa;

2) — o Netinstall só lista pacotes compatíveis com o equipamento detectado.

O procedimento em si, com os preparos feitos: abrir o Netinstall (como administrador), *Net booting* ativado com o Client IP, ligar o equipamento com o **botão de reset pressionado** até ele entrar em etherboot (os beeps e LEDs indicam; ~15–20 s), o equipamento aparece na lista, seleciona-se o pacote e **Install**. Minutos depois, reboot com sistema zerado — usuário **admin**, sem senha, configuração de fábrica.

Recursos que valem conhecer na janela do aplicativo:

- **Keep old configuration:** tenta preservar a configuração através da reinstalação — útil quando o objetivo é só regravar um sistema corrompido (mas lembre: se a hipótese é invasão, o objetivo é justamente **não** preservar nada);
- **Configure script:** aplica um script de configuração inicial após a instalação — o irmão do **run-after-reset** do capítulo anterior, e a base de provisionamento em massa (equipamento novo → Netinstall → nasce com a configuração-base do parque);
- **Downgrade:** instalar versão mais antiga é tão simples quanto escolher outro pacote — o caminho limpo quando um equipamento específico não se dá bem com uma versão nova.



Aviso

O Netinstall **apaga tudo**: sistema, configuração, usuários, arquivos — inclusive backups guardados no armazenamento interno (mais um motivo para a regra do 3: backup mora **fora** do equipamento). Sem *keep old configuration*, o equipamento volta absolutamente virgem. E não interrompa a gravação no meio — energia estável e paciência; um Netinstall interrompido só se resolve com... outro Netinstall.



O conceito mais importante do capítulo

Com o Netinstall no cinto de ferramentas, **não existe RouterBOARD perdida**: enquanto o hardware estiver são, qualquer estado de software — boot quebrado, senha esquecida, sistema comprometido — se resolve com um cabo, o aplicativo e alguns minutos. É a rede de segurança final de tudo o que você fará com MikroTik daqui em diante.

8.3 Laboratório

8.3.1 Ambiente

Excepcionalmente, este laboratório é **físico**: uma RouterBOARD qualquer (um hEX/hAP usado serve perfeitamente), um cabo Ethernet e um computador Windows com o aplicativo Netinstall (baixe de mikrotik.com/download, seção *Netinstall*) e o *main package* da arquitetura do seu equipamento.

Sem equipamento físico? Faça o “laboratório seco”: leia os passos executando cada preparo possível no seu computador (aplicativo aberto, IP fixo configurado, pacote baixado) e responda os desafios — o procedimento estará ensaiado para o dia em que a torre chamar.

8.3.2 Comandos passo a passo

Preparo do notebook. Configure IP fixo na interface Ethernet: 192.168.88.2, máscara 255.255.255.0 (sem gateway). Desative o firewall dessa interface. Conecte o cabo direto na **ether1** do equipamento (desligado). Abra o Netinstall como administrador, ative *Net booting* e defina o Client IP como 192.168.88.3.

Etherboot. Desconecte a energia do equipamento, pressione e **segure** o botão de reset, reconecte a energia mantendo o botão até o equipamento sinalizar o boot de rede (beep/LEDs, ou até ele aparecer na lista do aplicativo).

Instalação. O equipamento surge na lista do Netinstall com MAC e modelo. Selecione-o, marque o pacote RouterOS baixado e clique **Install**. Acompanhe a barra; ao final, *Installation finished* — e o equipamento reinicia.

Primeiro contato pós-instalação. O equipamento está de fábrica. Reconfigure o IP do notebook para DHCP (a default configuration entrega IP na LAN) ou use o WinBox por MAC (Capítulo 4) e aplique o ritual completo: remoção da default config se for o caso, checklist de segurança do 5, backup do 3. O ciclo do Volume 1, fechado num equipamento real.

8.3.3 Verificação

1. O equipamento apareceu na lista do Netinstall (prova do etherboot + cabo
 - IP corretos);
2. A instalação concluiu sem interrupção e o equipamento reiniciou;
3. O login pós-instalação é **admin** sem senha, e `/system resource print` mostra a versão que você instalou;
4. Ao final, o equipamento está com o checklist de segurança aplicado e um backup baixado — pronto para uso (ou para a próxima brincadeira).

8.3.4 Desafios

1. Por que o Netinstall pede cabo **direto** entre notebook e equipamento, em vez de usar a rede normal do escritório? Cite os dois problemas que a rede compartilhada introduz.
2. Um técnico esqueceu a senha de um rádio na torre, e o rádio **funciona** (clientes navegando). Netinstall já? Monte a árvore de decisão, considerando o que você sabe de acesso (Capítulo 4), backup
 - (3) e as proteções do RouterBOOT (Capítulo 3).

3. Você suspeita que um roteador de borda foi invadido. Por que *keep old configuration* seria um erro nesse caso — e qual é o fluxo correto de recuperação, do Netinstall ao retorno à produção?
4. Desenhe (em texto) a “bancada de provisionamento” de um provedor que ativa 30 rádios por mês: como Netinstall + *configure script* + o checklist do 5 viram uma linha de montagem.

Solução dos desafios

1. (a) **Concorrência de servidores:** na rede do escritório há DHCP e outros serviços que respondem ao boot de rede antes/junto do Netinstall, confundindo o processo; (b) **caminho não confiável:** switches, VLANs e firewalls no meio podem filtrar BOOTP/TFTP. O cabo direto elimina as duas variáveis — e é por isso que é o primeiro item de qualquer checklist de Netinstall que falhou.
2. Netinstall é a **última** parada, porque exige subir na torre e derruba os clientes durante o procedimento. Antes: (1) tentar as vias que não dependem de senha esquecida — outra credencial da equipe, acesso por outro usuário **full**; (2) se há backup binário recente e acesso físico fácil: reset + restaurar backup pode ser mais rápido; (3) sem nada disso — e sem **protected-routerboot** ativo —, Netinstall com janela de manutenção programada. Se **protected-routerboot** estiver ativo, nem o Netinstall entra: é o cenário de RMA que o capítulo 3 avisou.
3. Se o invasor alterou a configuração (usuário oculto, scheduler malicioso, regra de NAT espiã), *keep old configuration* **reinstala a porta dos fundos junto**. Fluxo correto: Netinstall limpo → reconstruir a configuração a partir de um **export conhecido e auditado** (de antes do incidente, lido linha a linha antes do import) → senhas novas em tudo → só então voltar à produção — idealmente entendendo por onde a invasão entrou (o hardening do Volume 3 existe para isso).
4. Linha de montagem: bancada com switch dedicado só para provisionamento; notebook com Netinstall e *configure script* apontando para o **.rsc**-base do parque (IP de gerência por DHCP da bancada, usuário nominal da equipe, serviços restritos, NTP — o checklist do capítulo 6 em forma de script); cada rádio novo: etherboot → install → o script aplica a base → etiqueta com identidade → backup/export arquivados → estoque de “prontos para torre”. Trinta rádios viram uma tarde de trabalho repetitivo e à prova de esquecimento.

8.4 Resumo

- **Netinstall reinstala o sistema;** o reset só limpa a configuração. É o caminho para boot quebrado, senha perdida, comprometimento e downgrade.
- Mecanismo: RouterBOOT em **etherboot** (botão de reset ao ligar) + o aplicativo Netinstall servindo boot e regravando o RouterOS (Figura 8.1).
- Receita à prova de frustração: **cabo direto**, notebook com IP fixo 192.168.88.2, firewall local desligado, pacote da **arquitetura certa**, e paciência durante a gravação.

- *Keep old configuration* preserva a configuração (nunca em caso de invasão); *configure script* provisiona — o **run-after-reset** do mundo físico.
- Netinstall apaga **tudo**, inclusive backups internos — que, você já sabe, deveriam estar fora do equipamento.
- Com ele, fecha-se o ciclo do Volume 1: não existe RouterBOARD irrecuperável.

Este capítulo encerra o Volume 1 — você conhece o sistema, o hardware, as licenças, os acessos, montou seu laboratório e domina o ciclo de vida completo de um equipamento. No Volume 2, mergulhamos de vez na linha de comando: a anatomia do terminal e a construção, peça por peça, de um roteador de borda funcional.

Bibliografia do capítulo

A página *Netinstall* da documentação oficial (MikroTik, 2026a) traz o passo a passo com capturas atualizadas e a tabela de solução de problemas que inspirou nossas regras práticas; o download do aplicativo e dos *main packages* por arquitetura está em mikrotik.com/download. As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte II

Volume 2 — CLI e configuração essencial

9 Anatomia do terminal

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Navegar pela árvore de menus do RouterOS como quem navega por pastas — caminho absoluto, caminho relativo e o retorno à raiz;
- Usar os cinco verbos que operam quase tudo: `print`, `add`, `set`, `remove` e `enable/disable`;
- Entender a **numeração de itens** — por que ela muda, quando usá-la e por que `[find ...]` é mais seguro;
- Explorar comandos desconhecidos sozinho, com `TAB` (completar) e `?` (ajuda embutida);
- Usar `export` como lupa (3) para conferir qualquer seção da configuração;
- Aplicar truques de produtividade: histórico, `/undo` e `/redo`, comentários e flags.

9.1 A árvore e os verbos

Você já usou o terminal do RouterOS em todos os capítulos do Volume 1 — agora vamos entendê-lo de verdade. O modelo mental cabe numa frase: **a configuração é uma árvore de menus; em cada menu vivem itens; e um pequeno conjunto de verbos opera esses itens.**

A árvore você já conhece de tanto digitar: `/ip address` é o menu de endereços dentro da seção IP; `/system identity`, a identidade dentro de system. A barra inicial é a **raiz** — como o `C:\` do Windows ou o `/` do Linux. E, como em qualquer árvore de diretórios, você pode:

<code>/ip address print</code>	<code># caminho absoluto: executa de onde estiver</code>
<code>/ip</code>	<code># entrar no menu ip</code>
<code>address</code>	<code># descer para ip > address</code>
<code>print</code>	<code># executar o comando do menu atual</code>
<code>/</code>	<code># voltar à raiz</code>
<code>..</code>	<code># subir um nível</code>

O prompt sempre mostra onde você está:

```
[vitor@lab-r1] > /ip address
[vitor@lab-r1] /ip/address> print
[vitor@lab-r1] /ip/address> ..
[vitor@lab-r1] /ip>
```

Entrar num menu compensa quando você fará várias operações nele; para comandos avulsos, o caminho absoluto documenta melhor — e é por isso que todos os blocos de comandos deste livro usam caminhos absolutos.

Sobre essa árvore agem os **verbos**. Cinco fazem 95% do trabalho:

Tabela 9.1: Os cinco verbos fundamentais

Verbo	Faz	Exemplo
<code>print</code>	lista os itens do menu	<code>/ip address print</code>
<code>add</code>	cria um item	<code>/ip address add address=... interface=...</code>
<code>set</code>	altera um item existente	<code>/ip service set ssh port=2222</code>
<code>remove</code>	apaga um item	<code>/ip address remove numbers=0</code>
<code>enable / disable</code>	liga/desliga sem apagar	<code>/ip service disable telnet</code>

Repare no padrão dos argumentos: **chave=valor**, separados por espaço, sem vírgulas. `add address=192.168.56.11/24 interface=ether1` lê-se como uma frase: “adicione o endereço X na interface Y”. Aspas só quando o valor tem espaços (`comment="link para a torre"`).

Analogia para guardar

O terminal do RouterOS é um sistema de arquivos onde cada “pasta” (menu) contém “planilhas” (itens em linhas numeradas), e os verbos são as operações de linha: listar, inserir, editar, apagar, ocultar. Quem pensa assim nunca se perde — só pergunta “em que menu estou e que linha quero mexer?”.

9.2 Itens, números e o perigo escondido

Faça um `print` em qualquer menu populado e observe a primeira coluna:

```
[vitor@lab-r1] > /ip service print
Columns: NAME, PORT, ADDRESS
#  NAME      PORT  ADDRESS
```

```
0 X telnet      23
1 X ftp         21
2 X www         80
3  ssh         22 192.168.56.0/24
```

O # é o **número do item** — o endereço da linha, usado pelos verbos: `set 3 port=2222`, `remove numbers=0,1`, `disable numbers=2`. Funciona, e você usará o tempo todo em sessões interativas. Mas guarde o aviso:

 Aviso

A numeração **não é estável**: ela é recalculada a cada `print` — remover um item renumera os seguintes, e menus diferentes têm numerações independentes. Número de item em **script**, em **procedimento documentado** ou em comando executado “de memória” sem um `print` imediatamente antes é receita para apagar a linha errada — às vezes, a que segurava seu acesso.

A alternativa profissional é selecionar por **conteúdo**, com `[find ...]`:

```
/ip address remove [find comment=teste-lab-a]
/ip service set [find name=ssh] port=2222
```

O `find` devolve os itens que casam com o critério, e o verbo age sobre eles — não importa em que posição estejam. Você já o usou no laboratório do Capítulo 5; nos scripts do Volume 12 ele será onipresente. Regra prática: **números na exploração interativa; find em tudo que será repetido**.

Duas colunas de metadados ajudam o `find` e os humanos:

- **comment** — todo item aceita um comentário. Em rede de provedor, comentário é documentação de campo: `comment="uplink operadora X"` salva madrugadas;
- **flags** — as letras à esquerda do `print`: X (desabilitado), D (dinâmico: criado por um processo, como DHCP — não se edita, se edita a origem), I (inválido: aponta para algo que não existe mais, ex.: endereço numa interface removida).

9.3 TAB e ?: o manual que mora no equipamento

O RouterOS carrega a própria documentação, e dois atalhos a liberam:

- **TAB** completa o que você começou a digitar: `/ip add + TAB` vira `/ip address`; dois TABs listam as opções possíveis. Além de poupar dedos, o TAB **valida** — se não completa, o comando não existe (ou você está no menu errado);
- **?** mostra a ajuda do ponto onde você está: os submenus de um menu, os argumentos de um comando, os valores válidos de um argumento (`address=?`).

A combinação transforma qualquer menu desconhecido em terreno explorável — foi assim que você descobriu o `/system shutdown` no primeiro capítulo, e é assim que se aprende recurso novo sem sair do terminal.

Complete o kit de exploração com:

- **histórico:** setas ↑/↓ navegam os comandos anteriores da sessão;
- **/undo e /redo:** desfazem/refazem a última ação de configuração — o Ctrl+Z do RouterOS (o Safe Mode do Capítulo 4 continua sendo o cinto de segurança para *sequências* de mudanças);
- **/system history print:** quem fez o quê, quando — o undo sabe até o que desfazer porque essa trilha existe.

9.4 print afiado e export como lupa

O `print` cru despeja tudo; os modificadores o transformam em ferramenta de precisão:

```
/ip address print detail          # todos os campos de cada item
/ip address print where interface=ether1  # filtra por condição
/ip route print count-only        # só conta
/interface print stats            # estatísticas (tráfego, erros)
/ip address print interval=2      # re-executa a cada 2s (monitor ao vivo)
```

O `where` merece atenção: é o irmão interativo do `find` — mesma sintaxe de condições, mas para `ver` em vez de agir. `print where` para investigar, `[find]` para operar.

E quando a pergunta é “como este menu está configurado, por inteiro?”, a resposta é o velho conhecido `export` (3), que funciona em **qualquer** menu:

```
/ip service export
```

```
# 2026-07-06 15:12:40 by RouterOS 7.15.3
/ip service
set telnet disabled=yes
set ftp disabled=yes
set www disabled=yes
set ssh address=192.168.56.0/24
```

`export` responde à pergunta que o `print` não responde: *o que foi mudado em relação à fábrica?* — só as alterações aparecem. Por isso ele é a lupa deste livro: sempre que um laboratório disser “confira a seção X”, o `export` da seção é a conferência.

! O conceito mais importante do capítulo

O terminal inteiro cabe numa gramática: **menu** (onde) + **verbo** (o quê) + **chave=valor** (como) + **seleção por número ou find** (em quem). Com TAB e ? para explorar e **export** para conferir, você opera qualquer recurso do RouterOS — inclusive os que este livro não cobre.

9.5 Laboratório

9.5.1 Ambiente

O laboratório padrão (Capítulo 5): apenas **lab-r1**, a partir do snapshot **seguranca-inicial**. Acesso: `ssh vitor@192.168.56.11`.

9.5.2 Comandos passo a passo

Passo 1 — navegar. Percorra a árvore dos dois jeitos e observe o prompt:

```
/ip
address
print
..
service
print
/
```

Passo 2 — explorar com TAB e ?. No menu raiz, digite `/ip a` e pressione TAB duas vezes: o RouterOS lista os menus que começam com **a** (**address**, **arp**, ...). Entre em `/ip dns` (que nunca usamos) e descubra, usando apenas `?`, qual comando mostra a configuração e qual argumento ativa a resposta a consultas remotas — **não altere nada**; a hora do DNS chega no Capítulo 12.

Passo 3 — os verbos num alvo inofensivo. Vamos exercitar `add/set/disable/remove` num menu sem risco: as entradas estáticas de DNS locais.

```
/ip dns static add name=teste1.lab address=10.99.0.1 comment="cobaia 1"
/ip dns static add name=teste2.lab address=10.99.0.2 comment="cobaia 2"
/ip dns static print
```

```
[vitor@lab-r1] > /ip dns static print
Columns: NAME, ADDRESS, TTL
# NAME      ADDRESS    TTL
0 teste1.lab 10.99.0.1 1d
1 teste2.lab 10.99.0.2 1d
```

Altere, desabilite, observe as flags:

```
/ip dns static set 0 address=10.99.0.100
/ip dns static disable 1
/ip dns static print
```

A entrada 1 agora exibe X. Confira a seção com a lupa:

```
/ip dns static export
```

Passo 4 — sentir a renumeração. Remova o item 0 e imprima de novo:

```
/ip dns static remove numbers=0
/ip dns static print
```

O antigo item 1 (teste2.lab) agora é o 0 — a prova concreta do aviso deste capítulo. Remova-o pelo caminho seguro, por conteúdo:

```
/ip dns static remove [find comment="cobaia 2"]
/ip dns static print
```

Passo 5 — undo e history. Crie uma cobaia nova, desfça e confira a trilha:

```
/ip dns static add name=teste3.lab address=10.99.0.3
/undo
/ip dns static print
/system history print
```

O print volta vazio, e o history mostra as duas ações (o add e o desfazer).

9.5.3 Verificação

1. Você navegou com prompt mudando (/ip/address> etc.) e voltou à raiz;
2. O TAB duplo listou menus, e o ? revelou os argumentos de /ip dns sem você alterar nada;
3. Após o Passo 4, /ip dns static print está vazio — e você viu a renumeração acontecer no meio do caminho;
4. /system history print conta a história completa do laboratório.

9.5.4 Desafios

1. Usando apenas TAB e ? (sem internet), descubra o que faz o comando `/interface ethernet print stats` e identifique duas colunas úteis para diagnosticar um cabo ruim.
2. Em uma linha, desabilite **todas** as entradas de `/ip dns static` que contenham a palavra `lab` no nome. (Dica: `find` aceita `name~"lab"` — o `~` casa por expressão regular.)
3. O comando `/ip service set [find name=ssh] port=2222` muda a porta do SSH. Se você o executasse agora, o que aconteceria com a sua sessão atual? E como você se conectaria depois? (Pense antes; se testar, use o Safe Mode e desfaça.)
4. Explique a diferença entre `print where interface=ether1` e `[find interface=ether1]` — quando cada um é a ferramenta certa?

Solução dos desafios

1. `/interface ethernet print stats` mostra contadores por porta física. Para cabo ruim, procure `rx-fcs-error/rx-align-error` (quadros corrompidos chegando — clássico de cabo/conector ruim) e `rx-too-short`/colisões. Contadores subindo continuamente = camada física suspeita.

2.

```
/ip dns static disable [find name~"lab"]
```

3. A sessão **atual não cai** — conexões TCP já estabelecidas sobrevivem à mudança do serviço; mas a **próxima** conexão precisa de `ssh -p 2222 vitor@192.168.56.11`. O risco real é esquecer a porta nova (ou um firewall no caminho que só libera a 22) — por isso: Safe Mode, testar a conexão nova antes de fechar a antiga (a mesma regra de ouro do 5 com usuários).

4. São a mesma sintaxe de seleção com destinos diferentes: `print where` **mostra** o subconjunto (investigação, sem efeito colateral); `[find]` **entrega os itens a um verbo** (`set`, `remove`, `disable`...) para agir sobre eles. Investigue com `where`, opere com `find` — e nunca opere com números decorados.

9.6 Resumo

- Configuração = **árvore de menus** + **itens numerados** + **verbos** (`print`, `add`, `set`, `remove`, `enable/disable`; Tabela 9.1), com argumentos `chave=valor`.
- Números de item **mudam** (renumeração após remoções): use-os só com um `print` fresco na tela; em tudo repetível, selecione com `[find ...]`.
- Flags contam estado: X desabilitado, D dinâmico, I inválido — e `comment` é a documentação de campo do provedor.
- TAB completa e valida; ? é o manual embutido; ↑/↓, `/undo`, `/redo` e `/system history` completam o kit interativo.

- `print` tem modificadores (`detail`, `where`, `stats`, `interval`); `export` em qualquer menu mostra **o que difere da fábrica** — a lupa para conferir laboratórios e auditar equipamentos.

No próximo capítulo, a gramática entra em serviço: endereçamento IP no RouterOS e a rota padrão — os dois primeiros tijolos do roteador de borda que construiremos ao longo deste volume.

Bibliografia do capítulo

A seção *Console* da documentação oficial (MikroTik, 2026a) detalha a navegação, os modificadores de `print` e a sintaxe completa de seleção (`find`, `where`, expressões regulares). Discher (2016) e Burgess (2011) dedicam capítulos iniciais à filosofia da CLI. As referências completas, em ABNT, estão no apêndice [Referências](#).

10 Endereçamento IP e rota padrão

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Ler e escrever endereços em notação **CIDR** (192.168.10.1/24) e escolher o prefixo certo para cada situação (enlaces /30, LANs /24);
- Configurar endereços em `/ip address` e explicar o que o RouterOS deriva sozinho (campo **network**, rotas conectadas);
- Ler a tabela de rotas (`/ip route print`) com suas flags — D, A, C, S — como quem lê um mapa;
- Criar **rotas estáticas** para redes remotas e a **rota padrão** (0.0.0.0/0), entendendo gateway e **distance**;
- Usar **check-gateway** para que uma rota morta saia de cena sozinha.

10.1 O endereço e a máscara, sem mistério

Um endereço IPv4 são 32 bits escritos em quatro números (192.168.10.1), e todo endereço em uso carrega junto uma informação crucial: **até onde vai a minha rede?** Essa é a função do **prefixo** — o /24 em 192.168.10.1/24, a notação **CIDR** que usaremos no livro inteiro.

O prefixo diz quantos bits iniciais identificam a **rede**; o resto identifica os **hospedeiros** dentro dela. /24 significa “os 3 primeiros números são a rede”: tudo que começa com 192.168.10. é vizinho direto, alcançável sem roteador. Os tamanhos que um provedor usa todo dia:

Tabela 10.1: Prefixos do dia a dia do provedor

Prefixo	Máscara	Endereços úteis	Uso típico
/30	255.255.255.252	2	enlace ponto a ponto entre roteadores
/29	255.255.255.248	6	bloco pequeno de IPs fixos de cliente
/24	255.255.255.0	254	LAN, rede de gerência
/22	255.255.252.0	1022	pool de assinantes

A conta dos “úteis” desconta dois endereços reservados de cada rede: o primeiro (**endereço da rede**, ex.: 192.168.10.0) e o último (**broadcast**, ex.: 192.168.10.255). É por isso

que o enlace entre dois roteadores casa perfeitamente num /30: 4 endereços, 2 reservados, 2 roteadores. Nada sobra, nada falta.

💡 Analogia para guardar

O endereço IP é “rua + número”: o prefixo diz onde termina o nome da rua e começa o número da casa. Dois vizinhos da mesma rua conversam pela janela (mesmo segmento, sem roteador); para outra rua, é preciso ir ao **cruzamento com o guarda de trânsito** — o gateway.

Uma convenção que você já viu no laboratório: redes **privadas** (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 (Rekhter et al., 1996)) são livres para uso interno e não existem na internet. O que acontece quando uma LAN privada precisa sair para a internet é o assunto do Capítulo 13.

10.2 /ip address: o que você dá e o que o RouterOS deduz

Configurar um endereço você já fez desde o Capítulo 4; agora vamos olhar o que acontece por dentro:

```
/ip address add address=10.0.12.1/30 interface=ether2 comment="enlace r1-r2"
/ip address print detail
```

```
[vitor@lab-r1] > /ip address print detail
Flags: X - disabled, I - invalid, D - dynamic
 0   ;;; enlace r1-r2
      address=10.0.12.1/30 network=10.0.12.0 interface=ether2
      actual-interface=ether2
```

Você deu **address** e **interface**; o RouterOS **derivou** o **network** (10.0.12.0, o primeiro endereço do /30). E fez mais uma coisa, silenciosa e importante — criou uma **rota conectada**. Espie a tabela de rotas:

```
/ip route print
```

```
[vitor@lab-r1] > /ip route print
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT
Columns: DST-ADDRESS, GATEWAY, DISTANCE
  DST-ADDRESS  GATEWAY  DISTANCE
DAc 10.0.12.0/30 ether2      0
DAc 192.168.56.0/24 ether1    0
```

Leia as flags como um crachá de cada rota:

- **D** (*dynamic*) — criada automaticamente por um processo (aqui, pelo endereço que você adicionou; adiante, por DHCP, PPPoE, OSPF...). Não se edita uma rota dinâmica; edita-se a origem dela;
- **A** (*active*) — em uso: é a vencedora para aquele destino agora;
- **c** (*connect*) — rota **conectada**: “esta rede está pendurada direto nesta interface”. É o RouterOS registrando quem são as ruas das suas janelas.

A regra de ouro que essa tabela ensina: **um roteador só alcança o que está na tabela de rotas**. Redes conectadas entram sozinhas; todo o resto — as redes que estão *atrás de outros roteadores* — precisa de rota. É o próximo assunto.

10.3 Rotas estáticas: ensinando o caminho

Imagine a topologia do nosso laboratório (Figura 10.1): **lab-r1** e **lab-r2** conversam pelo enlace 10.0.12.0/30, e atrás de **lab-r2** existe uma LAN 192.168.100.0/24. O **lab-r1** conhece o enlace (conectado), mas a LAN é invisível para ele — não há rota.

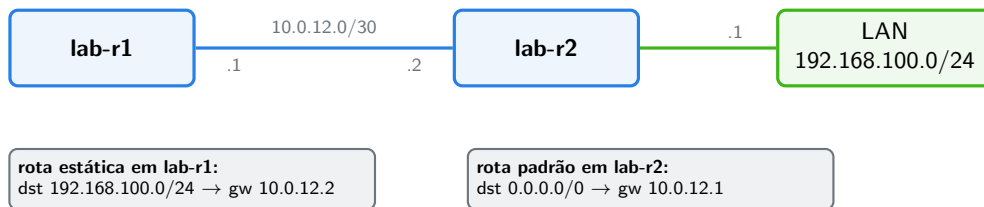


Figura 10.1: A topologia do laboratório deste capítulo, com as duas rotas que a fazem funcionar: a estática (r1 aprende a LAN) e a padrão (r2 manda “todo o resto” para r1).

A solução é uma **rota estática** — você, administrador, ensinando o caminho:

```
/ip route add dst-address=192.168.100.0/24 gateway=10.0.12.2 comment="LAN atrás do r2"
```

Traduzindo: “para chegar em 192.168.100.0/24, entregue o pacote a 10.0.12.2”. O **gateway** precisa ser alcançável por uma rede conectada — é o guarda do cruzamento da sua rua, não de uma rua distante.

No **print**, ela aparece com as flags **AS** (*active, static*). E a coluna que ainda não explicamos entra em cena:

10.3.1 distance: o desempate entre rotas

Duas rotas para o **mesmo destino** podem coexistir — enlace principal e backup, por exemplo. Quem decide a vencedora é a **distância administrativa** (**distance**): menor vence, e só a vencedora fica ativa (**A**). Rotas conectadas têm distância 0 (imbatíveis — a

rede está ali); estáticas nascem com 1; protocolos dinâmicos usam valores maiores (OSPF 110, BGP 20/200 — Volume 8).

O uso clássico no provedor:

```
/ip route add dst-address=0.0.0.0/0 gateway=10.0.12.1 distance=1 comment="saida principal"
/ip route add dst-address=0.0.0.0/0 gateway=10.0.99.1 distance=5 comment="saida backup"
```

A principal vence; se ela sumir da tabela, a backup assume. “Sumir” é o detalhe crucial — e nos leva às duas últimas peças.

10.3.2 A rota padrão e o check-gateway

`dst-address=0.0.0.0/0` casa com **qualquer destino** — é a **rota padrão** (*default route*): “tudo que eu não souber entregar, vai para este gateway”. É ela que leva sua LAN à internet, e é a rota mais importante de qualquer roteador de borda. Na escolha do caminho vale sempre o **prefixo mais específico primeiro**: um pacote para 192.168.100.7 prefere a rota /24 da LAN à rota /0 — a padrão só apanha as sobras. A `distance` só desempata rotas para o **mesmo** prefixo.

Falta o seguro contra gateway morto. Uma rota estática fica ativa enquanto o gateway for *alcançável na tabela* — mas “alcançável” não significa *vivo*: se o roteador do outro lado travar com a interface acesa, a rota continua ativa, despejando pacotes num buraco. O remédio básico:

```
/ip route set [find comment="saida principal"] check-gateway=ping
```

Com `check-gateway=ping`, o RouterOS pinga o gateway periodicamente (a cada 10 s); duas falhas seguidas **desativam a rota** — e a backup de distância 5 assume sozinha. (O failover de gente grande, com detecção de falha *além* do gateway, é assunto do Volume 14.)

! O conceito mais importante do capítulo

Um roteador é a sua **tabela de rotas** — ele não alcança nada fora dela. Redes conectadas entram sozinhas (DAc); o resto é decisão sua: rotas estáticas para o que está atrás de outros roteadores, uma rota padrão (0.0.0.0/0) para “todo o resto”, `distance` para hierarquizar caminhos e `check-gateway` para tirar de cena um caminho morto.

10.4 Laboratório

10.4.1 Ambiente

Laboratório padrão (Capítulo 5): **lab-r1** e **lab-r2**, snapshot base-limpa (em lab-r1, o snapshot `seguranca-inicial` também serve). Acessos: `ssh vitor@192.168.56.11` (r1) e

`ssh admin@192.168.56.12 (r2)`. Topologia da Figura 10.1: enlace pela rede interna lab-a (ether2 de ambos) e a “LAN” na ether3 de lab-r2 (rede interna lab-b, sem mais ninguém — ela existe só para ser anunciada).

10.4.2 Comandos passo a passo

Passo 1 — endereçar o enlace. Em lab-r1:

```
/ip address add address=10.0.12.1/30 interface=ether2 comment="enlace r1-r2"
```

Em lab-r2:

```
/ip address add address=10.0.12.2/30 interface=ether2 comment="enlace r1-r2"  
/ping 10.0.12.1 count=3
```

O ping deve fechar sem perdas — enlace vivo.

Passo 2 — criar a “LAN” atrás do r2. Em lab-r2:

```
/ip address add address=192.168.100.1/24 interface=ether3 comment="LAN simulada"
```

Passo 3 — sentir a falta da rota. De lab-r1, tente alcançar a LAN:

```
/ping 192.168.100.1 count=3
```

```
[vitor@lab-r1] > /ping 192.168.100.1 count=3  
SEQ HOST          SIZE TTL TIME   STATUS  
  0 192.168.100.1                no route to host  
  1 192.168.100.1                no route to host  
  2 192.168.100.1                no route to host  
sent=3 received=0 packet-loss=100%
```

no route to host: o r1 não tem rota — exatamente o diagnóstico que a tabela confirma (/ip route print não tem 192.168.100.0/24).

Passo 4 — a rota estática. Em lab-r1:

```
/ip route add dst-address=192.168.100.0/24 gateway=10.0.12.2 comment="LAN atras do r2"  
/ip route print
```

```
[vitor@lab-r1] > /ip route print
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT, s - STATIC
Columns: DST-ADDRESS, GATEWAY, DISTANCE
  DST-ADDRESS      GATEWAY    DISTANCE
As 192.168.100.0/24 10.0.12.2      1
DAc 10.0.12.0/30    ether2        0
DAc 192.168.56.0/24 ether1         0
```

E o mesmo ping agora:

```
/ping 192.168.100.1 count=3
```

Sem perdas. Você acabou de rotear entre redes pela primeira vez no livro.

Passo 5 — a rota padrão do r2, com seguro. O r2 conhece o enlace e a própria LAN; ensine-o a mandar “todo o resto” para o r1:

```
/ip route add dst-address=0.0.0.0/0 gateway=10.0.12.1 check-gateway=ping comment="default"
/ip route print where dst-address=0.0.0.0/0
```

Teste com um destino que o r2 **não** conhece — o IP de gerência do r1:

```
/ping 192.168.56.11 count=3
```

Funciona: a rota padrão entregou ao r1, que conhece a rede de gerência (conectada).

Passo 6 — matar o gateway e observar o seguro. Em **lab-r1**, derrube a ponta do enlace (é para isso que a gerência é separada — seu SSH não passa por ali):

```
/interface disable ether2
```

Em **lab-r2**, aguarde ~20 s e imprima:

```
/ip route print where dst-address=0.0.0.0/0
```

A rota padrão perdeu a flag A — o **check-gateway** detectou o gateway morto e a desativou. Reative em **lab-r1** (**/interface enable ether2**), aguarde e confirme que o A voltou.

10.4.3 Verificação

1. `/ip address print` em cada roteador confere com a Figura 10.1 (inclusive o `network` derivado);
2. O ping do Passo 3 falhou com `no route to host` e o do Passo 4 fechou com `packet-loss=0%` — antes e depois da rota;
3. `/ip route print` em `lab-r1` mostra a estática `As` e as conectadas `DAc`;
4. No Passo 6, a rota padrão de `lab-r2` ficou inativa com a `ether2` do `r1` desabilitada e reativou sozinha depois.

10.4.4 Desafios

1. O enlace `r1-r2` usa `/30`. Se você o tivesse endereçado como `/24` (`10.0.12.1/24` e `.2/24`), tudo funcionaria igual — então por que o `/30` é a boa prática em enlaces de provedor? (Duas razões.)
2. Em `lab-r2`, adicione uma **segunda** rota padrão: `gateway=192.168.100.254 distance=5`. Ela aparece ativa? Por quê? E se o `check-gateway` da principal desativá-la (repita o Passo 6), quem assume — e isso seria bom ou ruim nesta topologia?
3. De `lab-r1`, `/ping 192.168.100.1` funciona. Mas se houvesse um computador de verdade na LAN do `r2` (ex.: `192.168.100.50`), o ping de `lab-r1` até ele funcionaria com a configuração atual? Analise o caminho de **volta** do pacote.
4. Qual é a diferença entre remover uma rota (`remove`) e desabilitá-la (`disable`)? Em que situação de operação o `disable` é a escolha certa?

Solução dos desafios

1. (a) **Economia**: um `/24` queimaria 254 endereços úteis onde só existem 2 — em blocos públicos, isso é dinheiro; mesmo em privados, é organização. (b) **Contenção**: no `/30` não existe espaço para um terceiro “morador” — um equipamento intruso ou um erro de configuração não ganha endereço válido no enlace, e a documentação da rede fica inequívoca (cada `/30` é um enlace, com dono conhecido).
2. Ela entra na tabela, mas **sem** a flag **A**: para o mesmo prefixo (`0.0.0.0/0`), vence a menor distância ($1 < 5$). Com a principal desativada pelo `check-gateway`, a de distância 5 assume (**A**). Aqui isso seria **ruim**: `192.168.100.254` não existe (a LAN é simulada, vazia) — o tráfego cairia num buraco. Moral: rota de backup só ajuda se apontar para um caminho que existe; backup mentiroso é pior que nenhum.
3. **Não**. O pacote de ida chega (`r1 → r2 → LAN`), mas o computador responderia para `10.0.12.1` (o endereço de origem do ping) — e a volta depende de o **computador** saber voltar: o gateway dele precisa ser `192.168.100.1` (o `r2`), e o `r2` conhece `10.0.12.0/30` (conectada), então funcionaria **se** o computador tiver o gateway certo. A lição: rota se analisa **nos dois sentidos** — metade dos problemas de “não pinga” mora no caminho de volta.
4. `remove` apaga (e some da configuração, do export, de tudo); `disable` mantém o

item documentado, com comentário e parâmetros, apenas inativo (X). Em operação, **disable** é o certo para mudanças reversíveis e janelas de manutenção — “tirar o backup de cena por uma hora” — e para guardar a configuração de um enlace que volta semana que vem. **remove** é para o que não volta mais.

10.5 Resumo

- **CIDR**: o prefixo diz onde a rede termina; /30 para enlaces, /24 para LANs (Tabela 10.1); cada rede reserva o primeiro (rede) e o último (broadcast) endereços.
- `/ip address add address=X/NN interface=Y` — o RouterOS deriva o **network** e cria a **rota conectada** (DAc) sozinho.
- **A tabela de rotas é o roteador**: conectadas entram sós; o resto exige rota estática (As) com **gateway** alcançável, ou virá de protocolos dinâmicos (Volume 8).
- Rota mais específica vence; entre rotas para o **mesmo** prefixo, vence a menor **distance** (conectada 0, estática 1) — e só a vencedora fica Ativa.
- **0.0.0.0/0** é a rota padrão (“todo o resto”); com **check-gateway=ping**, um gateway morto desativa a rota e o backup de distância maior assume.
- Problema de alcance se analisa **nos dois sentidos** — ida e volta.

No próximo capítulo, tiramos os endereços do modo manual: DHCP client para a WAN receber endereço da operadora, e DHCP server para a LAN distribuir endereços sozinha.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip address` e `/ip route` (incluindo todas as flags e o comportamento do **check-gateway**). O RFC 1918 (Rekhter et al., 1996) define as faixas privadas usadas nos laboratórios. Para a teoria de endereçamento e roteamento IP, Tanenbaum; Feamster; Wetherall (2021) segue sendo a referência completa. As referências, em ABNT, estão no apêndice [Referências](#).

11 DHCP client e server

Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o diálogo DHCP (Discover → Offer → Request → Ack) e o papel do *lease* (empréstimo com prazo);
- Configurar o **DHCP client** numa interface WAN e controlar o que ele aceita do provedor (`add-default-route`, `use-peer-dns`);
- Montar um **DHCP server** completo na LAN — pool, network e server — e explicar o papel de cada peça;
- Gerenciar *leases*: ler a lista, fixar endereço de um equipamento (`make-static`) e escolher o `lease-time` adequado;
- Reconhecer os dois lados na rede de um provedor: onde se é cliente (upstream) e onde se é servidor (clientes internos, gerência de CPEs).

11.1 O empréstimo de endereços

No capítulo anterior, cada endereço foi digitado à mão. Funciona para roteadores — são poucos e não mudam de lugar. Mas a LAN de uma casa tem celulares, TVs e notebooks chegando e saindo; ninguém vai digitar endereço em cada um. O **DHCP** (*Dynamic Host Configuration Protocol*) automatiza: o dispositivo pergunta “alguém me dá um endereço?” e um servidor responde com o pacote completo — endereço, máscara, gateway e DNS.

O diálogo tem quatro passos, fáceis de lembrar pela sigla **DORA** (Figura 11.1):

O detalhe que muda a operação: o endereço é um **empréstimo** (*lease*) com prazo de validade. Na metade do prazo o cliente pede renovação; se o servidor sumir, o endereço expira e volta ao estoque. Esse prazo — o `lease-time` — será uma decisão sua adiante.

Analogia para guardar

DHCP é o balcão de um hotel: o hóspede chega sem quarto (Discover), a recepção oferece o 254 (Offer), o hóspede aceita (Request) e recebe a chave com data de saída (Ack + lease). Hóspede que renova a diária fica; quarto de quem partiu volta ao estoque. E o VIP (seu servidor, sua impressora) tem **quarto reservado** — o lease estático.

No dia a dia do provedor você vive **os dois lados do balcão**:

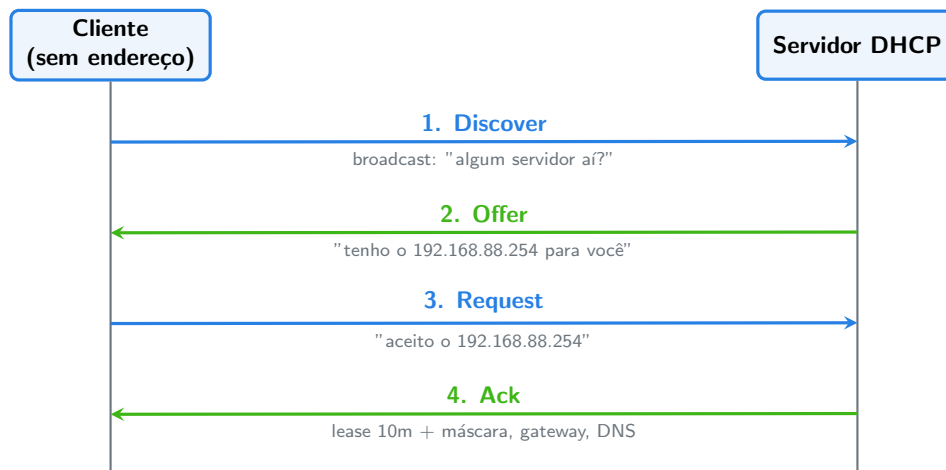


Figura 11.1: O diálogo DORA: o cliente descobre, o servidor oferece, o cliente pede, o servidor confirma — e o endereço vem emprestado, com prazo.

- **Cliente DHCP:** sua WAN recebendo endereço de um upstream — o link da operadora, uma ONU em modo bridge... e o nosso adaptador NAT do VirtualBox, que faz papel de operadora no laboratório;
- **Servidor DHCP:** distribuindo endereços na LAN do cliente final (o hAP na casa do assinante), na rede de gerência dos rádios de uma torre, na rede dos técnicos no POP.

11.2 DHCP client: a WAN que se configura sozinha

Você já rodou o comando no desafio do Capítulo 5; agora com entendimento completo:

```
/ip dhcp-client add interface=ether3
/ip dhcp-client print
```

```
[vitor@lab-r1] > /ip dhcp-client print
Columns: INTERFACE, USE-PEER-DNS, ADD-DEFAULT-ROUTE, STATUS, ADDRESS
# INTERFACE  USE-PEER-DNS  ADD-DEFAULT-ROUTE  STATUS  ADDRESS
0 ether3      yes           yes                bound   10.0.2.15/24
```

STATUS: bound = DORA completo, endereço emprestado. E repare no que veio de brinde — confira `/ip address print` e `/ip route print`: um endereço com flag **D** (dinâmico) e uma rota padrão **DAd** apontando para o gateway da operadora. É o Capítulo 10 acontecendo sozinho.

Os dois botões que importam num roteador de provedor:

- **add-default-route** (yes/no) — aceitar a rota padrão oferecida? No roteador de borda simples, sim. Num equipamento **interno** que só usa aquela WAN para um

serviço específico, não: a rota padrão da casa é outra, e uma rota intrusa de distância 1 causa estragos;

- **use-peer-dns** (yes/no) — aceitar os servidores DNS oferecidos? Veremos no Capítulo 12 por que um provedor frequentemente prefere no e seus próprios resolvedores.

O contrato de leitura: **configuração dinâmica se gerencia na origem**. O endereço D e a rota D não se editam; ajusta-se o dhcp-client que os criou (**release/renew** estão lá também, para forçar renegociação).

11.3 DHCP server: a LAN que se serve sozinha

O servidor tem três peças, e entender a divisão evita 90% das confusões:

1. **O pool** (/ip pool) — o estoque de endereços para emprestar:

```
/ip pool add name=pool-lan ranges=192.168.88.100-192.168.88.254
```

Repare que o pool **não** cobre a rede inteira: 192.168.88.1 é o roteador, e a faixa .2-.99 fica reservada para endereços fixos (impressoras, câmeras, APs) fora do alcance do sorteio.

2. **O network** (/ip dhcp-server network) — o que dizer aos clientes daquela rede (gateway, DNS, máscara):

```
/ip dhcp-server network add address=192.168.88.0/24 gateway=192.168.88.1 dns-server=1
```

3. **O server** (/ip dhcp-server) — quem atende o balcão, em qual interface, com qual pool e prazo:

```
/ip dhcp-server add name=dhcp-lan interface=ether2 address-pool=pool-lan lease-time=1
```

Pré-requisito que o RouterOS exige: o roteador precisa **ter endereço na rede servida** (192.168.88.1/24 na ether2) — afinal, ele é o gateway que o network anuncia. (Existe um assistente, /ip dhcp-server setup, que faz as três peças em sequência interativa; conhecendo as peças, você sabe o que ele monta — e sabe consertar quando alguém o usou errado.)

11.3.1 Leases: o livro de registros

Cada empréstimo vivo aparece em:

```
/ip dhcp-server lease print
```

```
[vitor@lab-r1] > /ip dhcp-server lease print
Columns: ADDRESS, MAC-ADDRESS, HOST-NAME, SERVER, STATUS
# ADDRESS          MAC-ADDRESS        HOST-NAME  SERVER   STATUS
0 192.168.88.254    08:00:27:3B:71:9E  lab-r2     dhcp-lan bound
```

Dois usos de operação:

- **Fixar o VIP:** `make-static` transforma o lease dinâmico em reserva — aquele MAC receberá **sempre** aquele endereço:

```
/ip dhcp-server lease make-static numbers=0
```

É assim que se garante endereço previsível para a câmera, o AP, o servidor do cliente corporativo — sem configurar nada no dispositivo;

- **lease-time com critério:** prazo curto (10m–1h) em redes de alta rotatividade (hotspot, eventos) devolve endereços rápido ao estoque; prazo longo (1d+) em LANs estáveis reduz tráfego de renovação e mantém endereços previsíveis. O padrão de 10 minutos do RouterOS é conservador — em LAN de cliente, 1h a 1d é o usual.

Aviso

Nunca ative um DHCP server “para testar” numa interface ligada a uma rede que **já tem** servidor DHCP — inclusive a sua rede de gerência do laboratório ou o escritório do provedor. Dois servidores na mesma rede = clientes recebendo configurações conflitantes, aleatoriamente — o clássico *rogue DHCP*, um dos defeitos mais confusos de diagnosticar em campo. (O RouterOS ajuda a caçá-los: `/ip dhcp-server alert`.)

O conceito mais importante do capítulo

DHCP é **estado emprestado com prazo**: o client materializa endereço e rota dinâmicos (flag D) que se gerenciam na origem; o server é pool + network + server, com o pool desenhado para **sobrar espaço fixo** fora do sorteio e leases estáticos para quem precisa de endereço previsível. Provedor opera os dois lados do balcão — e nunca deixa dois balcões na mesma rede.

11.4 Laboratório

11.4.1 Ambiente

Laboratório padrão (Capítulo 5): **lab-r1** e **lab-r2**, snapshot **base-limpa** (+ **seguranca-inicial** no r1, se preferir). Adaptador NAT (**ether3**) do **lab-r1** habilitado. Papéis: **lab-r1** será **cliente** DHCP na WAN (**ether3**, NAT do VirtualBox) e **servidor** DHCP na LAN (**ether2**, rede **lab-a**); **lab-r2** fará o papel do dispositivo do cliente final, pedindo endereço pela **ether2**.

11.4.2 Comandos passo a passo

Passo 1 — r1 como cliente (WAN). Em **lab-r1**:

```
/ip dhcp-client add interface=ether3 comment="WAN do lab (NAT VirtualBox)"
/ip dhcp-client print
```

Aguarde STATUS: bound e inspecione as criações dinâmicas:

```
/ip address print where dynamic
/ip route print where dst-address=0.0.0.0/0
/ping 8.8.8.8 count=3
```

O ping sai para a internet de verdade: WAN pronta, sem digitar um endereço.

Passo 2 — r1 como servidor (LAN). Ainda em lab-r1, as quatro peças na ordem: endereço do gateway, pool, network, server:

```
/ip address add address=192.168.88.1/24 interface=ether2 comment="LAN do lab"
/ip pool add name=pool-lan ranges=192.168.88.100-192.168.88.254
/ip dhcp-server network add address=192.168.88.0/24 gateway=192.168.88.1 dns-server=192.168.88.1
/ip dhcp-server add name=dhcp-lan interface=ether2 address-pool=pool-lan lease-time=1h
```

Passo 3 — r2 como “cliente da casa”. Em lab-r2:

```
/ip dhcp-client add interface=ether2 comment="simula dispositivo na LAN"
/ip dhcp-client print
```

```
[admin@lab-r2] > /ip dhcp-client print
Columns: INTERFACE, USE-PEER-DNS, ADD-DEFAULT-ROUTE, STATUS, ADDRESS
# INTERFACE  USE-PEER-DNS  ADD-DEFAULT-ROUTE  STATUS  ADDRESS
0 ether2     yes           yes               bound   192.168.88.254/24
```

O r2 recebeu endereço **do seu servidor** — DORA completo dentro do seu laboratório. Confira o que veio junto (/ip route print: rota padrão via 192.168.88.1).

Passo 4 — o livro de registros. De volta a lab-r1:

```
/ip dhcp-server lease print
```

O lease do r2 está lá, com MAC e hostname. Fixe-o e renove do lado do cliente para ver a reserva funcionando:

```
/ip dhcp-server lease make-static [find mac-address~":"]
/ip dhcp-server lease print
```

Em lab-r2, force a renegociação e confirme que o endereço **se manteve**:

```
/ip dhcp-client release numbers=0
/ip dhcp-client print
```

11.4.3 Verificação

1. Em lab-r1: dhcp-client bound na ether3, /ping 8.8.8.8 com 0% de perda;
2. Em lab-r1: /ip pool print, /ip dhcp-server network print e /ip dhcp-server print mostram as três peças com os nomes do laboratório;
3. Em lab-r2: endereço dinâmico 192.168.88.x na ether2 e rota padrão D via 192.168.88.1;
4. Em lab-r1: o lease aparece **sem** a flag D após o make-static (virou configuração), e o release/renew do r2 devolveu o mesmo endereço.

11.4.4 Desafios

1. Por que o pool (.100-.254) não começa no .2? Dê dois exemplos de equipamentos de um cliente corporativo que você colocaria na faixa fixa.
2. lab-r2 recebeu rota padrão via 192.168.88.1 — mas o r2 é um roteador do **seu** laboratório, com a própria rota padrão via enlace lab-a em outros capítulos. Que parâmetro do dhcp-client evitaria a rota intrusa, e quando você o usaria em produção?
3. No lab-r1, o que acontece se você apontar o address-pool do server para um pool de **outra** rede (ex.: 10.50.0.100-10.50.0.200)? Raciocine antes; se testar, observe o /log print e desfaça.
4. Um cliente reclama: “de vez em quando meu computador pega IP 169.254.x.x e a internet some”. O que esse endereço significa e quais as duas causas mais prováveis do lado do provedor?

Solução dos desafios

1. A faixa fora do pool é o território dos endereços **previsíveis**: o .1 é o gateway; .2-.99 ficam para leases estáticos e configurações manuais — ex.: a impressora de rede e o DVR de câmeras do cliente corporativo (ambos precisam de endereço fixo para port forwarding, assunto do Volume 3). Se o pool cobrisse tudo, um lease dinâmico poderia colidir com um fixo configurado à mão.
2. add-default-route=no no dhcp-client. Em produção: qualquer equipamento que usa uma WAN DHCP apenas como **serviço secundário** — ex.: um concentrador que recebe uma ONU de backup, um roteador interno que pega endereço de gerência via DHCP — e cuja rota padrão verdadeira vem de outro lugar (estática ou OSPF/BGP).
3. O server entrega um endereço do pool, mas o **network** continua sendo o 192.168.88.0/24 — e um cliente com endereço 10.50.0.x, gateway 192.168.88.1, fica incomunicável (gateway fora da própria rede). O RouterOS registra reclamações no log. Moral: pool, network e o endereço do roteador na interface precisam contar **a mesma história**.

4. 169.254.0.0/16 é *link-local* (APIPA): o sistema se autoendereço quando **nenhum servidor DHCP respondeu** dentro do prazo. Causas prováveis: (a) o DHCP server do CPE/concentrador fora do ar ou sem pool disponível (estoque esgotado — pool pequeno demais para a rotatividade); (b) problema de camada 2 entre o cliente e o servidor (porta, VLAN, rádio) engolindo o broadcast do Discover. Em ambas, o DORA nunca completa.

11.5 Resumo

- DHCP = **DORA** (Discover, Offer, Request, Ack) e endereço **emprestado** com prazo (`lease-time`) — renovado na metade do prazo (Figura 11.1).
- **Client** (`/ip dhcp-client`): WAN automática; controle o que aceita com `add-default-route` e `use-peer-dns`; o que ele cria tem flag D e se gerencia na origem.
- **Server** = três peças: `pool` (estoque, **menor** que a rede), `network` (gateway/DNS anunciados) e `server` (interface + pool + prazo) — mais o endereço do próprio roteador na interface.
- **Leases**: `print` é o livro de registros; `make-static` reserva endereço por MAC; `lease-time` curto para rotatividade, longo para estabilidade.
- Dois servidores na mesma rede = caos (*rogue DHCP*); há `/ip dhcp-server alert` para caçar intrusos.

No próximo capítulo, o terceiro serviço que toda LAN espera do roteador: **DNS** — resolver nomes, cachear respostas e as decisões de segurança que vêm junto.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip dhcp-client`, `/ip dhcp-server` (com o assistente `setup` e os alertas de rogue server) e `/ip pool`. O funcionamento do protocolo (DORA, renovação, opções) está em Tanenbaum; Feamster; Wetherall (2021). As referências completas, em ABNT, estão no apêndice [Referências](#).

12 DNS no RouterOS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o papel do DNS e o caminho de uma consulta — do dispositivo do cliente ao resolvidor upstream, passando pelo cache do roteador;
- Configurar o resolvidor do RouterOS (`/ip dns`): servidores upstream, cache e o significado exato de `allow-remote-requests=yes`;
- Reconhecer o risco do **open resolver** — por que um roteador respondendo DNS para a internet inteira é um problema seu e dos outros;
- Usar **entradas estáticas** para nomes internos da rede do provedor (e entender seus limites como ferramenta de bloqueio);
- Decidir entre os DNS da operadora (`use-peer-dns`), resolvidores públicos e resolvidor próprio.

12.1 O catálogo telefônico da internet

Nenhum cliente digita 142.250.79.78; ele digita `google.com`. O **DNS** (*Domain Name System*) é o serviço que traduz nomes em endereços — e para o assinante ele **é** a internet: quando o DNS falha, “a internet caiu”, mesmo com o link perfeito. Metade dos chamados de “internet lenta” de um provedor são, na verdade, DNS lento. Por isso este capítulo é curto no protocolo e longo na operação.

O caminho de uma consulta na rede que estamos montando (Figura 12.1):

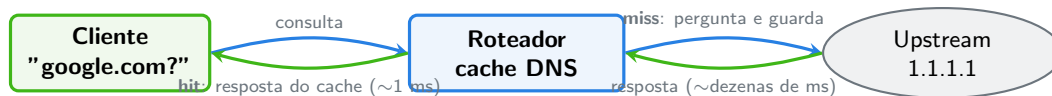


Figura 12.1: O roteador como intermediário com memória: cache hit responde na hora; cache miss consulta o upstream e guarda para o próximo.

O RouterOS não é um resolvidor recursivo completo (ele não sai caçando a resposta pela hierarquia do DNS); ele é um **encaminhador com cache** (*caching forwarder*): repassa a consulta a servidores upstream que você define e **guarda a resposta** pelo tempo indicado (TTL). Esse cache é o que interessa ao provedor: a segunda consulta a `google.com` sai do POP em um milissegundo, sem viajar até o upstream.

12.2 Configurando o resolvedor

Tudo mora num menu só:

```
/ip dns set servers=1.1.1.1,8.8.8.8
/ip dns print
```

```
[vitor@lab-r1] > /ip dns print
      servers: 1.1.1.1,8.8.8.8
dynamic-servers:
allow-remote-requests: no
      cache-size: 2048KiB
      cache-max-ttl: 1w
      cache-used: 32KiB
```

Os campos que decidem a operação:

- **servers** — os upstreams **estáticos** que você escolheu;
- **dynamic-servers** — os que chegaram sozinhos pelo `use-peer-dns=yes` do DHCP client (Capítulo 11). Se ambos existem, o RouterOS usa todos — fonte clássica de comportamento “misterioso”: você configura 1.1.1.1 e as consultas continuam indo para o DNS lento da operadora. Decida **um** dono da verdade: ou `use-peer-dns=no` no client, ou nenhum servidor estático;
- **allow-remote-requests** — o interruptor mais importante do menu, a seguir;
- **cache-size/cache-max-ttl** — o tamanho e a validade máxima do cache; os padrões servem até redes médias.

12.2.1 allow-remote-requests: servidor, e para quem?

Com `allow-remote-requests=no`, o resolvedor só atende o **próprio roteador** (pings e ferramentas locais). Com `yes`, o roteador vira **servidor DNS**: passa a ouvir na porta 53 e responder consultas de terceiros — e é isso que queremos para a LAN: foi por essa razão que o DHCP do Capítulo 11 anunciou `dns-server=192.168.88.1` aos clientes.

O problema: o RouterOS ouve na porta 53 de **todas** as interfaces — a LAN e a WAN. Um roteador com IP público e `allow-remote-requests=yes` sem proteção é um **open resolver**: a internet inteira pode usá-lo como DNS. Isso significa banda e CPU suas servindo estranhos e, pior, participação involuntária em **ataques de amplificação** — o atacante envia consultas pequenas com endereço de origem forjado (o da vítima), e seu roteador despeja respostas grandes na vítima. Seu provedor vira artilharia alheia, e o seu IP entra em listas de bloqueio.

Aviso

`allow-remote-requests=yes` num roteador com interface pública **exige** regras de firewall bloqueando a porta 53 (UDP e TCP) vinda da WAN. Ainda não temos firewall — ele é o assunto do Volume 3, e este é um dos primeiros buracos que fecharemos lá (Capítulo 17). No laboratório estamos seguros: a “WAN” NAT do VirtualBox não é alcançável de fora.

12.3 Entradas estáticas: o DNS interno do provedor

O menu `/ip dns static` — nosso campo de treino no Capítulo 9 — resolve nomes **antes** de consultar o upstream. Usos legítimos de provedor:

```
/ip dns static add name=core-r1.provedor.lan address=10.255.0.1 comment="gerencia core"
/ip dns static add name=noc.provedor.lan address=10.255.0.50
```

Pronto: a equipe digita `ssh core-r1.provedor.lan` em vez de decorar endereços de gerência — um “DNS interno” de custo zero, servido pelo roteador que a equipe já usa. As entradas aceitam expressão regular no nome (`regexp=`), TTL próprio e tipos além de A (AAAA, CNAME, TXT...).

E o uso tentador que exige honestidade: **bloqueio por DNS** (apontar `name=site-indesejado.com` para 127.0.0.1). Funciona como filtro de conveniência — rede de escritório, controle parental básico — e falha como segurança: qualquer dispositivo com DNS alternativo configurado (ou DoH, o DNS sobre HTTPS dos navegadores modernos) passa por cima sem esforço. Bloqueio de verdade se faz no firewall, e mesmo lá tem limites — Volume 3.

O conceito mais importante do capítulo

No provedor, o DNS é **serviço de infraestrutura**: escolha upstreams rápidos e confiáveis (e um só dono da verdade — estático **ou** `use-peer-dns`), sirva a LAN com cache local (`allow-remote-requests=yes`) e **nunca** deixe a porta 53 aberta para a WAN. DNS rápido = “internet rápida” na percepção do cliente; DNS aberto = seu roteador trabalhando para atacantes.

12.4 Laboratório

12.4.1 Ambiente

Continuação direta do laboratório do Capítulo 11 (se você o desfez, repita os Passos 1–3 de lá): `lab-r1` com WAN NAT (`ether3`) e DHCP server na LAN (`ether2`); `lab-r2` como cliente DHCP na `ether2`.

12.4.2 Comandos passo a passo

Passo 1 — um só dono da verdade. O dhcp-client da WAN trouxe DNS dinâmico (use-peer-dns=yes). Vamos assumir o controle: upstreams explícitos e nada de dinâmico:

```
/ip dhcp-client set [find interface=ether3] use-peer-dns=no
/ip dns set servers=1.1.1.1,8.8.8.8
/ip dns print
```

Confira: `servers` com os dois endereços, `dynamic-servers` vazio.

Passo 2 — o roteador resolve (para si). Teste o resolvedor local:

```
/resolve mikrotik.com
```

```
[vitor@lab-r1] > /resolve mikrotik.com
159.148.147.196
```

E veja a resposta guardada no cache:

```
/ip dns cache print where name="mikrotik"
```

Passo 3 — o roteador serve a LAN. Ligue o modo servidor:

```
/ip dns set allow-remote-requests=yes
```

Em `lab-r2` (que recebeu `dns-server=192.168.88.1` via DHCP no capítulo anterior — confira com `/ip dns print`, campo `dynamic-servers`), resolva um nome **através do r1**:

```
/ping google.com count=3
```

O ping traduz o nome (via r1) e sai pela rota padrão (via r1) — o mini-provedor do laboratório está completo até a camada de nomes.

Passo 4 — sentir o cache. Em `lab-r1`, meça a diferença entre miss e hit. Primeiro limpe o cache, depois resolva o mesmo nome duas vezes:

```
/ip dns cache flush
/resolve wikipedia.org
/resolve wikipedia.org
```

A primeira consulta demora dezenas de milissegundos (foi ao upstream); a segunda é instantânea (cache). Veja o estoque acumulado:

```
/ip dns cache print
```

Passo 5 — DNS interno. Crie o nome de gerência do próprio laboratório e use-o de lab-r2:

```
/ip dns static add name=r1.lab address=192.168.88.1 comment="gerencia lab"
```

Em lab-r2:

```
/ping r1.lab count=2
```

O nome interno resolve — servido pela entrada estática do r1, sem consultar upstream nenhum.

12.4.3 Verificação

1. `/ip dns print` em lab-r1: `servers=1.1.1.1,8.8.8.8`, `dynamic-servers` vazio, `allow-remote-requests=yes`;
2. `/ping google.com` funciona em lab-r2 (nome + rota, tudo via r1);
3. `/ip dns cache print` em lab-r1 acumula entradas conforme você resolve nomes — e flush as zera;
4. `/ping r1.lab` responde em lab-r2 (entrada estática servida à LAN).

12.4.4 Desafios

1. Explique a sequência de consulta quando lab-r2 pede `r1.lab` ao r1: por que a entrada estática responde mesmo com o upstream configurado? E se existisse um `r1.lab` real na internet?
2. No cache (`/ip dns cache print detail`), localize o TTL de uma entrada. Resolva o mesmo nome de novo e observe o TTL. O que está acontecendo, e por que o `cache-max-ttl=1w` raramente “vale” na prática?
3. Um colega sugere: “aponte `use-peer-dns=yes` de volta e deixe também os servers estáticos — redundância!”. Por que essa “redundância” costuma piorar o serviço em vez de melhorar?
4. Projete (só em texto) o DNS de um provedor de 5 mil assinantes: o que você anunciaria via DHCP/PPPoE aos clientes, onde ficaria o cache, e que upstream escolheria? Considere: latência, privacidade dos dados de consulta dos clientes e o que acontece se o upstream cair.

💡 Solução dos desafios

1. A ordem do resolvidor é: **estáticas primeiro**, depois cache, depois upstream. `r1.lab` casa com a estática e a consulta morre ali — rápido e sem vazamento. Se existisse um `r1.lab` público, os clientes do seu roteador **nunca** o veriam: a estática o “sombreia” (é exatamente assim que o bloqueio por DNS funciona — e por que entradas estáticas erradas causam defeitos misteriosos).
2. O TTL exibido **decrece** a cada segundo: é o prazo restante da resposta, herdado do TTL original definido pelo dono do domínio (minutos a horas, tipicamente). O `cache-max-ttl=1w` é apenas um **teto**: o RouterOS guarda pelo menor entre o TTL da resposta e o teto — e como quase nenhum domínio publica TTL de uma semana, o teto quase nunca decide.
3. O RouterOS distribui consultas entre **todos** os servidores (estáticos + dinâmicos), sem preferência por qualidade: parte das consultas irá ao DNS da operadora mesmo com upstreams melhores configurados. Resultado: desempenho inconsistente (ora 5 ms, ora 80 ms) e diagnóstico difícil. Redundância boa é ter **dois upstreams bons** nos **servers** — não misturar bons e ruins.
4. Desenho razoável: anunciar aos assinantes **o IP do próprio provedor** (o cache no concentrador/POP, ou um par de resolvidores dedicados — dois CHRs com `allow-remote-requests=yes` atrás de firewall); upstreams `1.1.1.1+8.8.8.8` (latência baixa e independência entre si) — ou resolvidor recursivo próprio (ex.: Unbound em servidor Linux) se a privacidade das consultas dos clientes for requisito, pois upstream público vê tudo que seus clientes resolvem. Queda de upstream: com dois independentes, o cache segura o grosso; recursivo próprio elimina a dependência. O que **não** fazer: anunciar upstream público direto aos clientes (perde-se o cache local e o controle).

12.5 Resumo

- O RouterOS é um **encaminhador com cache**: estáticas → cache → upstream (Figura 12.1); cache local faz a “internet do cliente” parecer rápida.
- Um só dono da verdade: **servers** estáticos **ou** `use-peer-dns` — os dois juntos misturam upstreams bons e ruins.
- `allow-remote-requests=yes` liga o servidor DNS para a LAN — e abre a porta 53 em **todas** as interfaces: sem firewall na WAN, você é um **open resolver** servindo ataques de amplificação.
- `/ip dns static`: DNS interno de gerência (ótimo), bloqueio de conteúdo (frágil — DoH e DNS alternativo passam por cima).
- `/resolve`, `/ip dns cache print` e `flush` são o kit de diagnóstico.

No próximo capítulo, a última peça que falta ao nosso roteador de borda: **NAT masquerade** — como uma LAN inteira de endereços privados navega na internet com um único IP público.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip dns` (resolvedor, cache, entradas estáticas e tipos de registro). Sobre o funcionamento do DNS em profundidade (hierarquia, recursão, TTLs), Tanenbaum; Feamster; Wetherall (2021) é a referência. O risco de open resolvers e amplificação é documentado pelo CERT.br em cert.br/docs/ (recomendações para provedores brasileiros). As referências completas, em ABNT, estão no apêndice [Referências](#).

13 NAT masquerade

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar por que uma LAN de endereços privados (Rekhter et al., 1996) não navega na internet sem tradução — analisando o caminho de **volta** do pacote;
- Descrever o que o NAT de origem faz com cada pacote e como o roteador “lembra” cada conexão traduzida;
- Criar a regra clássica de **masquerade** e ler a tabela de conexões que ela alimenta;
- Diferenciar **action=masquerade** de **action=src-nat** — e saber quando o IP fixo vale a troca;
- Reconhecer os limites: NAT não é firewall, quebra a conectividade fim a fim e tem efeitos colaterais que reaparecerão no CGNAT (Volume 9).

13.1 O problema: cartas sem remetente válido

Nosso laboratório terminou o capítulo anterior num estado curioso: **lab-r2** resolve nomes, tem rota padrão via **lab-r1**, o **lab-r1** alcança a internet... e ainda assim um **/ping 8.8.8.8** de **lab-r2** **falha**. Rota existe, DNS existe — o que falta?

Siga o pacote. Ele sai de **lab-r2** com origem 192.168.88.254, o **r1** o encaminha pela WAN, e ele até pode chegar ao destino. O problema é a **resposta**: ela seria endereçada a 192.168.88.254 — um endereço privado (Rekhter et al., 1996), que **não existe na internet**. Nenhum roteador do mundo tem rota para ele (e os provedores descartam esses destinos). A resposta morre no caminho; para quem pinga, é como se a internet não respondesse. É a lição do Capítulo 10 levada ao extremo: o caminho de volta é metade do problema — e na internet pública, para origem privada, **não há volta**.

A solução tem nome pomposo — *Network Address Translation* de origem (**srcnat**) — e lógica simples: na saída, o roteador **troca o remetente**.

13.2 Como a máscara funciona

Com NAT de origem ativo, cada pacote que sai da LAN para a WAN passa por uma transformação (Figura 13.1):

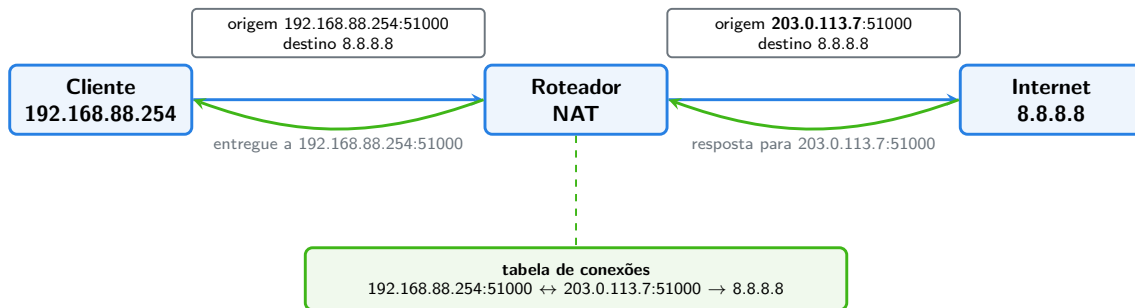


Figura 13.1: O NAT de origem em ação: troca de remetente na ida, anotação na tabela de conexões, tradução desfeita na volta.

Três observações fazem o mecanismo inteiro:

1. **Na ida**, a origem privada é substituída pelo endereço **público** do roteador (e, se preciso, a porta de origem também muda — é assim que centenas de clientes compartilham um IP sem misturar as respostas);
2. **A memória**: cada tradução vira uma linha na **tabela de conexões** do roteador — quem pediu, por qual porta, para onde. Essa tabela (*connection tracking*) é uma peça central do RouterOS e será a estrela do primeiro capítulo do Volume 3;
3. **Na volta**, a resposta chega ao IP público, o roteador consulta a tabela, desfaz a tradução e entrega ao cliente certo.

💡 Analogia para guardar

O NAT é a **portaria do prédio**: as cartas dos moradores saem com o endereço do prédio como remetente, e o porteiro anota num caderno quem mandou o quê. A resposta chega ao prédio, o porteiro consulta o caderno e entrega no apartamento certo. Ninguém lá fora conhece os apartamentos — conhecem o prédio.

13.3 A regra no RouterOS

O NAT vive em `/ip firewall nat` — sim, dentro do firewall: NAT e filtro compartilham o motor de classificação de pacotes (chains, matchers) que o Volume 3 destrinchará. Por ora, o essencial: regras de NAT de origem vivem na chain **srcnat**, e a regra clássica do roteador de borda é uma linha:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=masquerade comment="NAT da L
```

Lendo por partes:

- **chain=srcnat** — “esta regra trata da origem dos pacotes que saem”;

- `out-interface=ether3` — só pacotes saindo pela WAN. É o **matcher**: a condição que seleciona quem sofre a ação. Sem ele, você mascararia até o tráfego interno;
- `action=masquerade` — “troque a origem pelo endereço **que estiver** na interface de saída”.

13.3.1 masquerade ou src-nat?

O `masquerade` descobre o endereço público sozinho, a cada conexão — feito sob medida para WAN de endereço **dinâmico** (DHCP, PPPoE): mudou o IP, a regra continua certa. A alternativa é dizer o endereço explicitamente:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=src-nat to-addresses=203.0.1
```

`src-nat` com IP fixo é mais leve (não consulta a interface a cada conexão) e mais previsível — a escolha natural quando o IP público é estático, e obrigatória quando se traduz para um endereço que não é o da interface (situação comum no CGNAT, Volume 9). A regra de bolso: **WAN dinâmica → masquerade; IP fixo → src-nat**.

Um efeito colateral do `masquerade` que vale conhecer desde já: quando o link cai e volta (ou o IP muda), as conexões antigas na tabela apontam para o endereço velho — e ficam zumbis até expirar, causando aqueles minutos de “internet meio quebrada” depois que o link retorna. Guarde o sintoma; o remédio (limpar conexões, ajustar timeouts) aparece com o connection tracking no Volume 3.

13.3.2 O que o NAT não é

- **NAT não é firewall.** A tabela de conexões só deixa *voltar* o que *saiu* — o que parece proteção. Mas qualquer serviço do próprio roteador (o DNS do capítulo anterior, o WinBox...) continua exposto na WAN, e regras de NAT mal escritas abrem caminhos inesperados. Proteção é assunto de **filter rules** — Volume 3, sem atalhos;
- **NAT quebra o fim a fim.** Ninguém de fora consegue *iniciar* conexão com um cliente atrás do NAT — ótimo para esconder, péssimo para servir. Publicar um serviço interno (o DVR do cliente, o servidor da empresa) exigirá a tradução inversa, **dstnat** — também Volume 3;
- **NAT em escala tem custo:** portas são finitas (~64 mil por IP público), e milhares de assinantes por IP esbarram nisso — o mundo do CGNAT, com suas obrigações legais brasileiras, é o Volume 9.

! O conceito mais importante do capítulo

NAT de origem = **troca de remetente + memória por conexão**. A regra `chain=srcnat out-interface=WAN action=masquerade` é uma linha, mas o que ela cria é estado: cada conexão traduzida vive na tabela de conexões do roteador. Entender essa tabela — não a regra — é o que separa quem configura NAT de quem diagnostica

NAT. (E é por ela que o Volume 3 começa.)

13.4 Laboratório

13.4.1 Ambiente

Continuação direta dos capítulos Capítulo 11 e Capítulo 12: lab-r1 com WAN NAT (ether3, dhcp-client) e LAN (ether2, 192.168.88.1/24 + DHCP server); lab-r2 como cliente DHCP da LAN. É o estado em que o capítulo anterior terminou.

13.4.2 Comandos passo a passo

Passo 1 — constatar a falha. Em lab-r2:

```
/ping 8.8.8.8 count=3
```

```
[admin@lab-r2] > /ping 8.8.8.8 count=3
  SEQ HOST      SIZE TTL TIME   STATUS
    0 8.8.8.8                timeout
    1 8.8.8.8                timeout
    2 8.8.8.8                timeout
sent=3 received=0 packet-loss=100%
```

Repare: **timeout**, não no **route to host**. Rota existe (a padrão via r1); o que não existe é resposta — o sintoma clássico de origem privada sem tradução.

Passo 2 — a regra. Em lab-r1:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=masquerade comment="NAT da LAN"
/ip firewall nat print
```

```
[vitor@lab-r1] > /ip firewall nat print
Flags: X - disabled, I - invalid; D - dynamic
 0    ;;; NAT da LAN para a WAN
      chain=srcnat action=masquerade out-interface=ether3
```

Passo 3 — o milagre. Em lab-r2, repita:

```
/ping 8.8.8.8 count=3
/ping google.com count=3
```

Zero de perda nos dois — endereço, rota, DNS e agora NAT: o pacote completo do acesso à internet, montado peça por peça por você.

Passo 4 — ver a memória. Enquanto o `lab-r2` gera tráfego (deixe um `/ping 8.8.8.8` rodando sem `count`), veja em `lab-r1` a tabela de conexões:

```
/ip firewall connection print where dst-address~"8.8.8.8"
```

```
[vitor@lab-r1] > /ip firewall connection print where dst-address~"8.8.8.8"
Columns: PROTOCOL, SRC-ADDRESS, DST-ADDRESS, REPLY-SRC-ADDRESS, REPLY-DST-ADDRESS
# PROTOCOL SRC-ADDRESS      DST-ADDRESS  REPLY-SRC-ADDRESS  REPLY-DST-ADDRESS
0 icmp      192.168.88.254    8.8.8.8      8.8.8.8            10.0.2.15
```

A linha conta a história completa: origem real (192.168.88.254), e a resposta (REPLY-DST) endereçada ao IP da WAN (10.0.2.15) — a tradução, anotada. Interrompa o ping no r2 (Ctrl+C).

13.4.3 Verificação

1. O ping do Passo 1 falhou com `timeout` (e você sabe explicar por que não foi `no route to host`);
2. `/ip firewall nat print` mostra a regra única de masquerade com o comentário;
3. Após a regra, `lab-r2` pinga IP e nome com 0% de perda;
4. A tabela de conexões exibiu a tradução (origem privada → endereço da WAN).

13.4.4 Desafios

1. No Passo 4, a resposta vai para 10.0.2.15 — que também é um endereço privado! Por que o NAT funciona mesmo assim no laboratório? (Pense em quantas “portarias” existem entre o `lab-r2` e o 8.8.8.8.)
2. Remova o matcher: crie a regra só com `chain=srcnat action=masquerade` (sem `out-interface`), desabilitando a original. O laboratório continua funcionando — então qual é o perigo? Analise o que acontece com o tráfego `lab-r2 → 192.168.56.1` (gerência) e por que “funciona” não é “está certo”. Desfaça ao final.
3. Sua WAN ganhou IP fixo 203.0.113.7. Escreva a regra `src-nat` equivalente e explique **dois** ganhos práticos sobre o masquerade nesse cenário.
4. Um cliente corporativo atrás do seu NAT quer acessar as câmeras dele a partir do celular (de fora). Com o que você aprendeu até aqui, isso é possível? O que falta, conceitualmente — e em que volume mora a resposta?

Solução dos desafios

1. Porque há **NAT em cascata**: o `lab-r1` traduz 192.168.88.254 → 10.0.2.15, e o NAT do VirtualBox traduz 10.0.2.15 → IP do seu computador, que por sua vez

pode estar atrás do NAT do seu roteador doméstico (→ IP público da sua casa). Cada portaria anota no próprio caderno e desfaz na volta. É também um retrato honesto da internet IPv4 atual — e do que o CGNAT (Volume 9) faz em escala de provedor.

2. Sem `out-interface`, a regra mascara **tudo que sai por qualquer interface** — inclusive o tráfego para a rede de gerência: o 192.168.56.1 passa a ver conexões vindas de 192.168.56.11 (o endereço do r1 naquela interface) em vez do endereço real do r2. Você perde a rastreabilidade interna (logs, permissões por origem) e cria defeitos finíssimos de diagnosticar. “Funciona” — correto: NAT se aplica **na fronteira**, não no meio da casa.

3.

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=src-nat to-addresses=203.0
```

Ganhos: (a) menos trabalho por conexão (não resolve o IP da interface a cada vez — relevante em concentrador com milhares de conexões novas por segundo); (b) previsibilidade — o endereço de origem público é sempre o mesmo, o que facilita liberações em firewalls de terceiros, reputação de IP e diagnóstico.

4. Não é possível ainda: o NAT de origem só cria traduções para conexões **iniciadas de dentro**; alguém de fora não tem como alcançar a câmera (o fim a fim está quebrado, por design). Falta a tradução inversa — **dstnat/port forwarding**: “o que chegar ao meu IP público na porta X, entregue ao IP interno Y”. É o Capítulo 19, no Volume 3 — junto com o firewall que impede que esse portão aberto vire problema.

13.5 Resumo

- LAN privada não navega sem tradução: a **resposta** para um endereço RFC 1918 não tem rota na internet — o sintoma é **timeout** com rota existente.
- NAT de origem: **troca o remetente na saída, anota na tabela de conexões, desfaz na volta** (Figura 13.1) — portas de origem distinguem os clientes que compartilham o IP.
- A regra clássica: `chain=srcnat out-interface=WAN action=masquerade` — matcher primeiro, ação depois. **WAN dinâmica** → **masquerade**; **IP fixo** → **src-nat** (mais leve e previsível).
- NAT **não é firewall** (os serviços do roteador seguem expostos), quebra o fim a fim (publicar serviço = `dstnat`, Volume 3) e em escala vira CGNAT (Volume 9).
- `/ip firewall connection print` é a janela para a memória do NAT — e a ponte para o connection tracking do Volume 3.

No próximo capítulo, nada de conceito novo: o **projeto integrador** do volume — um roteador de borda completo, montado do zero, peça por peça, com as mãos.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall nat` (chains, actions e matchers) e a tabela de conexões. O RFC 1918 (Rekhter et al., 1996) define os endereços privados que tornam o NAT necessário. Tanenbaum; Feamster; Wetherall (2021) discute NAT, seus efeitos no fim a fim e a transição para IPv6 — nosso assunto no Volume 9. As referências completas, em ABNT, estão no apêndice [Referências](#).

14 Projeto: roteador de borda

i Objetivos de aprendizagem

Este capítulo não introduz conceitos novos — ele **integra** tudo do volume. Ao final, você será capaz de:

- Transformar uma “ordem de serviço” em uma sequência de configuração planejada, executada e verificada;
- Montar um roteador de borda completo, do equipamento zerado ao cliente navegando: segurança inicial → WAN → LAN → DHCP → DNS → NAT;
- Verificar o serviço em camadas (enlace → endereço → rota → tradução → nome), na ordem que o diagnóstico profissional usa;
- Entregar o serviço **documentado**: export comentado e backup baixado.

14.1 A ordem de serviço

Situação real de qualquer semana num provedor: um cliente corporativo contratou um link dedicado, e você vai configurar o roteador que fica na sede dele. A ordem de serviço diz:

Cliente: Padaria Pão Quente (matriz) **WAN:** via ONU em bridge, endereço por DHCP do concentrador **LAN:** 192.168.10.0/24, roteador no .1 **Serviços:** DHCP na LAN (faixa .100–.199), DNS com cache local, internet para todos os dispositivos **Reservas:** impressora fiscal sempre no .10 **Padrão da empresa:** checklist de segurança, identidade `cli-paoquente-r1`, export e backup arquivados ao final

A Figura 14.1 mostra o que a ordem descreve:

Antes de digitar qualquer coisa, o hábito profissional: **planejar a sequência**. A ordem importa — cada camada depende da anterior, e é a mesma ordem em que se diagnostica depois:

Tabela 14.1: O plano de execução

#	Etapa	Capítulo de referência
0	Base limpa + identidade + segurança	5
1	WAN: DHCP client	Capítulo 11

#	Etapa	Capítulo de referência
2	LAN: endereço do gateway	Capítulo 10
3	DHCP server (pool, network, server) + reserva	Capítulo 11
4	DNS: upstream + cache para a LAN	Capítulo 12
5	NAT masquerade	Capítulo 13
6	Verificação em camadas	todos
7	Documentar: export + backup para fora	3

💡 Analogia para guardar

Configurar rede é construir prédio: fundação (acesso seguro), estrutura (endereços e rotas), instalações (DHCP, DNS, NAT) e a vistoria (verificação em camadas). Ninguém azuleja antes de rebocar — e quem pula a vistoria descobre o vazamento pelo telefone do cliente.

14.2 Laboratório

Este capítulo é o laboratório — o texto acima era o projeto; agora, mãos à obra.

14.2.1 Ambiente

Laboratório padrão (Capítulo 5), com os papéis do projeto:

- **lab-r1** é o roteador do cliente (**cli-paoquente-r1**): restaure o snapshot **base-limpa** — o projeto começa do zero de verdade, sem aproveitar nada dos capítulos anteriores. Adaptador NAT (**ether3**) habilitado: ele fará o papel da ONU/concentrador do provedor;
- **lab-r2** é “um computador da padaria”: também em **base-limpa**, conectado pela **ether2** (rede **lab-a**).

Acesso inicial ao r1 pelo console da VM (equipamento zerado não tem gerência configurada — como na vida real).

14.2.2 Comandos passo a passo

Etapa 0 — base. Pelo console de **lab-r1**, o ritual do 5, condensado (adapte usuário e senha):

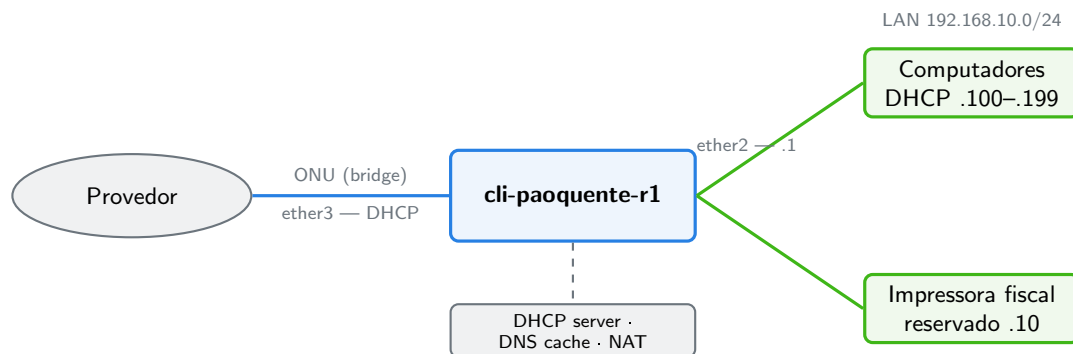


Figura 14.1: O projeto do capítulo: um roteador de borda de cliente corporativo, com WAN por DHCP e LAN completa.

```

/system identity set name=cli-paoquente-r1
/ip address add address=192.168.56.11/24 interface=ether1 comment="gerencia lab"
/user add name=vitor group=full password=Lab!Segura2026 comment="tecnico"
/user disable admin
/ip service disable telnet,ftp,www,api,api-ssl
/ip service set ssh address=192.168.56.0/24
/ip service set winbox address=192.168.56.0/24
/system clock set time-zone-name=America/Sao_Paulo

```

Daqui em diante, trabalhe por SSH (`ssh vitor@192.168.56.11`) — e lembre do Safe Mode (`Ctrl+X`) como cinto.

Etapas 1 — WAN. O link do provedor entrega endereço por DHCP:

```

/ip dhcp-client add interface=ether3 use-peer-dns=no comment="WAN via provedor"
/ip dhcp-client print

```

Espere `bound` e valide a camada: `/ping 8.8.8.8 count=3 do roteador` — a WAN funciona antes de existir LAN. (Repare no `use-peer-dns=no`: a ordem de serviço pede cache DNS com upstream nosso — um dono da verdade, Capítulo 12.)

Etapas 2 — LAN. O gateway da padaria:

```

/ip address add address=192.168.10.1/24 interface=ether2 comment="LAN cliente"

```

Etapas 3 — DHCP server. Pool (repare: `.100-.199`, deixando o resto fixo), `network` e `server` — e o NTP aproveitando a WAN nova:

```
/ip pool add name=pool-lan ranges=192.168.10.100-192.168.10.199
/ip dhcp-server network add address=192.168.10.0/24 gateway=192.168.10.1 dns-server=192.16
/ip dhcp-server add name=dhcp-lan interface=ether2 address-pool=pool-lan lease-time=1h
/system ntp client set enabled=yes
/system ntp client servers add address=a.st1.ntp.br
```

Etapas 4 — DNS. Upstreams e o modo servidor para a LAN:

```
/ip dns set servers=1.1.1.1,8.8.8.8 allow-remote-requests=yes
```

Etapas 5 — NAT. A máscara na fronteira:

```
/ip firewall nat add chain=srcnat out-interface=ether3 action=masquerade comment="NAT LAN-
```

Etapas 6 — o cliente chega. Em lab-r2 (fazendo papel de computador):

```
/system identity set name=pc-padaria
/ip dhcp-client add interface=ether2
/ip dhcp-client print
```

Recebeu 192.168.10.1xx? Então a prova final, na ordem das camadas — gateway, internet por IP, internet por nome:

```
/ping 192.168.10.1 count=2
/ping 8.8.8.8 count=2
/ping google.com count=2
```

Três pings limpos = serviço entregue. Falta a reserva da impressora: em lab-r1, fixe o lease do “computador” (na vida real seria o MAC da impressora) no .10:

```
/ip dhcp-server lease print
/ip dhcp-server lease make-static numbers=0
/ip dhcp-server lease set 0 address=192.168.10.10
```

Em lab-r2, remova (/ip dhcp-client release numbers=0) e confirme que veio o .10.

Etapas 7 — documentar. Em lab-r1:

```
/system backup save name=cli-paoquente-r1-entrega
/export file=cli-paoquente-r1-entrega
```

E, do seu computador, **para fora** (3):

```
scp vitor@192.168.56.11:cli-paoquente-r1-entrega.backup .  
scp vitor@192.168.56.11:cli-paoquente-r1-entrega.rsc .
```

Abra o `.rsc`: ele é o projeto inteiro em uma página — e o gabarito para o próximo cliente igual.

14.2.3 Verificação

A vistoria de entrega, em camadas (decore esta ordem — ela é o seu roteiro de diagnóstico para sempre):

1. **Enlace:** `/interface print` no `r1` — `ether2` e `ether3` com flag `R`;
2. **Endereço:** `/ip address print` — gerência, LAN `.1` e WAN dinâmica (`D`);
3. **Rota:** `/ip route print` — conectadas `DAd` + padrão `DAd` via provedor;
4. **Serviços:** `bound` no `dhcp-client`; `lease` do “`pc`” no `.10` (estático); `/ip dns print` com `upstreams` e `allow-remote-requests=yes`;
5. **Tradução:** regra de `masquerade` presente; `/ip firewall connection print` mostrando conexões do `192.168.10.10` traduzidas;
6. **Experiência do cliente:** os três pings do `lab-r2` (`gateway` → `IP` → `nome`) sem perdas;
7. **Documentação:** `.backup` e `.rsc` no seu computador.

14.2.4 Desafios

1. A padaria abriu uma filial — mesma ordem de serviço, mas LAN `192.168.20.0/24`. Liste **apenas** os comandos que mudam em relação ao projeto (e note quantos são: essa é a força do padrão).
2. O dono quer WiFi para os clientes **separado** da rede do caixa. Com as ferramentas deste volume (sem VLANs ainda — Volume 4), como você atenderia usando a `ether4` do roteador? Esboce os comandos das etapas 2–5 para essa segunda LAN (`192.168.99.0/24`) e diga o que essa solução **não** separa de verdade.
3. Simule o chamado clássico: em `lab-r1`, desabilite o `dhcp-client` da WAN (`/ip dhcp-client disable 0`) e, “sem saber disso”, diagnostique a partir do sintoma do cliente (“caiu a internet”) usando a vistoria em camadas. Em que camada o defeito aparece, e quais camadas continuam OK? Reabilite ao final.
4. O `.rsc` exportado serve de gabarito, mas aplicá-lo cru num equipamento novo tem **duas** lacunas (pense no que o `export` não carrega —
3) e **um** risco (pense no que é dinâmico no projeto). Quais?

Solução dos desafios

1. Mudam apenas os comandos com o endereço da LAN — identidade e cinco linhas:

```

/system identity set name=cli-paoquente-r2
/ip address add address=192.168.20.1/24 interface=ether2 comment="LAN cliente"
/ip pool add name=pool-lan ranges=192.168.20.100-192.168.20.199
/ip dhcp-server network add address=192.168.20.0/24 gateway=192.168.20.1 dns-server=192.
/ip dhcp-server lease set 0 address=192.168.20.10

```

Todo o resto (usuários, serviços, DNS, NAT, NTP) é idêntico — é isso que um `.rsc`-gabarito bem feito captura.

2. Segunda LAN na `ether4`: repetir as etapas com `address=192.168.99.1/24` `interface=ether4`, pool/network próprios, um segundo `/ip dhcp-server` na `ether4` — DNS e NAT existentes já atendem (o masquerade casa por `out-interface`, não por origem). O que **não** separa: tráfego entre as duas LANs — o roteador roteia entre elas livremente; um notebook no WiFi dos clientes alcança a impressora do caixa. Separação de verdade exige **firewall entre as redes** (Volume 3) — e, em escala, VLANs (Volume 4).

3. A vistoria denuncia na camada 2–3 da **WAN**: enlace OK (interfaces R), mas `/ip address print` perdeu o endereço D da `ether3` e `/ip route print` perdeu a rota padrão DAd — enquanto a LAN continua perfeita (cliente pega IP, pinga o gateway). Sintoma no cliente: “pego WiFi mas não navego”; no roteador: `bound` sumiu do `dhcp-client`. Camadas OK: 1 (enlaces), LAN inteira; defeito: endereço/rota da WAN. É o padrão de raciocínio que separa “trocar cabo por desespero” de diagnóstico.

4. Lacunas do export: (a) **senhas de usuários** não vêm — o `/user add` precisa ser refeito (ou mantido num `.rsc`-gabarito escrito à mão); (b) itens **dinâmicos** não são configuração: o lease estático da impressora vem, mas o `bound` da WAN depende do provedor real. O risco: o export carrega o `allowed-address=192.168.56.0/24` dos serviços — aplicado num cliente sem essa rede de gerência, você se tranca para fora no momento do import. Gabarito se **lê e adapta**, não se despeja.

14.3 Resumo

- Projeto profissional = **ordem de serviço** → **plano sequenciado** → **execução** → **vistoria** → **documentação** (Tabela 14.1) — nunca comandos soltos de memória.
- A sequência do roteador de borda: base segura → WAN (`dhcp-client`) → LAN (endereço) → DHCP server (pool menor que a rede + reservas) → DNS (um dono da verdade, cache para a LAN) → NAT (na fronteira) — validando **cada camada antes da seguinte**.
- A vistoria em camadas (enlace → endereço → rota → serviços → tradução → experiência) é o mesmo roteiro do **diagnóstico** — configurar bem é aprender a diagnosticar.
- Entrega termina com `.rsc` e `.backup` **fora** do equipamento — e o `.rsc` de hoje é o gabarito (adaptado, nunca despejado) do cliente de amanhã.

Fecha-se o Volume 2: você domina o terminal e constrói um roteador de borda completo. Mas ele está **nu**: DNS aberto na WAN, WinBox exposto, nada filtrando o forward. O

Volume 3 veste a armadura — firewall, do connection tracking ao hardening completo.

Bibliografia do capítulo

Este capítulo integra os anteriores; as referências são as deles — em especial a documentação oficial (MikroTik, 2026a) (páginas *First Time Configuration* e *Securing your router*, que espelham nossa sequência). Discher (2016) traz projetos guiados semelhantes. As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte III

Volume 3 — Firewall

15 Connection tracking

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que é o **connection tracking** (conntrack) e por que ele é a fundação de todo o firewall do RouterOS;
- Classificar pacotes pelos quatro **estados** de conexão — **new**, **established**, **related** e **invalid** — e prever qual estado um pacote receberá;
- Ler a tabela de conexões (`/ip firewall connection`): protocolos, timeouts, flags e o que cada coluna conta;
- Explicar como o conntrack enxerga protocolos “sem conexão” (UDP, ICMP) e protocolos com canais secundários (FTP);
- Dimensionar e ajustar o conntrack: tamanho máximo da tabela, timeouts e o custo de memória/CPU em equipamentos de provedor.

15.1 A memória que muda tudo

No Capítulo 13 você viu o NAT anotar cada tradução numa tabela e consultá-la na volta. Aquela tabela não pertence ao NAT: ela é o **connection tracking** — o subsistema do RouterOS que observa **todo** pacote roteado e mantém um registro vivo de cada conversa em andamento. O NAT é apenas seu primeiro cliente; o segundo, e mais importante, é o firewall que construiremos neste volume.

Por quê? Porque um firewall sem memória é obrigado a julgar cada pacote isoladamente — e pacotes isolados mentem. Um pacote vindo da internet para o seu cliente pode ser um ataque **ou** a resposta legítima de um site que o cliente acabou de acessar; olhando só o pacote, é impossível distinguir. Com memória, a pergunta muda de “este pacote parece bom?” para “**este pacote pertence a uma conversa que alguém de dentro começou?**” — uma pergunta que a tabela de conexões responde em microssegundos.

Esse é o conceito de **firewall com estado** (*stateful*), e é o modelo de praticamente toda regra que escreveremos: julgamos **conexões** na abertura, e os pacotes seguintes herdaram o julgamento.

💡 Analogia para guardar

O conntrack é a **recepcionista com caderno**: a primeira vez que alguém sai para uma reunião, ela anota (conexão **new**). Quando a resposta chega, ela olha o caderno — “ah, é da reunião do fulano” (**established**) — e deixa passar sem interrogatório. Entregador que chega junto com a encomenda esperada (**related**) também entra. Estranho que aparece dizendo “estou voltando de uma reunião” que não está no caderno (**invalid**)? Porta.

15.2 Os quatro estados

Cada pacote que o conntrack examina recebe um **connection state** — e é sobre esses estados que as regras do firewall operam:

Tabela 15.1: Os estados de conexão

Estado	Significado	Tratamento típico
new	primeiro pacote de uma conversa nova	julgar com as regras
established	pertence a uma conversa já registrada	aceitar cedo
related	conversa nova, mas filha de uma registrada	aceitar cedo
invalid	não pertence a nada e não abre nada válido	descartar sempre

Vale detalhar os dois menos óbvios:

- **related**: alguns protocolos abrem canais secundários — o FTP clássico negocia a transferência numa conexão e move os dados por outra; o ICMP de erro (“destino inalcançável”) é resposta a uma conexão que já existe. O conntrack usa *helpers* (em `/ip firewall service-port`) para reconhecer esses filhotes e marcá-los **related** — sem isso, seriam **new** vindos de fora e morreriam no filtro;
- **invalid**: o estado do lixo — pacotes TCP no meio de uma conversa que a tabela não conhece (sobras de conexão expirada, pacotes fora de ordem extremos, varreduras e ataques). **invalid não** é o mesmo que **new**: um SYN legítimo é **new**; um ACK órfão é **invalid**. A regra universal: descartar.

A consequência prática, que será o esqueleto de todas as nossas chains no Capítulo 16:

```
accept established,related    # o grosso do tráfego, barato
drop invalid                  # o lixo
# ... e só o que for NEW é julgado pelas regras seguintes
```


Essa ordem não é estética — é **desempenho**. Numa rede de provedor, mais de 95% dos pacotes pertencem a conexões já estabelecidas: eles casam com a primeira regra e saem do firewall imediatamente, sem atravessar a lista inteira. As regras caras julgam apenas o primeiro pacote de cada conversa.

15.3 Lendo a tabela

A tabela mora em `/ip firewall connection` — você já a espiou no NAT; agora com olhos completos:

```
/ip firewall connection print detail
```

```
[vitor@lab-r1] > /ip firewall connection print detail
Flags: E - expected, S - seen-reply, A - assured, C - confirmed, D - dying,
F - fasttrack, s - srcnat, d - dstnat
0    SAC protocol=tcp src-address=192.168.10.105:51234
      dst-address=142.250.79.78:443 reply-src-address=142.250.79.78:443
      reply-dst-address=10.0.2.15:51234 tcp-state=established
      timeout=23h59m58s orig-packets=52 orig-bytes=8402
      repl-packets=48 repl-bytes=61003
```

O que cada pedaço conta:

- **src/dst-address** — a conversa como o cliente a iniciou; **reply-*** — como ela volta (denunciando NAT no caminho, flag **s**);
- **tcp-state=established** + flags **S** (viu resposta) e **A** (*assured*: tráfego nos dois sentidos consolidado) — uma conexão saudável;
- **timeout** — o prazo de vida sem tráfego novo. Cada protocolo/estado tem o seu: TCP estabelecido dura horas; UDP, segundos a minutos; TCP semiaberto (**syn-sent**), poucos segundos. Tráfego renova o prazo; silêncio deixa a entrada morrer;
- **contadores (orig/repl-packets/bytes)** — quanto já passou em cada sentido: ferramenta de diagnóstico de primeira (“a conexão existe, mas só tem tráfego de ida...”).

E os protocolos “sem conexão”? O conntrack finge que têm: um par pergunta/resposta **UDP** vira uma “conexão” com timeout curto; um **ICMP** echo request/reply idem. É o que permite ao firewall stateful tratar DNS, NTP e ping com a mesma lógica do TCP — e é por isso que o `/ping` do laboratório de NAT apareceu na tabela como **protocol=icmp**.

15.4 O custo da memória

Nada é de graça: cada entrada consome memória, e cada pacote novo consome CPU para busca/criação na tabela. Três números para operar com consciência em `/ip firewall connection tracking`:

```
/ip firewall connection tracking print
```

- **max-entries** — o teto da tabela (calculado da RAM por padrão). Tabela **cheia** = **conexões novas descartadas**: o sintoma é “internet intermitente” com CPU baixa — clássico em concentrador PPPoE subdimensionado ou sob ataque de flood de conexões;
- **timeouts ajustáveis** — `udp-timeout`, `tcp-established-timeout` etc. Encurtar timeouts derruba entradas ociosas mais cedo (alívio em tabela cheia), ao custo de matar conversas legítimas muito quietas;
- **enabled=auto** — o `conntrack` liga sozinho quando existe regra que precisa dele (NAT, filter por estado). Desligá-lo (`no`) transforma o equipamento num roteador “burro e rápido” — sem NAT, sem firewall stateful: faz sentido em roteador de **core** puro que só encaminha (Volume 8), jamais na borda. E há um meio-termo importante, o **RAW/notrack**, que isenta tráfego escolhido a dedo — assunto do

4.

! O conceito mais importante do capítulo

O firewall do RouterOS não julga pacotes — julga **conversas**. O `conntrack` dá a cada pacote um estado (`new/established/related/invalid`), e o firewall eficiente decide caro **uma vez por conexão** (no `new`) e barato em todos os demais pacotes (`established,related` aceitos cedo, `invalid` no lixo). Quem entende a tabela de conexões lê o firewall inteiro; quem não entende, coleciona regras por superstição.

15.5 Laboratório

15.5.1 Ambiente

O roteador de borda pronto do Capítulo 14: `lab-r1` (`cli-paoquente-r1`, WAN NAT + LAN + DHCP + DNS + NAT) e `lab-r2` (`pc-padaria`, cliente DHCP). Se não guardou, remonte pelas etapas 0–6 de lá (10 minutos — o gabarito `.rsc` ajuda). Vamos **observar** o `conntrack`; o filtro começa no próximo capítulo.

15.5.2 Comandos passo a passo

Passo 1 — a tabela em repouso. Em `lab-r1`:

```
/ip firewall connection print
```

Poucas linhas (talvez o seu SSH da gerência, NTP). Anote quantas.

Passo 2 — **nascimento de uma conexão**. Em `lab-r2`, dispare um ping contínuo:

```
/ping 8.8.8.8
```

Em **lab-r1**, encontre-a e examine:

```
/ip firewall connection print detail where protocol=icmp
```

Observe: `src-address=192.168.10.x`, `reply-dst-address` com o IP da WAN (o NAT, flag `s`), timeout de ~10 s **renovando-se** a cada pacote (rode o print duas vezes e compare).

Passo 3 — morte por timeout. Pare o ping no r2 (Ctrl+C) e, no r1, repita o print a cada poucos segundos:

```
/ip firewall connection print where protocol=icmp
```

O `timeout` decresce até a entrada sumir — silêncio mata a conexão, e você assistiu.

Passo 4 — TCP conta mais história. Em **lab-r2**, abra uma conexão TCP de verdade (o fetch baixa uma página):

```
/tool fetch url=https://mikrotik.com keep-result=no
```

Em **lab-r1**, rápido (a conexão fecha ao terminar):

```
/ip firewall connection print detail where protocol=tcp and dst-port=443
```

Compare com o ICMP: `tcp-state=established`, timeout de **horas**, flags `SAC`. Depois que o fetch termina, o RouterOS de lá fecha a conexão educadamente — e a entrada some rápido (estados de fechamento têm timeout curto).

Passo 5 — os limites da casa. Ainda em **lab-r1**:

```
/ip firewall connection tracking print
```

Localize `max-entries` e `total-entries`. Com 256 MB de RAM, o CHR sustenta dezenas de milhares de entradas — guarde a razão `total/max-entries` como um dos sinais vitais de um concentrador.

15.5.3 Verificação

1. Você viu a mesma conexão ICMP com o timeout **renovando** (tráfego) e depois **decrecendo até sumir** (silêncio);
2. A entrada TCP mostrou `tcp-state=established`, flags `SAC` e timeout longo — e você sabe dizer por que difere do ICMP;
3. As flags `s` (`srcnat`) apareceram nas conexões da LAN — a herança do Capítulo 13 explicada;
4. Você sabe onde olhar `max-entries/total-entries` e qual o sintoma de tabela cheia.

15.5.4 Desafios

1. Um pacote TCP ACK chega da internet e a tabela não tem nada sobre aquela conversa. Que estado ele recebe? E um TCP SYN nas mesmas condições? Por que a diferença importa para o firewall?
2. Em `lab-r2`, rode `/resolve mikrotik.com` e ache a conexão correspondente na tabela do `r1`. Protocolo? Timeout? Por que o `conntrack` trata assim um protocolo que “não tem conexão”?
3. Concentrador com 4 GB de RAM, 8 mil assinantes, média de 40 conexões por assinante: estime as entradas em uso. Num flood que abre 100 conexões UDP **por segundo por assinante** infectado (200 infectados), quanto tempo até encher uma tabela de 1M de entradas (UDP timeout 30 s — considere o regime estacionário)? O que o operador vê?
4. `connection tracking enabled=no` deixaria o `lab-r1` mais rápido. O que quebraria **imediatamente** na nossa topologia — e em que tipo de equipamento de provedor essa troca vale a pena?

Solução dos desafios

1. ACK órfão → **invalid** (não está na tabela e não é um começo válido de conversa). SYN → **new** (é exatamente como conversas TCP começam). A diferença é o firewall inteiro: **new** merece julgamento (talvez seja um cliente legítimo abrindo conexão); **invalid** nunca é legítimo em operação normal — descarta-se sem dó. Tratar os dois igual — para qualquer um dos lados — cria ou um buraco ou um firewall que quebra conexões válidas.
2. `protocol=udp dst-port=53` (a consulta ao upstream aparece também, vinda do próprio `r1`). Timeout curto (~10 s pós-resposta). O `conntrack` inventa a “conexão” UDP pareando pergunta e resposta por endereços/portas — é o que permite à resposta do DNS voltar através do NAT e do firewall stateful sem regra especial de entrada.
3. Regime normal: $8\,000 \times 40 = \mathbf{320\,mil}$ entradas — folga numa tabela de 1M. Flood: $200 \times 100 = 20\,mil$ conexões novas/s; com timeout de 30 s, o regime estacionário adiciona $20\,000 \times 30 = \mathbf{600\,mil}$ entradas — somadas às 320 mil, a tabela satura (920 mil — 1M) em ~30 s de ataque. O operador vê `total-entries` cravado no teto, clientes reclamando de conexões novas falhando (páginas que não abrem) com CPU e banda aparentemente normais. Remédios: RAW/notrack e rate-limit de **new** — 4 e Capítulo 21.
4. Quebra na hora: o **NAT** (masquerade depende da tabela) — a LAN inteira para de navegar; e qualquer filtro por estado que existisse. Vale a pena em roteador **interno de core/trânsito**: só encaminha pacotes entre redes públicas, sem NAT e sem stateful (a proteção mora nas bordas) — aí o `conntrack` é custo puro, e desligá-lo libera CPU. Nunca na borda, nunca no concentrador.

15.6 Resumo

- O **connection tracking** é a memória do RouterOS: toda conversa roteada vira uma entrada com estado, timeout e contadores — NAT e firewall stateful são consumidores dela.
- Estados (Tabela 15.1): **new** (julgado), **established/related** (aceitar cedo — é o grosso do tráfego), **invalid** (descartar sempre; e **invalid new**).
- UDP e ICMP ganham “conexões” por pareamento pergunta/resposta com timeouts curtos; helpers marcam canais secundários como **related**.
- A tabela conta histórias: **reply-*** denuncia NAT, **tcp-state** e flags dizem a saúde, contadores dizem o volume, timeout diz o futuro.
- Custo real: memória por entrada e CPU por conexão nova — monitore **total/max-entries**; tabela cheia = internet intermitente misteriosa.

No próximo capítulo, os trilhos por onde as regras correm: as **chains** `input`, `forward` e `output`, e a lógica de avaliação que decide o destino de cada pacote.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall connection` e `connection tracking` (estados, timeouts, helpers em `service-port`). O modelo stateful e o conntrack derivam do Netfilter do Linux, cuja documentação conceitual complementa; Tanenbaum; Feamster; Wetherall (2021) cobre TCP/UDP/ICMP por baixo de tudo isso. As referências completas, em ABNT, estão no apêndice [Referências](#).

16 Chains e a lógica do firewall

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Dizer, para qualquer pacote, em qual **chain** ele será julgado — **input**, **forward** ou **output** — usando uma única pergunta;
- Explicar a avaliação **de cima para baixo, primeira regra que casa decide** — e por que a ordem das regras é metade do firewall;
- Usar as **actions** fundamentais (**accept**, **drop**, **reject**, **log**, **jump**) e escolher entre **drop** e **reject** com critério;
- Montar o **esqueleto stateful** de uma chain (**established/related** → **invalid** → julgamento do **new**) e explicar cada linha;
- Usar contadores e a action **log** para ver regras trabalhando — inclusive no modo “ensaio”, antes de qualquer bloqueio real.

16.1 Três tribunais, uma pergunta

O firewall de filtro vive em `/ip firewall filter`, e cada regra pertence a uma **chain** (corrente/fila de julgamento). As três nativas separam os pacotes por uma pergunta só: **qual a relação do pacote com o roteador?**

- **input** — o pacote é **para** o roteador (destino: um IP dele). O SSH da gerência, o DNS que a LAN consulta, o ping no gateway — tudo **input**;
- **forward** — o pacote **atravessa** o roteador (origem e destino são de terceiros). A navegação dos clientes é 99% disso;
- **output** — o pacote **nasce** no roteador (ele mesmo consultando o upstream de DNS, o NTP, o ping que você dispara do terminal).

Quem decide a chain não é você — é o **destino do pacote**, avaliado logo após a decisão de roteamento (Figura 16.1):

A separação é a sua primeira ferramenta de projeto: **proteger o roteador** é escrever regras no **input** (próximo capítulo); **proteger e policiar os clientes** é escrever no **forward** (Capítulo 18). Misturar os dois é o erro de iniciante mais comum — regras de cliente no **input** não fazem nada, e regras de roteador no **forward** protegem a caixa errada.

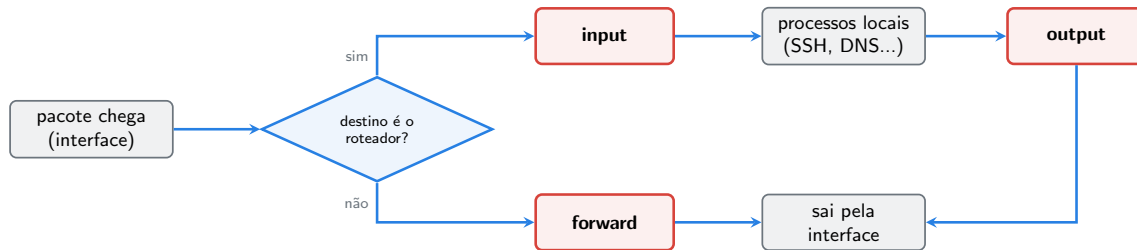


Figura 16.1: O caminho do pacote: a decisão de roteamento distribui cada um à sua chain — para o roteador (**input**), através dele (**forward**) ou nascido nele (**output**).

16.2 A lógica: de cima para baixo, primeira que casa decide

Dentro de cada chain, as regras formam uma **lista ordenada**. Cada pacote da chain percorre a lista do topo para baixo; na primeira regra cujas condições (*matchers*) ele satisfaz, a **action** é executada — e, se ela for terminal (**accept**, **drop**, **reject**), o julgamento **acaba ali**. As regras seguintes nem são olhadas.

Duas consequências que valem um firewall inteiro:

1. **Ordem é semântica.** **drop** geral antes de **accept** específico = o **accept** nunca executa. As regras baratas e frequentes vão no topo (é o esqueleto stateful do Capítulo 15); as exceções vêm **antes** das regras gerais que elas excetua;
2. **O fim da lista é uma decisão.** Se nenhuma regra casar, o RouterOS **aceita** o pacote — a política padrão é permissiva. Um firewall de verdade termina, portanto, com uma regra explícita de **drop** do resto (“**drop all**”) — e tudo que deve viver, vive **acima** dela.

💡 Analogia para guardar

Uma chain é uma **fila de carimbos**: o pacote passa de mesa em mesa, e o primeiro fiscal cuja regra o descreve dá o carimbo final — entra ou lixo. Se chegar ao fim da fila sem carimbo, a porta está aberta (política padrão: aceitar). Por isso o último fiscal de um firewall sério carimba “lixo” em tudo que sobrou.

16.2.1 As actions

Tabela 16.1: Actions essenciais do filter

Action	Efeito	Quando usar
accept	deixa passar, fim do julgamento	tráfego aprovado
drop	descarta em silêncio	o padrão para lixo externo

Action	Efeito	Quando usar
reject	descarta e avisa a origem (ICMP/TCP reset)	redes internas, onde a resposta rápida ajuda
log	registra no log e continua para a próxima regra	diagnóstico e ensaio
jump	desvia para uma chain sua (jump-target)	organizar firewalls grandes
return	volta da chain desviada	fecha o ciclo do jump

Dois pares merecem nota:

- **drop × reject**: o **drop** silencia — quem sonda a sua WAN não descobre nem que existe um firewall (e cada pacote de resposta economizado importa sob ataque). O **reject** responde “recusado” — em rede **interna**, poupa usuários e aplicações de esperar timeouts. Regra de bolso: **fora, drop; dentro, reject quando a experiência importar**;
- **log não é terminal**: o pacote é registrado e **segue** para as regras seguintes. Isso permite o padrão mais valioso deste capítulo — o **ensaio**: coloque um **log** onde o futuro **drop** ficará, observe alguns dias o que ele pegaria e só então troque por **drop**. Firewall de produção se estreia sem surpresas. (A action **log** aceita **log-prefix="texto"** — use sempre: log sem prefixo é charada.)

16.2.2 Chains próprias: organização por jump

Além das três nativas, você cria chains suas e desvia tráfego para elas:

```
/ip firewall filter add chain=forward src-address=192.168.10.0/24 action=jump jump-target=
/ip firewall filter add chain=clientes protocol=tcp dst-port=25 action=drop comment="bloqu
/ip firewall filter add chain=clientes action=return
```

O pacote que casa com o jump é julgado pelas regras da chain **clientes**; um **return** (ou o fim da chain) devolve-o para onde parou na **forward**. Em firewalls de provedor com centenas de regras, chains temáticas (**da-wan**, **clientes**, **gerencia**) mantêm a lista legível — e mais rápida, pois grupos inteiros de regras só são percorridos por quem casa com o jump.

16.3 O esqueleto que você vai repetir para sempre

Juntando conntrack (Capítulo 15) + chains + ordem, nasce o esqueleto de **toda** chain de filtro bem construída:


```

/ip firewall filter
add chain=input action=accept connection-state=established,related comment="aceita conversas"
add chain=input action=drop connection-state=invalid comment="lixo do conntrack"
# ... daqui para baixo, só pacotes NEW chegam:
# regras que julgam QUEM PODE ABRIR conversa nova
# ...
add chain=input action=drop comment="drop all: o que nao foi aceito acima, morre"

```

Leia-o como narrativa: *conversas aprovadas passam barato (linha 1); lixo morre cedo (linha 2); pedidos novos são julgados um a um (miolo); e tudo que sobrar, morre (última linha)*. O miolo muda conforme a chain e o papel do equipamento — os dois próximos capítulos são exatamente sobre esses miolos.

! O conceito mais importante do capítulo

Firewall no RouterOS é responder três perguntas, nesta ordem: **em qual chain?** (qual a relação do pacote com o roteador), **em que posição?** (primeira que casa decide; baratas no topo, drop all no fim) e **com qual action?** (fora drop, dentro reject, ensaio com log). Quem domina as três perguntas escreve qualquer regra; quem não domina, copia listas da internet sem saber o que protegem.

16.4 Laboratório

16.4.1 Ambiente

O de sempre neste volume: **lab-r1** (borda completa do Capítulo 14) e **lab-r2** (cliente na LAN). Vamos montar o esqueleto do **input em modo ensaio** — nenhum bloqueio real ainda; o aperto de verdade é no próximo capítulo, com rede de segurança.

16.4.2 Comandos passo a passo

Passo 1 — o esqueleto, com ensaio no lugar do drop. Em **lab-r1**:

```

/ip firewall filter add chain=input action=accept connection-state=established,related comment="aceita conversas"
/ip firewall filter add chain=input action=drop connection-state=invalid comment="lixo do conntrack"
/ip firewall filter add chain=input action=log log-prefix="INPUT-NEW: " comment="ENSAIO: o que vem de fora"

```

Repare: onde um dia haverá julgamento + drop all, por ora há um **log** — o firewall observa e **não bloqueia nada** (política padrão aceita o que o log deixa passar).

Passo 2 — gerar tráfego e ler contadores. Em **lab-r2**, movimente a rede: `/ping 192.168.10.1 count=5` (input do r1!), `/ping google.com count=5` (forward + o DNS é input!). Em **lab-r1**:

```
/ip firewall filter print stats
```

```
[vitor@lab-r1] > /ip firewall filter print stats
Flags: X - disabled, I - invalid; D - dynamic
Columns: CHAIN, ACTION, BYTES, PACKETS
#  CHAIN  ACTION  BYTES  PACKETS
0  input  accept  12 480    156
1  input  drop      0      0
2  input  log      1 344    18
```

Os contadores são o eletrocardiograma do firewall: a regra 0 engorda a cada pacote de conversas vivas; a 2 conta só as **aberturas** (new). Zere-os e observe do zero quando quiser: `/ip firewall filter reset-counters-all`.

Passo 3 — ler o ensaio. Veja quem o log pegou:

```
/log print where message~"INPUT-NEW"
```

```
15:41:02 firewall,info INPUT-NEW: input: in:ether2 out:(unknown 0),
proto UDP, 192.168.10.105:44231->192.168.10.1:53, len 62
15:41:07 firewall,info INPUT-NEW: input: in:ether2 out:(unknown 0),
proto ICMP, 192.168.10.105->192.168.10.1, len 84
```

Cada linha é um pedido de conversa nova com o roteador: o DNS da LAN (porta 53), o ping no gateway, o seu SSH da gerência (`in:ether1`, porta 22)... Este é o **inventário real** do que o próximo capítulo precisa permitir antes do drop all.

Passo 4 — sentir a ordem. Prove que ordem é semântica: suba uma regra de log específica **abaixo** da genérica e veja que nunca casa:

```
/ip firewall filter add chain=input protocol=icmp action=log log-prefix="PING: " comment="
```

Pingue o r1 de novo (do r2) e confira: o contador da regra nova avança? Sim — porque **log não é terminal**: o pacote passou pela regra 2 (log genérico) e **continuou** até a 3. Agora troque o log genérico por um **accept** temporário e repita:

```
/ip firewall filter set 2 action=accept
/ip firewall filter reset-counters-all
```

Ping de novo: agora a regra **PING:** fica **zerada** — o **accept** da regra 2 é terminal e ninguém abaixo dela vê pacote algum. Ordem + terminalidade, demonstradas. Desfaça o experimento:

```
/ip firewall filter set 2 action=log
/ip firewall filter remove [find comment="teste de ordem"]
```

16.4.3 Verificação

1. `/ip firewall filter print` mostra o esqueleto: `accept established/related → drop invalid → log` ensaio;
2. Os contadores contam a história certa: regra 0 com milhares de pacotes, regra 1 quase zerada, log só com aberturas;
3. O `/log print` lista as conversas novas reais da sua rede (DNS, ping, SSH) — o inventário do próximo capítulo;
4. Você demonstrou que `log` deixa passar adiante e `accept` termina o julgamento (Passo 4).

16.4.4 Desafios

1. Classifique na chain certa: (a) cliente da LAN navegando; (b) você pingando 8.8.8.8 **do terminal do r1**; (c) um atacante tentando WinBox no IP público do r1; (d) o r2 consultando o DNS do r1; (e) o r1 consultando o upstream 1.1.1.1.
2. No esqueleto, por que o `drop invalid` vem **depois** do `accept established/related`, e não antes? Há diferença prática?
3. Escreva (sem aplicar) a regra que registraria no log, com prefixo `WAN-NEW:`, apenas as tentativas de conexão **novas** chegando pela `ether3` — e diga em que posição da chain ela deveria entrar.
4. Um colega propõe terminar o input com `action=reject` “para o atacante desistir mais rápido”. Argumente contra, com dois motivos.

Solução dos desafios

1. (a) `forward`; (b) `output` (nasce no roteador); (c) `input` (destino é o roteador); (d) `input` (o serviço DNS é do roteador); (e) `output`. O par (d)/(e) é o mesmo “pedido de DNS” visto de dois lados — ótimo teste de compreensão.

2. Funcionalmente, quase nada muda (um pacote não é os dois estados ao mesmo tempo); a razão é **desempenho**: `established/related` é a esmagadora maioria dos pacotes — casando primeiro, eles saem da chain na regra 1. O `invalid` é raro; colocá-lo antes faria todo pacote bom pagar uma verificação inútil. Regra frequente em cima: o princípio de ordenação em uma linha.

3.

```
/ip firewall filter add chain=input in-interface=ether3 connection-state=new action=log
```

Posição: **depois** do `accept established/related` e do `drop invalid` (para só ver aberturas reais), **antes** de qualquer `accept` específico — no nosso esqueleto atual, junto do log de ensaio. É o ensaio focado na fronteira.

4. (a) **reject responde** — cada pacote de sonda gera um pacote seu de volta: sob flood, você paga banda e CPU para ajudar o atacante (e pode virar refletor contra origens forçadas); (b) a resposta **confirma que há um equipamento vivo** e filtrado ali, convidando a sondagem mais cuidadosa. O silêncio do **drop** não custa nada e não informa nada — na WAN, é a escolha certa. (O reject tem lugar: dentro de casa, poupando timeouts.)

16.5 Resumo

- Três chains, uma pergunta: **para** o roteador (**input**), **através** (**forward**), **dele** (**output**) — quem decide é o destino do pacote (Figura 16.1).
- Avaliação **top-down**, **primeira que casa decide**; actions terminais encerram; sem casar nada, a política padrão **aceita** — por isso o firewall sério termina em drop all explícito.
- Actions (Tabela 16.1): fora **drop** (silêncio), dentro **reject** (experiência), **log** para diagnóstico e **ensaio** (não é terminal!), **jump/return** para organizar em chains temáticas.
- O esqueleto universal: accept **established,related** → drop **invalid** → julgamento dos **new** → drop all.
- Contadores (**print stats**) e logs com prefixo são os olhos do firewall — e o modo ensaio (log no lugar do futuro drop) é como se estreia firewall sem derrubar produção.

No próximo capítulo, o esqueleto ganha músculos: fechamos o **input** de verdade — serviços, gerência, ICMP e o drop all — sem perder o próprio acesso no processo.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) descreve o packet flow completo do RouterOS (diagramas de fluxo com todas as chains e etapas) e a referência de **/ip firewall filter** (matchers e actions). Discher (2016) dedica capítulos didáticos à construção de chains. As referências completas, em ABNT, estão no apêndice [Referências](#).

17 Protegendo o roteador

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Aplicar a filosofia **default deny** à **chain input**: tudo fechado, e cada abertura é uma decisão documentada;
- Organizar interfaces em **interface lists** (WAN, LAN, **gerencia**) e escrever regras que sobrevivem a mudanças de topologia;
- Inventariar os serviços que o roteador **precisa** servir (gerência, DNS da LAN, DHCP, ICMP) e liberá-los com o mínimo de exposição;
- Definir uma política de **ICMP** sensata — nem “bloqueia tudo”, nem porta aberta a flood;
- Fechar o **input** com **drop all** sem se trancar para fora — usando Safe Mode, ensaio com log e teste de cada acesso antes do aperto final.

17.1 Default deny: a inversão que muda tudo

Até aqui, o **input** do nosso roteador aceita tudo que as regras não descartam — a política permissiva de fábrica. Este capítulo faz a inversão profissional: **negar por padrão** e abrir apenas o necessário. A diferença filosófica é enorme:

- No modelo permissivo, cada serviço esquecido aberto é uma vulnerabilidade — e você descobre pelo pior caminho (o DNS aberto do Capítulo 12 virando refletor de ataque, o WinBox exposto sendo forçado);
- No modelo default deny, cada porta aberta é uma **decisão consciente**, com dono e comentário. O que você esqueceu fica **fechado** — esquecer vira seguro, não risco.

O preço é o cuidado: fechar o **input** de um roteador remoto é serrar o galho em que se está sentado. Por isso este capítulo é tanto sobre **método** (ensaio → liberações → teste → aperto) quanto sobre regras.

17.2 Interface lists: regras que sobrevivem à topologia

Escrever **in-interface=ether3** funciona — até o dia em que a WAN vira um PPPoE, ganha uma segunda operadora ou muda de porta. A ferramenta profissional são as **interface lists**

(que você conheceu no 5): grupos nomeados de interfaces, usados como matcher:

```
/interface list add name=WAN comment="interfaces de fronteira"  
/interface list add name=LAN comment="interfaces de clientes"  
/interface list member add list=WAN interface=ether3  
/interface list member add list=LAN interface=ether2
```

Daqui em diante, as regras dizem **in-interface-list=WAN** — e quando a topologia mudar, você atualiza a **lista**, não o firewall. Num parque de centenas de equipamentos com o mesmo firewall-base, essa indireção é o que torna o padrão aplicável a todos (o hardening do Capítulo 21 depende disso).

A lista **gerencia** do 5 completa o trio. Nossa convenção de projeto, deste capítulo até o fim do livro:

Tabela 17.1: As três interface lists do projeto

Lista	Contém	Papel no firewall
WAN	interfaces de fronteira (operadora, internet)	de onde nada abre conversa com o roteador
LAN	interfaces de clientes/serviços	abre só os serviços servidos (DNS, DHCP)
gerencia	interfaces (e redes) da equipe	única origem de SSH/WinBox

17.3 O inventário: o que o input precisa aceitar

O log de ensaio do Capítulo 16 respondeu empiricamente; agora o inventário raciocinado do nosso roteador de borda:

1. **Gerência** — SSH e WinBox, **apenas** da lista **gerencia** (no laboratório, a **ether1**; na vida real, a rede de gerência do provedor). Note a defesa em camadas: o **allowed-address** do **/ip service**
(5) continua lá — o firewall é a segunda tranca;
2. **DNS para a LAN** — o roteador é o resolvidor dos clientes (Capítulo 12): UDP e TCP 53, **da LAN apenas**. É exatamente a regra que fecha o buraco do open resolver: da WAN, ninguém alcança a porta 53;
3. **DHCP na LAN** — detalhe que derruba iniciantes: o pedido DHCP (Discover) chega por broadcast **antes de o cliente ter IP**, e é julgado no **input**. Sem liberar UDP 67–68 da LAN, o DHCP server para de servir — e o defeito (“ninguém pega IP depois do firewall!”) é dos que fazem técnico duvidar da própria sanidade;
4. **ICMP** — a seguir, com política própria;

5. **Nada mais.** NTP, upstream de DNS, atualizações — tudo isso o roteador **inicia** (output → volta como established). Não se abre porta para resposta; o conntrack cuida (Capítulo 15).

17.3.1 A política de ICMP

Há duas superstições opostas sobre ping: “bloqueie tudo” (esconde nada e quebra diagnóstico — e pior, quebra *path MTU discovery*, causando defeitos bizarros de páginas que não carregam) e “libere tudo” (porta aberta a flood). O equilíbrio de provedor:

```
add chain=input protocol=icmp limit=20,10:packet action=accept comment="ICMP com teto"
```

Aceita ICMP — ping, os erros de que o TCP depende — mas com **limite de taxa**: 20 pacotes/s com rajada de 10. Diagnóstico funciona; flood de ICMP encontra teto. (Quem quiser esconder o roteador de ping da WAN pode restringir por lista — mas mantenha o ICMP de **erro** vivo, ou pagará em chamados estranhos.)

17.4 A sequência completa

O input final do nosso roteador de borda — o esqueleto do Capítulo 16 com o miolo preenchido:

```
/ip firewall filter
add chain=input action=accept connection-state=established,related comment="conversas em a
add chain=input action=drop connection-state=invalid comment="lixo do conntrack"
add chain=input action=accept in-interface-list=gerencia comment="gerencia: SSH/WinBox/tud
add chain=input action=accept protocol=icmp limit=20,10:packet comment="ICMP com teto"
add chain=input action=accept in-interface-list=LAN protocol=udp dst-port=53 comment="DNS
add chain=input action=accept in-interface-list=LAN protocol=tcp dst-port=53 comment="DNS
add chain=input action=accept in-interface-list=LAN protocol=udp dst-port=67-68 comment="D
add chain=input action=drop comment="drop all"
```

Oito regras. Leia de novo, linha a linha, e note as escolhas: a gerência aceita **tudo** (é rede de confiança — e se ela for comprometida, o firewall é o menor dos problemas); a LAN só alcança os **serviços que o roteador serve a ela**; a WAN não aparece em accept nenhum — para ela, só resta o drop all. O silêncio é a resposta.

! O conceito mais importante do capítulo

No input, a pergunta de projeto é: “**quem precisa abrir conversa com o roteador, e para quê?**” — e a resposta vira uma lista curta de accepts (gerência; serviços servidos à LAN; ICMP com teto) sobre o esqueleto stateful, selada por um drop all. Da WAN, nenhum accept: roteador de borda não conversa com estranhos — ele apenas

encaminha.

17.5 Laboratório

17.5.1 Ambiente

lab-r1 (borda completa) e **lab-r2** (cliente na LAN), continuando do capítulo anterior (o esqueleto com log de ensaio já aplicado). Trabalhe por SSH **da gerência** — e o laboratório inteiro dentro do **Safe Mode** (**Ctrl+X**): é o exercício definitivo do cinto de segurança.

17.5.2 Comandos passo a passo

Passo 0 — Safe Mode. Na sessão SSH do r1, **Ctrl+X**. Confirme o **<SAFE>** no prompt. Se qualquer passo der errado a ponto de derrubar seu acesso, a reversão automática desfaz tudo.

Passo 1 — as listas. Formalize a topologia:

```
/interface list add name=WAN comment="fronteira"  
/interface list add name=LAN comment="clientes"  
/interface list member add list=WAN interface=ether3  
/interface list member add list=LAN interface=ether2
```

(A lista **gerencia** já existe desde o 5, com a **ether1**.)

Passo 2 — o miolo, acima do log. As liberações entram **entre** o **drop invalid** e o log de ensaio (use **place-before** apontando para o número da regra de log — confira com **print**):

```
/ip firewall filter print  
/ip firewall filter add chain=input action=accept in-interface-list=gerencia comment="gere  
/ip firewall filter add chain=input action=accept protocol=icmp limit=20,10:packet comment  
/ip firewall filter add chain=input action=accept in-interface-list=LAN protocol=udp dst-p  
/ip firewall filter add chain=input action=accept in-interface-list=LAN protocol=tcp dst-p  
/ip firewall filter add chain=input action=accept in-interface-list=LAN protocol=udp dst-p
```

Passo 3 — auditar o ensaio. Com as liberações no lugar, o log de ensaio (última regra) só deve pegar o que **morreria** no futuro **drop all**. Zere contadores, gere tráfego normal e leia:

```
/ip firewall filter reset-counters-all
```


Em lab-r2: `/ping 192.168.10.1 count=3`, `/ping google.com count=3`, renove o DHCP (`/ip dhcp-client release numbers=0`). Em lab-r1:

```
/ip firewall filter print stats
/log print where message~"INPUT-NEW"
```

Os accepts engordaram; o log deve estar **silencioso** (ou mostrando só ruído da WAN — que é exatamente o que queremos matar). Se algo legítimo aparecer no log, é serviço que faltou liberar: **investigue antes de apertar**.

Passo 4 — o aperto. Troque o log de ensaio pelo drop all:

```
/ip firewall filter set [find comment~"ENSAIO"] action=drop log=no comment="drop all"
```

Passo 5 — testar tudo, antes de confirmar. A bateria de testes, **ainda dentro do Safe Mode**:

- Sua sessão SSH continua viva (gerência OK — você está nela);
- Em lab-r2: `/ping 192.168.10.1` (ICMP OK), `/ping google.com` (DNS + forward OK), renovar DHCP (DHCP OK);
- Em lab-r2, o teste de **bloqueio**: `ssh vitor@192.168.10.1` deve **falhar** — a LAN não é gerência; o drop all a barrou. (Reflita: antes deste capítulo, esse acesso funcionava — só o `allowed-address` do serviço segurava.)

Tudo verde? `Ctrl+X` para **sair do Safe Mode confirmando**. O firewall está valendo.

Passo 6 — documentar. O ritual do 3:

```
/ip firewall filter export file=input-borda-v1
/system backup save name=pos-input
```

17.5.3 Verificação

1. `/ip firewall filter print` mostra as 8 regras na ordem do capítulo, todas comentadas;
2. A bateria do Passo 5 passou: gerência viva, LAN com DNS/DHCP/ping, e SSH **da LAN** morto no drop all;
3. `print stats` conta a história certa (established/related engordando, drop all pegando só ruído);
4. O export do filter está fora do equipamento.

17.5.4 Desafios

1. Por que a regra da gerência (`accept in-interface-list=gerencia`) está **acima** do ICMP e dos serviços da LAN? O que mudaria (funcional e em desempenho) se ela fosse a última antes do drop?
2. Um técnico esqueceu o notebook em campo e precisa acessar o r1 **da LAN do cliente**, excepcionalmente, por 1 hora. Escreva a liberação mais **estrita** possível (origem única, serviço único) e o plano de remoção — sem editar as regras permanentes.
3. Sua WAN ganhou IPv6? Não — mas o roteador tem um segundo IP público na mesma ether3 (um /29 da operadora). O firewall deste capítulo o protege? Por quê — o que os matchers usam, endereço ou interface?
4. Reproduza o desastre com rede de segurança: dentro do Safe Mode, **desabilite** a regra da gerência e confirme que sua sessão SSH morre em seguida (tente digitar). O que acontece ~15 s depois? Explique a mágica revendo Capítulo 4.

Solução dos desafios

1. Funcionalmente quase nada mudaria (não há drop entre elas que pegasse gerência), mas: (a) **desempenho** — todo pacote da gerência percorreria as regras de ICMP/DNS/DHCP antes de ser aceito; (b) **robustez** — a posição no topo garante que nenhuma regra futura inserida no miolo possa, por engano, descartar gerência antes do `accept`. Regra de sobrevivência primeiro; o resto depois.

2.

```
/ip firewall filter add chain=input src-address=192.168.10.105 protocol=tcp dst-port=22
```

(Como o serviço SSH tem `allowed-address` da gerência, seria preciso também um ajuste temporário lá — as duas trancas existem.) Plano de remoção: comentário com prazo + remover com `remove [find comment~"TEMP"]` — e, elegante, um agendamento no `/system scheduler` que remove sozinho (Volume 12).

3. Protege. Os `accepts` usam **interface lists** e a WAN não tem `accept` algum — o drop all barra conversas novas para *qualquer* endereço do roteador chegando por *qualquer* interface. É a vantagem do default deny por interface sobre listas de IPs: endereços novos nascem protegidos por padrão.

4. A sessão congela (os pacotes do seu SSH — `established` — continuam aceitos pela regra 0... até você digitar? Não: `established` continua passando! O que morre é **conversa nova**). Surpreso? Teste de verdade: com a regra da gerência desabilitada, a sessão atual sobrevive (`established/related` está acima), mas uma **nova** conexão SSH falha. Para derrubar tudo, seria preciso desabilitar também a regra 0 — e aí sim, ao travar a sessão, o Safe Mode detecta a queda e **reverte tudo** em segundos, devolvendo as duas regras. A mágica: o Safe Mode (Capítulo 4) desfaz mudanças não confirmadas quando a sessão que as fez morre. (E a lição extra: o esqueleto stateful protege até você dos seus erros — por camadas.)

17.6 Resumo

- **Default deny** no input: cada abertura é decisão documentada; o esquecido nasce fechado.
- **Interface lists** (WAN/LAN/*gerencia*, Tabela 17.1) desacoplam regras de topologia — o firewall-base do parque agradece.
- O miolo do input de borda: *gerência* (tudo), ICMP **com teto de taxa** (nunca bloquear ICMP inteiro — path MTU!), DNS e DHCP **da LAN** (DHCP = UDP 67–68 no input — a pegadinha clássica), e **nenhum accept da WAN**.
- Método de aplicação: Safe Mode + ensaio com log + auditoria dos logs + bateria de testes + aperto + export. Nessa ordem, sempre.
- As duas trancas convivem: **allowed-address** nos serviços e firewall — defesa em camadas.

No próximo capítulo, o tribunal dos clientes: a chain **forward** — o que pode atravessar o roteador — e as **address lists**, a ferramenta que transforma listas de IPs em política.

Bibliografia do capítulo

A página *Securing your router* e a seção *Building Advanced Firewall* da documentação oficial (MikroTik, 2026a) trazem as recomendações que este capítulo adapta ao roteador de borda de provedor. Discher (2016) cobre proteção de input com exemplos comentados. As referências completas, em ABNT, estão no apêndice [Referências](#).

18 Forward e address-lists

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Projetar a chain **forward** de um provedor — que é **diferente** da de uma empresa: o cliente paga por internet aberta, não por censura;
- Aplicar os filtros de forward que todo provedor sério tem: esqueleto stateful, antispoofing e o bloqueio clássico da porta 25;
- Criar e usar **address lists** estáticas — a ferramenta que separa *política* (regras) de *dados* (quem);
- Criar address lists **dinâmicas** com `add-src-to-address-list + timeout` — listas que se alimentam sozinhas;
- Reconhecer os usos de operação: corte de inadimplentes, armadilhas para scanners e listas de proteção da infraestrutura.

18.1 O forward de um provedor não é o de uma empresa

A chain **forward** julga tudo que **atravessa** o roteador (Capítulo 16) — a navegação dos clientes, portanto. E aqui o projeto exige uma reflexão que firewall de empresa não tem: numa empresa, o forward implementa a política do dono (“RH não acessa rede social”); num **provedor**, o cliente paga por **acesso à internet** — filtrar o tráfego dele é, além de má prática, problema jurídico (a neutralidade de rede do Marco Civil brasileiro, que reencontraremos no Volume 9).

O forward do provedor filtra pouco, e o pouco certo:

1. **O esqueleto stateful** de sempre — eficiência e higiene (Capítulo 15);
2. **Antispoofing** — pacotes *saindo* da sua rede devem ter origem *da sua rede*. Um cliente infectado emitindo pacotes com origem forjada participa de ataques DDoS refletidos mundo afora com o **seu** nome (seu bloco IP, sua reputação). A regra: o que entra pela LAN com origem que não é da LAN, morre;
3. **Porta 25 (SMTP) saindo** — o bloqueio mais consagrado: máquinas infectadas despejando spam pela porta 25 direta são a via expressa para seu bloco IP entrar em listas negras de e-mail (e todos os seus clientes junto). Bloqueia-se 25; e-mail legítimo usa 587 (submissão autenticada) há vinte anos. É a exceção de neutralidade aceita universalmente — o CERT.br a recomenda formalmente;

4. **Proteção da infraestrutura** — clientes não têm por que alcançar a rede de **gerência** do provedor. Uma regra fecha o assunto.

E só. Nada de bloquear “sites”, jogos, VPNs — provedor é estrada, não alfândega.

18.2 Address lists: separar política de dados

Repare no antispoofing: “origem que não é da LAN”. E o corte de inadimplente: “estes 37 IPs não navegam até pagar”. E a gerência: “estas redes são intocáveis”. Todos são a mesma pergunta — **o endereço está neste conjunto?** — e a resposta errada é criar 37 regras.

A ferramenta certa é a **address list** (`/ip firewall address-list`): um conjunto nomeado de endereços/redes, referenciado pelas regras via `src-address-list/dst-address-list`:

```
/ip firewall address-list add list=inadimplentes address=192.168.10.105 comment="contrato
/ip firewall address-list add list=inadimplentes address=192.168.10.130 comment="contrato
/ip firewall filter add chain=forward src-address-list=inadimplentes action=reject reject-
```

Uma regra, dados à parte. Cortar/religar cliente vira operação de lista (o sistema de gestão faz via API — Volume 12), sem tocar no firewall. A regra usa **reject** e não **drop** — o cliente cortado recebe resposta imediata em vez de timeouts (lembre a regra de bolso do Capítulo 16: dentro de casa, experiência importa; e um cliente cortado continua sendo seu cliente).

Listas aceitam **redes** (10.0.0.0/8), têm **comment** por entrada e — a parte que muda o jogo — podem ser alimentadas **pelas próprias regras do firewall**.

18.3 Listas dinâmicas: o firewall que aprende

Toda **action** de filter pode, em vez de (ou além de) aceitar/descartar, **anotar o endereço numa lista**:

```
add chain=input in-interface-list=WAN protocol=tcp dst-port=23 action=add-src-to-address-l
add chain=input in-interface-list=WAN src-address-list=scanners action=drop comment="scann
```

Leia o par: ninguém legítimo tenta Telnet no seu IP público (o serviço nem existe — 5); quem tenta é robô de varredura. A primeira regra **anota** a origem na lista **scanners** com validade de 1 dia (**timeout** — a entrada nasce com flag D e expira sozinha); a segunda descarta **tudo** que venha de quem está na lista. O firewall aprendeu: sondou a porta-armadilha, perdeu acesso ao roteador por 24 h.

O mecanismo (Figura 18.1) é um ciclo:

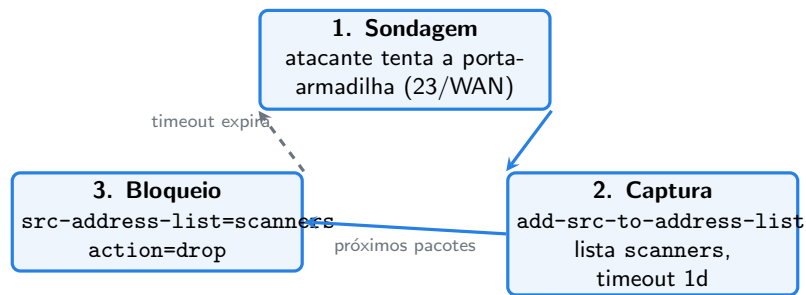


Figura 18.1: O ciclo da lista dinâmica: sondou a armadilha, entrou na lista, perdeu o acesso — até o timeout devolver o benefício da dúvida.

O padrão captura-e-bloqueia serve para muito mais que armadilhas: força bruta em SSH (capturar quem abre conexões demais), rajadas de conexões novas (o flood do desafio do Capítulo 15), lista de quem baixou de um servidor de atualização... O Capítulo 21 usa esse tijolo várias vezes.

! O conceito mais importante do capítulo

Address lists separam **política** (poucas regras, estáveis) de **dados** (listas que mudam todo dia — por operação, por API ou pelo próprio firewall via `add-src-to-address-list + timeout`). No forward do provedor, a política é curta: stateful, antispoofting, porta 25, infraestrutura protegida — e neutralidade no resto. Quem precisar de “mais firewall” que isso está no lugar errado da rede (o CPE do cliente) ou do livro (Capítulo 21).

18.4 Laboratório

18.4.1 Ambiente

lab-r1 (borda com input fechado do Capítulo 17) e lab-r2 (cliente na LAN). Safe Mode como hábito.

18.4.2 Comandos passo a passo

Passo 1 — o esqueleto do forward. Em lab-r1:

```
/ip firewall filter add chain=forward action=accept connection-state=established,related
/ip firewall filter add chain=forward action=drop connection-state=invalid comment="fwd: 1
```

Passo 2 — antispoofting e porta 25. As duas regras de cidadania:

```
/ip firewall filter add chain=forward in-interface-list=LAN src-address=!192.168.10.0/24 action=drop
/ip firewall filter add chain=forward in-interface-list=LAN protocol=tcp dst-port=25 action=drop
```

(O ! nega o matcher: “origem que **não** é a LAN”. Teste o SMTP de lab-r2: `/tool fetch url=telnet://smtp.google.com:25` falha na hora — **reject** avisando.)

Passo 3 — proteger a gerência. Clientes não alcançam a rede de gerência:

```
/ip firewall filter add chain=forward in-interface-list=LAN dst-address=192.168.56.0/24 action=drop
```

De lab-r2, prove: `/ping 192.168.56.1` — morto. (Antes funcionava — o r1 roteava para a gerência sem cerimônia.)

Passo 4 — corte por lista. Simule a inadimplência do “pc-padaria”:

```
/ip firewall address-list add list=inadimplentes address=192.168.10.105 comment="contrato rescindido"
/ip firewall filter add chain=forward src-address-list=inadimplentes connection-state=new action=drop
```

(Ajuste o .105 para o IP real do r2 — `/ip dhcp-server lease print`.) Em lab-r2: `/ping 8.8.8.8` — cortado. **Religue** o cliente removendo-o da lista (a regra fica):

```
/ip firewall address-list remove [find list=inadimplentes]
```

Ping de novo: navegando. Corte e religação sem tocar em regra — a operação do dia a dia.

Passo 5 — a armadilha dinâmica. No input (a fronteira é da WAN, mas testaremos da LAN para ver o mecanismo — a LAN também não tem por que telnetar o roteador):

```
/ip firewall filter add chain=input protocol=tcp dst-port=23 action=add-src-to-address-list address-list=scanners list-timeout=1h
/ip firewall filter add chain=input src-address-list=scanners action=drop comment="scanner bloqueado"
```

Em lab-r2, caia na armadilha e constate a sentença:

```
/system telnet 192.168.10.1
/ping 192.168.10.1 count=3
```

O telnet falha e o ping morre junto — você está na lista. Em lab-r1, veja o prisioneiro e o prazo:

```
/ip firewall address-list print
```

```
[vitor@lab-r1] > /ip firewall address-list print
Flags: X - disabled, D - dynamic
Columns: LIST, ADDRESS, TIMEOUT
# LIST ADDRESS TIMEOUT
0 D scanners 192.168.10.105 9m41s
```

Flag D, timeout decrescendo: em 10 minutos a pena prescreve (ou solte-o já: `remove numbers=0`).

18.4.3 Verificação

1. O forward tem: esqueleto, antispoofing, SMTP 25 (reject), gerência protegida e a regra de corte — todos comentados (`/ip firewall filter print where chain=forward`);
2. O corte por lista funcionou nos dois sentidos (cortou ao entrar na lista; religou ao sair) **sem editar regras**;
3. A armadilha capturou o r2 (entrada D com timeout) e o drop por lista o bloqueou por completo;
4. lab-r2 navegando normal ao final (fora das listas), e SMTP 25 continua morto.

18.4.4 Desafios

1. O antispoofing usa `src-address=!192.168.10.0/24`. Num concentrador com 30 pools de clientes, essa abordagem escala mal. Reescreva-a com address list — e diga de onde a lista seria alimentada na prática.
2. A regra de corte usa `connection-state=new`. O que muda para o cliente cortado, comparado ao corte sem esse matcher? Qual é mais “justo” na religação?
3. Monte (comandos) a defesa clássica anti força-bruta de SSH **na WAN**: quem abrir mais de 3 conexões novas TCP/22 em 1 minuto entra em `ssh-bruta` por 1 hora. (Dica: uma lista-contador com timeout de 1m como “memória curta” + a captura definitiva; matchers `connection-state=new` e `src-address-list`.)
4. Por que a armadilha do Passo 5 ficou **acima** do drop all mas o drop-por-lista ficou acima **dela**? Desenhe o caminho de um pacote do scanner na segunda visita.

Solução dos desafios

1.

```
/ip firewall address-list add list=redes-clientes address=192.168.10.0/24
/ip firewall filter set [find comment~"antispoofing"] src-address=!0.0.0.0 src-address-l
```

(Na prática: regra nova `in-interface-list=LAN src-address-list=!redes-clientes action=drop`.) A lista é alimentada pelo provisionamento: cada pool/rede de clientes nova entra na lista junto com sua criação — por script ou API (Volume 12). Política parada, dados acompanhando o crescimento.

2. Com **new**, as conexões **existentes** do cliente sobrevivem ao corte (a regra 0 do esqueleto as aceita antes) — só não abre nada novo; sem o matcher, tudo morre na hora. Mais importante é a **religação**: sem **new**, remover da lista já resolve; com qualquer resíduo, conexões travadas expiram sozinhas. O corte com **new** é mais suave (downloads em andamento terminam) e mais usado; o corte total é para abuso. Justiça é critério comercial — o firewall oferece os dois.

3.

```
/ip firewall filter add chain=input in-interface-list=WAN protocol=tcp dst-port=22 conntrack=established
/ip firewall filter add chain=input in-interface-list=WAN protocol=tcp dst-port=22 conntrack=established
/ip firewall filter add chain=input in-interface-list=WAN src-address-list=ssh-bruta action=drop
```

A ordem importa: quem já é **ssh-suspeitos** (memória de 1 m) e tenta de novo sobe para **ssh-bruta** (1 h); senão, apenas é anotado. (Refinável em mais estágios — o padrão em cascata do hardening.)

4. A armadilha precisa vir antes do drop all para sequer executar (depois dele, código morto — nada passa do drop). O drop-por-lista vem antes da armadilha para que o **reincidente** morra imediatamente, sem nem renovar a anotação... quer dizer: na segunda visita, o pacote desce: established? não → invalid? não → drop-por-lista: **casa e morre**. Se o drop viesse depois da armadilha, o scanner na lista ainda “renovaria” o próprio timeout a cada tentativa na porta 23 — detalhe fino: às vezes *queremos* isso (pena renovável!). As duas ordens são defensáveis; o que não se negocia é ambas acima do drop all.

18.5 Resumo

- Forward de **provedor**: esqueleto stateful, **antispoofing** (origem da LAN deve ser da LAN), **porta 25 fechada** (reputação do bloco), **gerência intocável** — e neutralidade no resto; filtro de conteúdo não é papel da estrada.
- **Address lists** = política separada de dados: regras estáveis (**src/dst-address-list**), listas mudando por operação/API — corte de inadimplente é **add/remove** na lista, nunca edição de regra.
- Listas **dinâmicas**: **action=add-src-to-address-list + address-list-timeout** — entradas D que expiram sozinhas; o padrão captura-e-bloqueia (Figura 18.1) constrói armadilhas, defesa de força bruta e rate-limiting por reincidência.
- **reject** para clientes (experiência), **drop** para a WAN (silêncio) — a regra de bolso continua.

No próximo capítulo, o caminho inverso: **dstnat** — publicar serviços de dentro para fora (port forwarding), com o hairpin NAT e os cuidados que todo portão aberto exige.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall address-list` e os `matchers/actions` de lista. A recomendação de gerência da porta 25/TCP para redes de provedores é do CERT.br (cert.br/docs/), alinhada ao MAAWG. Sobre antispooofing na origem, o RFC 2827/BCP 38 é o texto clássico. As referências completas, em ABNT, estão no apêndice [Referências](#).

19 dstnat e port forwarding

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o **dstnat** como o espelho do **srcnat**: troca de **destino** na entrada, para publicar serviços internos;
- Escrever a regra clássica de **port forwarding** e entender o momento em que ela age (antes do roteamento — e antes do filter);
- Fazer firewall e dstnat conversarem: por que o **forward** enxerga o pacote **já traduzido** e como aceitá-lo com precisão (**connection-nat-state**);
- Diagnosticar e resolver o **hairpin NAT** — o clássico “funciona de fora, mas não funciona de dentro”;
- Usar **redirect** (dstnat para o próprio roteador) e publicar serviços com o mínimo de exposição.

19.1 O espelho do masquerade

O Capítulo 13 encerrou com uma porta fechada: ninguém de fora inicia conexão com quem está atrás do NAT — o fim a fim está quebrado, por design. Mas a padaria quer ver as câmeras do celular; o cliente corporativo quer publicar o sistema da empresa; você quer alcançar um equipamento na LAN de um cliente. A resposta é o **dstnat**: se o **srcnat** troca o *remetente* na saída, o **dstnat** troca o **destinatário na entrada**.

A regra clássica — o **port forwarding** — diz: “o que chegar ao meu IP público na porta X, entregue ao IP interno Y, porta Z”:

```
/ip firewall nat add chain=dstnat in-interface-list=WAN protocol=tcp dst-port=8080 action=
```

Lendo por partes, como sempre: **chain=dstnat** (tradução de destino, na entrada), matchers (**in-interface-list=WAN protocol=tcp dst-port=8080** — só o que chega da fronteira àquela porta), action **dst-nat** com o destino real (**to-addresses/to-ports** — que podem diferir da porta pública, como aqui: 8080 fora, 80 dentro).

O detalhe de arquitetura que explica tudo neste capítulo: **o dstnat age antes da decisão de roteamento** (e o **srcnat**, depois). A ordem completa da travessia:

chegada → dstnat → decisão de roteamento → filter forward → srcnat → saída

Duas consequências imediatas:

1. É *por isso* que o dstnat funciona: o destino é trocado **antes** de o roteador decidir para onde encaminhar — a decisão já vê o endereço interno;
2. O **filter forward enxerga o pacote traduzido**: destino 192.168.10.10:80, não IP-público:8080. Suas regras de filtro devem casar com o mundo *pós*-dstnat.

19.2 Firewall e dstnat: aceitando com precisão

E o nosso forward aceita esse pacote? Revise o esqueleto do Capítulo 18: conversas novas vindas da WAN não têm accept — na política permissiva de fábrica passariam, mas o hardening completo do Capítulo 21 fechará o forward com drop all. O jeito **errado** de liberar é abrir `dst-address=192.168.10.10 dst-port=80` — funciona, mas duplica a informação (regra de NAT + regra de filtro dizendo o mesmo, que alguém um dia atualiza pela metade). O jeito certo usa o conntrack, que marca conexões traduzidas:

```
/ip firewall filter add chain=forward connection-state=new connection-nat-state=dstnat in-
```

`connection-nat-state=dstnat` casa com **qualquer** conexão que passou pelo dstnat: uma regra de filtro serve todas as publicações, presentes e futuras — a lista de “o que está publicado” mora num lugar só (o NAT). É o mesmo princípio das address lists: política estável, dados num ponto único.

⚠ Aviso

Todo port forwarding é um **portão na muralha**: o serviço interno passa a receber a internet inteira — inclusive os robôs que varrem portas 24/7. Antes de publicar, pergunte: (1) o serviço precisa mesmo ser público, ou uma VPN de gerência resolve (Volume 10)? (2) o destino aguenta exposição (DVRs e câmeras antigos são o recheio favorito das botnets)? (3) dá para restringir a origem (`src-address-list` de IPs confiáveis)? Porta “escondida” (8080 em vez de 80) **não é segurança** — scanners acham em minutos; é só higiene contra ruído.

19.3 Hairpin NAT: o defeito clássico

Publique o DVR e teste **de dentro da LAN** pelo IP público: não abre. De fora (4G), abre. Bem-vindo ao defeito mais famoso do NAT — e à pergunta de entrevista favorita do mundo ISP. A Figura 19.1 mostra o porquê:

Siga os passos: o cliente da LAN mira o IP público (1); o dstnat troca o destino, mas **a origem continua sendo o vizinho de LAN** (2); o servidor responde **direto** pela rede

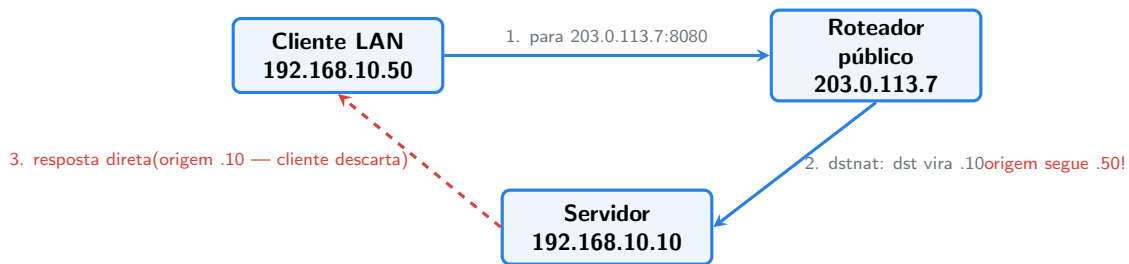


Figura 19.1: O hairpin: a ida faz a volta pelo roteador, mas a resposta corta caminho pela LAN — e chega com o remetente “errado”.

local (3) — e o cliente descarta: ele esperava resposta de 203.0.113.7:8080, recebeu de 192.168.10.10:80. Conversa assimétrica, conexão morta.

A solução: forçar a volta a passar pelo roteador, mascarando **também** a origem quando cliente e servidor são da mesma LAN:

```
/ip firewall nat add chain=srcnat src-address=192.168.10.0/24 dst-address=192.168.10.10 pr
```

Agora o servidor vê a conexão vinda **do roteador** (192.168.10.1) e responde a ele — que desfaz as duas traduções e entrega ao cliente com o remetente esperado. (O preço: o servidor perde o IP real do cliente interno nos logs — todos parecem vir do gateway. A alternativa elegante é *split DNS*: o nome do serviço resolvendo para o IP **interno** quando consultado de dentro — uma entrada em `/ip dns static`, Capítulo 12.)

19.4 redirect: dstnat para o próprio roteador

Uma variação com usos de provedor: `action=redirect` traduz o destino para **o próprio roteador**, na porta indicada:

```
/ip firewall nat add chain=dstnat in-interface-list=LAN protocol=udp dst-port=53 action=re
```

Essa regra intercepta **qualquer** consulta DNS da LAN — mesmo a do dispositivo teimoso configurado com 8.8.8.8 — e a entrega ao resolvidor local (Capítulo 12). Todos usam o cache do provedor, queira o dispositivo ou não. (Com a chegada do DoH essa técnica perdeu alcance — TLS na 443 não se intercepta — mas segue útil para a massa de consultas UDP/53.) O mesmo **redirect** é a base do *transparent proxy* e de portais de captura de hotspot.

! O conceito mais importante do capítulo

Grave a linha do tempo: **dstnat** → **roteamento** → **filter** → **srcnat**. Dela saem as três lições do capítulo: port forwarding funciona porque o destino muda *antes*

do roteamento; o filter deve casar com o pacote *já traduzido* (e `connection-nat-state=dstnat` aceita tudo que o NAT publicou, sem duplicar dados); e o hairpin existe porque a *volta* não repassa pelo roteador — até você mascarar a origem interna também.

19.5 Laboratório

19.5.1 Ambiente

Os três CHRs (Capítulo 5), com um ajuste **só para este capítulo**: no VirtualBox, mova o Adaptador 2 de `lab-r3` da rede interna `lab-b` para a `lab-a` — o `r3` vira um segundo dispositivo na LAN da padaria (cliente do teste de hairpin). Papéis: `lab-r1` = borda (como está), `lab-r2` = “DVR” (servidor a publicar), `lab-r3` = “notebook do dono”.

Em `lab-r3` (console): `/ip dhcp-client add interface=ether2` — ele deve receber um IP `192.168.10.1xx` do `r1`.

19.5.2 Comandos passo a passo

Passo 1 — o “DVR”. O `r2` será o servidor: seu WebFig (porta 80) fará papel da interface do DVR. Em `lab-r2`, garanta o serviço ativo e anote o IP (nos passos abaixo usamos `.105` — substitua pelo real):

```
/ip service enable www
/ip address print where interface=ether2
```

Passo 2 — publicar. Em `lab-r1`, descubra o IP da WAN e crie o port forwarding (8080 fora → 80 dentro):

```
/ip address print where interface=ether3
/ip firewall nat add chain=dstnat in-interface-list=WAN protocol=tcp dst-port=8080 action=
```

E a regra de filtro que aceita o publicado (entra antes de futuros drops do forward):

```
/ip firewall filter add chain=forward connection-state=new connection-nat-state=dstnat in-
```

Passo 3 — testar “de fora” (opcional, mas ótimo). O NAT do VirtualBox esconde o `r1` do mundo, mas o próprio VirtualBox faz port forward: com a VM `lab-r1` selecionada, **Configurações → Rede → Adaptador 3 → Avançado → Redirecionamento de portas**: adicione TCP, porta do hospedeiro 18080, porta do convidado 8080. No navegador do **seu computador**: `http://localhost:18080` → deve abrir o WebFig do `r2`. Você atravessou dois NATs até o “DVR” — de fora, funciona.

Passo 4 — o defeito de dentro. Em lab-r3 (o notebook do dono), acesse pelo IP público do r1 (o da ether3, ex.: 10.0.2.15):

```
/tool fetch url=http://10.0.2.15:8080 keep-result=no
```

Trava e expira — o hairpin da Figura 19.1, ao vivo. (Em lab-r1, a tabela de conexões mostra a conexão presa em SYN: `/ip firewall connection print where dst-port=80.`)

Passo 5 — a cura. Em lab-r1, o masquerade do hairpin:

```
/ip firewall nat add chain=srcnat src-address=192.168.10.0/24 dst-address=192.168.10.105 p
```

Repita o fetch no r3: `status: finished`. De dentro e de fora, o mesmo endereço funciona.

Passo 6 — desfazer o especial. Ao terminar: remova o port forward do VirtualBox (Passo 3), devolva o Adaptador 2 do r3 para lab-b e desative o `www` no r2 (`/ip service disable www`) — o laboratório volta ao padrão.

19.5.3 Verificação

1. `http://localhost:18080` no hospedeiro abriu o WebFig do r2 (se fez o Passo 3) — `dstnat` + filtro por `connection-nat-state` funcionando;
2. O fetch do r3 ao IP público **falhou** antes do Passo 5 e **funcionou** depois — hairpin demonstrado e curado;
3. `/ip firewall nat print` no r1: masquerade original, `dst-nat` da publicação e o masquerade do hairpin, todos comentados;
4. A tabela de conexões mostrou a flag `d` (`dstnat`) nas conexões publicadas.

19.5.4 Desafios

1. Sem executar: um pacote chega da WAN para IP-público:8080. Escreva a sequência completa de processamento no r1 (chains e traduções) até sair para o r2 — e a da resposta.
2. O dono quer as câmeras acessíveis **só do celular dele** (IP móvel muda, mas a operadora publica as faixas). Melhore a publicação do laboratório com uma `address list` `acesso-dvr` — quais regras mudam?
3. Explique por que o split DNS (entrada em `/ip dns static` apontando o nome do serviço para o IP interno) elimina o hairpin — e cite a desvantagem operacional que ele traz num parque com muitos clientes publicados.
4. Um técnico publicou o WinBox do próprio r1 na WAN (“pra facilitar”): `chain=dstnat dst-port=8291 action=redirect to-ports=8291`. Aponte os dois erros — um de segurança, um conceitual.

💡 Solução dos desafios

1. Ida: chega pela ether3 → **dstnat** casa (dst vira 192.168.10.105:80) → decisão de roteamento (destino é a LAN) → **filter forward** (o pacote já traduzido casa com **connection-nat-state=dstnat** → accept; conexão anotada com flag d) → srcnat (masquerade não casa — **out-interface** é LAN, e a regra de hairpin só casa se a origem for da LAN) → entrega ao r2. Volta: r2 responde ao cliente externo → forward (established) → na saída pela WAN o conntrack **desfaz** o dstnat (origem volta a ser IP-público:8080). Simetria por estado, não por regra.
2. Cria-se a lista e aperta-se o **filtro** (o NAT pode ficar):

```
/ip firewall address-list add list=acesso-dvr address=177.0.0.0/8 comment="faixa da oper  
/ip firewall filter set [find comment="fwd: aceita publicados"] src-address-list=acesso-
```

(Se houver mais publicações com políticas distintas, regras de filtro separadas por serviço — a genérica deixa de servir.)

3. De dentro, o nome resolve para 192.168.10.105: o cliente fala **direto** com o servidor, sem tocar no IP público — não há tradução, não há assimetria, e os logs do servidor veem o IP real do cliente interno (o que o hairpin-masquerade esconde). Desvantagem: é **um cadastro a mais** por serviço/cliente, no DNS de cada roteador envolvido — esquecer de atualizar ao mudar o IP interno gera o defeito inverso (“de fora funciona, de dentro abre a coisa errada”). Automação (Volume 12) ou disciplina.
4. Segurança: expor a **gerência** do roteador à internet é o pecado capital — força bruta e exploits contra WinBox são varredura de rotina; gerência remota se faz por VPN (Volume 10). Conceitual: **redirect** já entrega ao próprio roteador — mas o WinBox **já escuta** na 8291; a regra é inútil (não fosse o **/ip service** com **allowed-address** e o input com drop all, que — felizmente — continuam barrando o acesso; defesa em camadas salvando o dia de novo).

19.6 Resumo

- **dstnat** troca o destino **na entrada, antes do roteamento** — a linha do tempo dstnat → roteamento → filter → srcnat explica o capítulo inteiro.
- Port forwarding: **chain=dstnat in-interface-list=WAN dst-port=X action=dstnat to-addresses=interno to-ports=Z** — e o filtro aceita por **connection-nat-state=dstnat**, sem duplicar dados.
- Publicar é abrir portão: questione a necessidade (VPN?), restrinja origem quando possível, e lembre que porta alta não é segurança.
- **Hairpin**: de dentro, a resposta corta caminho e chega com remetente errado (Figura 19.1) — cura-se mascarando a origem interna (ou, mais elegante, com split DNS).
- **redirect** entrega ao próprio roteador: interceptação de DNS, proxy transparente, portais de hotspot.

No próximo capítulo, as duas tabelas que faltam no arsenal: **mangle** (marcar pacotes e conexões — a fundação do QoS do Volume 7) e **RAW** (agir antes do contrack — a defesa barata contra floods).

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall nat` (actions `dst-nat`, `redirect`, `netmap`) e o diagrama de packet flow que fundamenta a linha do tempo do capítulo; a página *Hairpin NAT* traz o caso clássico com variações. As referências completas, em ABNT, estão no apêndice [Referências](#).

20 Mangle e RAW

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Situar as tabelas **mangle** e **RAW** na linha de montagem do pacote — e por que a posição de cada uma define seu poder;
- Usar o mangle para **marcar** conexões e pacotes (**mark-connection/mark-packet**) com o padrão eficiente “marca a conexão uma vez, herda nos pacotes”;
- Entender para quem as marcas trabalham: filas de QoS (Volume 7) e decisões de rota (**routing-mark**, Volumes 8 e 14);
- Usar o RAW para agir **antes do connection tracking**: descartar floods barato (**drop**) e isentar tráfego do conntrack (**notrack**);
- Prever os efeitos colaterais do **notrack** — o que quebra quando um pacote vira invisível para NAT e firewall stateful.

20.1 A linha de montagem completa

Você já conhece três estações da fábrica: o filter julga, o NAT traduz, o conntrack anota. Faltavam duas — e a posição delas na esteira é a chave do capítulo (Figura 20.1):

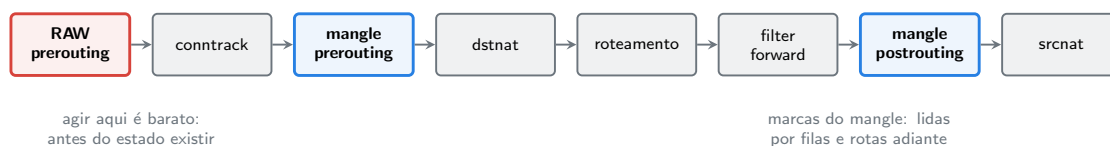


Figura 20.1: A esteira do pacote que atravessa o roteador: o RAW é a primeira estação (antes do conntrack); o mangle marca ao longo do caminho para consumo das estações seguintes.

- O **RAW** é a **primeira** estação: vê o pacote cru, antes mesmo de o conntrack procurar/criar a conexão. Poder: agir **barato** — e decidir se o conntrack sequer verá aquele tráfego;
- O **mangle** tem regras em **todas** as etapas (prerouting, input, forward, output, postrouting) e não aceita nem descarta: ele **anota** — marcas invisíveis que viajam com o pacote *dentro do roteador* e são lidas pelas estações seguintes.

20.2 Mangle: o almoxarifado de etiquetas

O mangle marca três coisas, cada uma com seu consumidor:

Tabela 20.1: As marcas do mangle e seus consumidores

Marca	Action	Quem consome
de conexão	<code>mark-connection</code>	o próprio mangle (herança)
de pacote	<code>mark-packet</code>	filas de QoS (Volume 7)
de rota	<code>mark-routing</code>	decisão de roteamento (Volumes 8 e 14)

As etiquetas não mudam o pacote na rede — são metadados internos. E o padrão de uso profissional aproveita o `conntrack` para marcar **barato**:

```
/ip firewall mangle
add chain=forward connection-state=new protocol=udp dst-port=53 action=mark-connection new-connection-mark=conn-dns
add chain=forward connection-mark=conn-dns action=mark-packet new-packet-mark=pkt-dns pass
```

Leia o par com atenção — ele voltará no QoS inteiro:

1. A primeira regra classifica **só o primeiro pacote** da conexão (`connection-state=new`) e grava a marca **na conexão** (na tabela do `conntrack` — você a verá lá no laboratório);
2. A segunda marca **cada pacote** consultando apenas a marca da conexão (`connection-mark=conn-dns`) — sem reavaliar portas/protocolos a cada pacote. Classificação cara $1\times$ por conexão; etiquetagem barata por pacote. É o `established,related` do mangle;
3. **passthrough: yes** = o pacote continua descendo a lista do mangle (permite acumular marcas: conexão + pacote); **no** = para aqui (a etiquetagem final é terminal). O padrão dos pares classifica-e-etiqueta é exatamente esse: **yes** na primeira, **no** na segunda.

Um terceiro uso do mangle, sem marca alguma, resolve um defeito clássico de provedor: **change-mss** — ajustar o tamanho máximo de segmento TCP quando o caminho tem MTU menor (túneis, PPPoE). Guarde o nome; o Volume 6 o usará no concentrador (`action=change-mss ... tcp-flags=syn`), e ele explica metade dos chamados “abre uns sites e outros não”.

20.3 RAW: agir antes de existir

As regras do RAW (`/ip firewall raw`) vivem em `prerouting` (e `output`), **antes do `conntrack`** — e essa posição paga duas contas:

1. **action=drop** — a defesa barata contra flood. Relembre o desafio do Capítulo 15: um flood de conexões novas enche a tabela e derruba clientes legítimos. Um **drop no filter** descarta o pacote, mas *tarde demais* — o `conntrack` já gastou CPU e criou a entrada. O mesmo drop no **RAW** mata o pacote antes de qualquer estado nascer:

```
/ip firewall raw add chain=prerouting in-interface-list=WAN src-address-list=atacantes act
```

Sob ataque volumétrico, essa diferença é o roteador de pé ou caído. O padrão de operação: detectar as origens (listas dinâmicas do Capítulo 18, ou à mão durante o incidente) e despachá-las no RAW.

2. action=notrack — isentar do conntrack. Tráfego que não precisa de NAT nem de firewall stateful pode ser dispensado da contabilidade:

```
/ip firewall raw add chain=prerouting src-address=10.255.0.0/24 dst-address=10.255.0.0/24
```

Cada pacote isento economiza a busca/criação na tabela — em roteadores de core ou enlaces de infraestrutura com milhões de pacotes por segundo, é CPU de volta ao caixa. Mas o **notrack** tem dentes:

Aviso

Pacote **notrack** é **invisível** para tudo que depende do **conntrack**: NAT não o traduz, matchers de **connection-state** não casam (o estado vira **untracked**) e as regras **established,related** **não o aceitam**. Aplicar **notrack** em tráfego que passa por NAT quebra a navegação na hora — o laboratório demonstra de propósito. Use-o apenas em tráfego que comprovadamente não precisa de estado (infraestrutura, roteamento puro) e **teste os firewalls no caminho**: um esqueleto que só aceita **established,related** + drop all descarta o **untracked** silenciosamente.

O conceito mais importante do capítulo

Posição é poder: o **RAW** age antes do estado existir (defesa barata, isenção de contabilidade — com os riscos de quem fica invisível), e o **mangle** anota ao longo da esteira para as estações seguintes decidirem (filas, rotas). Nenhum dos dois “bloqueia ou libera” — RAW poupa o roteador; mangle informa o roteador. O filter continua sendo o juiz.

20.4 Laboratório

20.4.1 Ambiente

lab-r1 (borda completa dos capítulos anteriores) e **lab-r2** (cliente na LAN). O **r3** já voltou ao padrão (Capítulo 19, Passo 6).

20.4.2 Comandos passo a passo

Passo 1 — o par classifica-e-etiqueta. Em lab-r1, marque o tráfego ICMP dos clientes (nosso “tráfego prioritário” de mentira — no Volume 7 será VoIP):

```
/ip firewall mangle add chain=forward connection-state=new protocol=icmp action=mark-connection
/ip firewall mangle add chain=forward connection-mark=conn-ping action=mark-packet new-packet-mark=conn-ping
```

Passo 2 — ver as etiquetas. Em lab-r2, um ping contínuo a 8.8.8.8. Em lab-r1, veja a **marca na conexão** e os contadores do par:

```
/ip firewall connection print detail where protocol=icmp
/ip firewall mangle print stats
```

Na conexão: `connection-mark=conn-ping`. Nos contadores: a regra 0 (classifica) com **1** pacote; a regra 1 (etiqueta) crescendo a cada ping — o padrão barato, visível nos números. Pare o ping.

Passo 3 — RAW drop: o porteiro de fora. Simule o despacho de um atacante. Em lab-r1:

```
/ip firewall raw add chain=prerouting src-address=192.168.10.0/24 dst-address=1.1.1.1 action=drop
```

Em lab-r2: `/ping 1.1.1.1 count=3` — morto. Agora o detalhe fino: em lab-r1, procure a conexão:

```
/ip firewall connection print where dst-address~"1.1.1.1"
```

Nada. O pacote morreu antes do conntrack — compare com um drop de filter, que deixaria... também nada de conexão (drop no new impede a entrada), mas o contraste real é o custo: no RAW, nem a **busca** na tabela aconteceu. Remova o teste:

```
/ip firewall raw remove [find comment="teste RAW drop"]
```

Passo 4 — notrack quebrando o NAT (de propósito). A demonstração mais instrutiva do capítulo. Em lab-r1:

```
/ip firewall raw add chain=prerouting src-address=192.168.10.0/24 protocol=icmp action=notrack
```

Em lab-r2: `/ping 8.8.8.8 count=3` — **falha**. Por quê? O ping saiu `untracked`: o masquerade (que depende do conntrack) não o traduziu, e o pacote foi para a internet com origem 192.168.10.x — o timeout do Capítulo 13 de novo, agora causado por *excesso de otimização*. Confirme a invisibilidade:

```
/ip firewall connection print where protocol=icmp
```

Vazio, mesmo com o ping rodando. Desfaça e confirme que voltou:

```
/ip firewall raw remove [find comment="teste notrack"]
```

Passo 5 — limpar o mangle de teste. As marcas de ping cumpriram seu papel didático:

```
/ip firewall mangle remove [find comment~"ping"]
```

20.4.3 Verificação

1. A conexão ICMP exibiu `connection-mark=conn-ping` e os contadores do par mostraram 1 pacote na classificação e N na etiquetagem;
2. Com o RAW drop, o destino de teste morreu **sem** gerar entrada no conntrack;
3. Com o notrack, o ping dos clientes quebrou (NAT cego) e a tabela de conexões ficou vazia para ICMP — e você sabe explicar a cadeia causal;
4. Tudo removido ao final (`/ip firewall raw print` e `.. mangle print` limpos).

20.4.4 Desafios

1. No par classifica-e-etiqueta, o que aconteceria se a **primeira** regra usasse `passthrough=no`? E se a **segunda** usasse `passthrough=yes`?
2. Sob um flood UDP de 500 mil pps contra um cliente, compare o custo por pacote de três defesas: drop no filter forward, drop no RAW prerouting, e blackhole na operadora (pedir para o upstream descartar). Quando cada uma é a resposta certa?
3. Escreva a regra de mangle `change-mss` para clientes PPPoE com MTU 1492 (MSS 1452) — chain, matchers de tcp-flags e sentido do tráfego — e explique por que ela casa só com pacotes SYN.
4. Um roteador de core sem NAT e sem filtro stateful roda `connection tracking enabled=auto`. Há regra RAW `notrack` para tudo. O conntrack está de fato desligado? O que o `auto` decide — e qual a diferença prática entre “notrack em tudo” e `enabled=no`?

Solução dos desafios

1. Com `passthrough=no` na primeira, o primeiro pacote da conexão para ali: recebe marca de conexão mas **nunca chega** à regra de pacote — o primeiro pacote de cada conexão viaja sem etiqueta (em filas, o SYN escaparia da fila certa; sutil e real). Com `yes` na segunda, os pacotes etiquetados continuariam descendo o mangle — inofensivo se não há mais regras, mas paga-se o percurso; em listas grandes, é desperdício por pacote.
2. Filter forward: o pacote já passou por RAW+conntrack+mangle+ roteamento —

CPU cheia por pacote; certo quando o volume é pequeno ou a decisão exige estado. RAW: só o custo de chegada + um matcher — aguenta uma ordem de grandeza a mais; a resposta certa on-box para floods. Mas a 500 kpps o **enlace** já está pago: se o flood satura a banda de entrada, descartar dentro de casa não devolve banda — a resposta certa é o **blackhole no upstream** (a operadora descarta lá, e o seu link respira). Escada: filter < RAW < upstream, conforme o gargalo sobe.

3.

```
/ip firewall mangle add chain=forward protocol=tcp tcp-flags=syn action=change-mss new-m
```

O MSS é negociado **no aperto de mão**: só os pacotes SYN carregam a opção — ajustar os demais é inútil. O matcher `tcp-mss=1453-65535` evita *aumentar* MSS já menores. (No Volume 6, a versão nos dois sentidos e a discussão de MTU completa.)

4. Com **auto**, o `conntrack` liga se **existir** regra que o use — e a própria regra RAW conta como uso do subsistema: o módulo fica ativo, processando o notrack por pacote (barato, mas não zero) e mantendo a infraestrutura da tabela. `enabled=no` desliga o subsistema inteiro: nenhuma consulta, nenhum estado, custo mínimo absoluto — e sem exceções possíveis. “Notrack em tudo” = flexível (dá para excetuar a gerência, p.ex.); `no` = mais rápido e radical. Core puro: `no`; core que ainda faz alguma coisa stateful: `notrack` seletivo.

20.5 Resumo

- A esteira completa (Figura 20.1): **RAW** → **conntrack** → **mangle** → **dstnat** → **roteamento** → **filter** → **mangle** → **srcnat** — posição é poder.
- **Mangle** anota (conexão, pacote, rota — Tabela 20.1) para consumidores adiante (filas, roteamento); o padrão **classifica-1×-e-etiqueta-por-herança** com `passthrough` certo é a base de todo QoS eficiente.
- **change-mss** no mangle: o remédio dos “sites que não abrem” em caminhos de MTU reduzida (PPPoE, túneis).
- **RAW**: `drop` antes do `conntrack` é a defesa barata contra floods; `notrack` isenta da contabilidade — e torna o tráfego invisível para NAT e stateful (**quebra NAT**, escapa de `established,related`).
- A escada anti-flood: filter → RAW → blackhole no upstream, conforme o gargalo sobe.

No próximo capítulo, o gran finale do volume: o **hardening completo** — todos os tijolos deste volume montados num checklist consolidado de roteador de provedor.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/ip firewall mangle` (marcas e `change-mss`), `/ip firewall raw` e o diagrama de packet flow que este capítulo resume. A página

DDoS protection da mesma documentação traz padrões RAW + address lists dinâmicas. As referências completas, em ABNT, estão no apêndice [Referências](#).

21 Hardening completo

i Objetivos de aprendizagem

Este capítulo consolida o volume. Ao final, você será capaz de:

- Enxergar a segurança do roteador como **camadas independentes** — da superfície de serviços ao RAW — e saber o que mora em cada uma;
- Montar e aplicar o **firewall-base** do provedor: o `.rsc` comentado que todo equipamento do parque recebe;
- Acrescentar as peças finais: defesa de força bruta em cascata, descarte de bogons, FastTrack (e seu preço) e logging do que importa;
- Auditar um roteador em produção com um **checklist** repetível;
- Manter o firewall-base **vivo**: versionado, auditado e aplicado ao parque inteiro.

21.1 Defesa em camadas: o mapa do volume

Cada capítulo deste volume construiu uma camada; o hardening é reconhecê-las como um sistema (Figura 21.1):

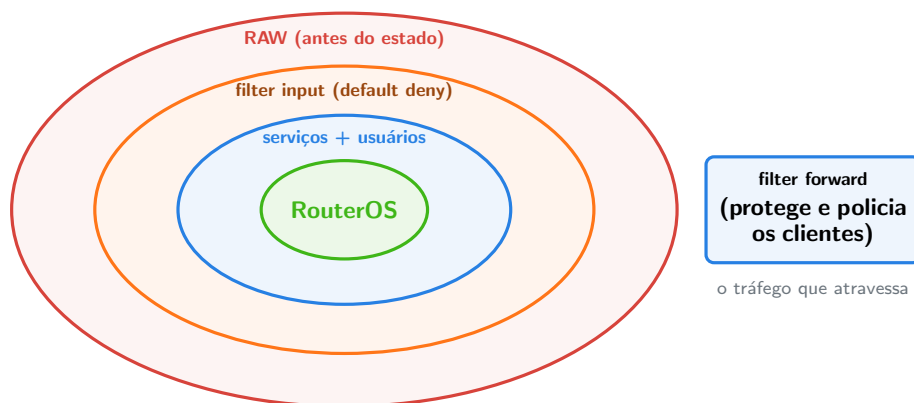


Figura 21.1: As camadas do hardening: cascas concêntricas protegem o roteador; o forward, ao lado, cuida do que atravessa.

A propriedade que importa: as camadas são **independentes**. O `allowed-address` do serviço segura mesmo se o firewall for esvaziado; o input segura mesmo se um serviço for habilitado por engano; o RAW segura a CPU mesmo com o input sob flood. Segurança de

provedor não é uma muralha alta — são várias muralhas medíocres que falham em momentos diferentes.

21.2 As peças finais

Quatro adições transformam o firewall dos capítulos anteriores no firewall-base definitivo.

1. Bogons na fronteira. Pacotes chegando da WAN com origem privada (RFC 1918), loopback ou reservada são, por definição, forjados ou vazados — nenhum deles tem o que fazer batendo na sua borda:

```
/ip firewall address-list
add list=bogons address=10.0.0.0/8 comment="RFC1918"
add list=bogons address=172.16.0.0/12 comment="RFC1918"
add list=bogons address=192.168.0.0/16 comment="RFC1918"
add list=bogons address=127.0.0.0/8 comment="loopback"
add list=bogons address=100.64.0.0/10 comment="CGNAT space"
add list=bogons address=169.254.0.0/16 comment="link-local"

/ip firewall raw add chain=prerouting in-interface-list=WAN src-address-list=bogons action=drop
```

No RAW, pela lição do 4: lixo certo morre barato. (Atenção do mundo real: se a sua “WAN” recebe endereço de CGNAT da operadora — 100.64.0.0/10 — ajuste a lista; o laboratório com NAT do VirtualBox usa 10.0.2.x, e o laboratório tratará disso.)

2. Força bruta em cascata. O desafio do Capítulo 18, agora oficial — a escada de listas dinâmicas que pega quem insiste no SSH da WAN sem punir o técnico que errou a senha uma vez:

```
/ip firewall filter
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=bogons action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN protocol=tcp dst-port=22 connection-state=new src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
add chain=input in-interface-list=WAN src-address-list=ssh-bloqueado action=drop comment="SSH blocked"
```

(Repare a ordem invertida — a regra da 4ª tentativa vem **primeiro**, senão toda conexão pararia no estágio 1. E lembre: no nosso desenho, SSH da WAN nem deveria existir — a cascata é a rede de segurança para o dia em que alguém abrir uma exceção.)

3. FastTrack — desempenho com letras miúdas. A regra que faz um hEX de R\$ 300 rotear como gente grande:

```
/ip firewall filter add chain=forward action=fasttrack-connection hw-offload=yes connection
```

fasttrack-connection marca a conexão para pular quase toda a esteira da Figura 20.1 nos pacotes seguintes — multiplicando o throughput. O preço: pacotes fastrackeados **não passam** por mangle, filas e pela maior parte do filter. Num roteador de borda simples, ligue; num concentrador com QoS (Volume 7), a decisão muda — as filas precisam ver os pacotes. A regra convive com a gêmea **accept** logo abaixo (nem toda conexão é fastrackeável; a **accept** apanha o resto).

4. Logar o que importa. Log de tudo é ruído; log de nada é cegueira. O equilíbrio do provedor: logar **negações interessantes** (tentativas na gerência, publicados) e **mudanças de listas dinâmicas** — os **add-to-list** já contam a história dos ataques em `/ip firewall address-list print`. Uma regra exemplar:

```
/ip firewall filter add chain=input in-interface-list=WAN protocol=tcp dst-port=8291 actio
```

Se um dia esse prefixo aparecer em rajada no log, você saberá que estão procurando por você — antes de acharem.

21.3 O checklist consolidado

O volume inteiro numa tabela — a auditoria repetível de qualquer roteador do parque:

Tabela 21.1: Checklist de hardening do roteador de provedor

Camada	Item	Referência
Superfície	serviços mínimos + allowed-address ; admin off ; usuários nominais	5
Superfície	discovery e MAC-server só na gerencia	5
Listas	WAN/LAN/gerencia refletem a topologia real	Capítulo 17
Input	esqueleto stateful + gerência + ICMP c/ teto + serviços da LAN + drop all	Capítulo 17
Input	cascata anti força bruta nas portas de gerência	este capítulo

Camada	Item	Referência
Forward	esqueleto + antispoofing + porta 25 + gerência intocável + drop de publicações órfãs	Capítulo 18
Forward	<code>connection-nat-state=dstnat</code> para publicados; FastTrack conforme o papel	Capítulo 19, este
RAW	bogons da WAN; despacho de flood; notrack só onde provado	4, este
Operação	<code>export</code> versionado; contadores/logs revisados; teste de acesso pós-mudança	3

! O conceito mais importante do capítulo

Hardening não é um evento — é um **padrão versionado**. O produto deste volume é um `firewall-base.rsc` comentado, guardado em repositório, que todo equipamento novo recebe (Netinstall + configure script, Capítulo 8) e contra o qual todo equipamento antigo é auditado (Tabela 21.1). Segurança que depende de memória humana regride; segurança que mora num arquivo versionado acumula.

21.4 Laboratório

21.4.1 Ambiente

`lab-r1` com tudo do volume aplicado (é o estado em que o 4 o deixou) e `lab-r2` na LAN. O trabalho final: completar, testar como um auditor e **exportar o padrão**.

21.4.2 Comandos passo a passo

Passo 1 — bogons. Aplique a lista e a regra RAW da seção anterior — com **uma adaptação de laboratório**: a nossa “WAN” do VirtualBox usa 10.0.2.x, que está dentro de 10.0.0.0/8... incluir essa linha bloquearia o gateway do laboratório. Aplique a lista **sem** a primeira entrada (e anote no comentário o porquê — auditoria futura agradece):

```

/ip firewall address-list
add list=bogons address=172.16.0.0/12 comment="RFC1918"
add list=bogons address=192.168.0.0/16 comment="RFC1918"
add list=bogons address=127.0.0.0/8 comment="loopback"
add list=bogons address=169.254.0.0/16 comment="link-local"
add list=bogons address=100.64.0.0/10 comment="CGNAT space"
/ip firewall raw add chain=prerouting in-interface-list=WAN src-address-list=bogons action

```

Passo 2 — cascata anti-bruta. Aplique as cinco regras da seção anterior (copie do texto). Em seguida, o teste: a nossa WAN não recebe conexões externas, então simule **pela LAN** trocando temporariamente o matcher da 1ª tentativa (`in-interface-list=LAN`)... ou melhor: entenda o mecanismo com o desafio 4 do Capítulo 18 já feito e apenas confira a sintaxe com `print`. (Auditor também sabe quando **não** repetir um teste destrutivo.)

Passo 3 — FastTrack. No nosso roteador de borda, sem filas por enquanto — liga:

```

/ip firewall filter add chain=forward action=fasttrack-connection hw-offload=yes connection

```

Em `lab-r2`, gere tráfego (`/tool fetch url=https://mikrotik.com keep-result=no`) e veja as conexões marcadas em `lab-r1`:

```

/ip firewall connection print where fasttrack

```

A flag `F` nas conexões: o atalho está ativo.

Passo 4 — o log-sentinela. A regra do WinBox-da-WAN logado (copie da seção). Confira a posição: acima do `drop all`, abaixo dos `accepts`.

Passo 5 — auditoria completa. Agora, vista o chapéu de auditor e percorra a Tabela 21.1 item a item contra o roteador real:

```

/ip service print
/user print
/interface list member print
/ip firewall filter print where chain=input
/ip firewall filter print where chain=forward
/ip firewall raw print
/ip firewall address-list print

```

Cada linha do checklist deve apontar para algo que você **vê** na saída. Achou divergência? Corrija agora — é exatamente para isso que a auditoria existe.

Passo 6 — a bateria de fumaça. O teste funcional de sempre, agora completo: sessão de gerência viva; `lab-r2` navega (IP e nome); DHCP renova; SSH da LAN ao `r1` morre; SMTP 25 morre com aviso; gerência inalcançável do `r2`; ping ao `r1` funciona (com teto).

Passo 7 — o padrão nasce. Exporte e guarde:

```
/export file=firewall-base-v1
/system backup save name=hardening-completo
```

Baixe os dois (`scp`) e — hábito novo — **versione**: o `firewall-base-v1.rsc` é o primeiro commit do padrão de segurança do seu provedor fictício. Todo capítulo futuro que mexer em firewall parte dele.

E crie o snapshot **hardening-completo** nas VMs `lab-r1` e `lab-r2`: os próximos volumes partem daqui.

21.4.3 Verificação

1. Auditoria do Passo 5 sem divergências (ou com as correções feitas e anotadas);
2. Bateria de fumaça do Passo 6: 7 de 7;
3. Conexões com flag **F** (FastTrack vivo) e bogons na RAW com o comentário de adaptação do laboratório;
4. `firewall-base-v1.rsc` fora do equipamento e snapshot **hardening-completo** criado.

21.4.4 Desafios

1. A cascata anti-bruta tem um falso positivo clássico: o próprio técnico com automação (Ansible, scripts) abrindo várias conexões SSH legítimas em sequência. Proponha duas mitigações que não desliguem a defesa.
2. Com FastTrack ligado, o par classifica-e-etiqueta do 4 voltaria a marcar todos os pacotes de uma conexão? Teste mentalmente (ou no laboratório) e explique — e diga qual regra teria que mudar para um futuro QoS funcionar.
3. Ordene os itens do checklist (Tabela 21.1) do **maior para o menor impacto** se estiverem errados, na sua avaliação — e defenda as duas primeiras posições.
4. Seu parque tem 300 equipamentos com firewalls “cada um de um jeito”. Desenhe (em texto) o plano de convergência para o firewall-base em 90 dias sem derrubar ninguém: fases, ferramentas deste livro e critérios de sucesso.

Solução dos desafios

1. (a) **Isentar a origem certa**: os accepts da gerência (lista `gerencia/address list` dos IPs do NOC) ficam **acima** da cascata — quem é de casa nunca entra na contagem; (b) **contar só falhas de fato**: subir o limiar (estágios com timeout maior, ou contar a partir da 2ª conexão em janela curta) e/ou usar portas distintas para automação (SSH de gestão em porta/endereço dedicados de gerência, fora da WAN). A defesa é para a fronteira; automação não deveria vir de lá.
2. Não: pacotes fastrackeados **pulam o mangle** — só o primeiro punhado de pacotes de cada conexão (antes de a marca FastTrack pegar) seria etiquetado. Filas veriam quase nada. Para QoS funcionar, a regra de FastTrack precisa **excluir** o tráfego que se quer enfileirar — p.ex. `connection-mark=no-mark` (fastrackear só o não classificado)

ou remover o FastTrack no concentrador. É a tensão desempenho × visibilidade do Volume 7.

3. Resposta modelo (defensável, não única): 1º **input sem drop all** — o roteador inteiro exposto é comprometimento de infraestrutura, raio de dano máximo (atacante dentro = todas as outras camadas viram decoração); 2º **superfície de serviços aberta** (senha fraca no admin + WinBox na WAN é a dupla que mais derruba provedor no mundo real — explora sem “furar” firewall nenhum, só entrando pela porta). Os demais causam danos graves porém mais contidos (spoofing/25 = reputação; bogons/RAW = resiliência; logs = tempo de detecção).

4. Fases: (1) **Inventário** (semanas 1–2): coletar /export de todos (script/API — Volume 12), diffs contra o base, classificar por distância; (2) **Base mínima sem risco** (semanas 3–6): aplicar em massa o que não muda comportamento (usuários, serviços, listas, logs) via script testado em laboratório-espelho; (3) **Firewall em ensaio** (semanas 7–10): input/forward novos com drop all em modo log por 1 semana por lote de equipamentos, auditar logs, então apertar — lotes pequenos, horário de manutenção, Safe Mode/backup antes de cada um; (4) **Selo** (semanas 11–13): auditoria da Tabela 21.1 por amostragem + alarme de desvio (export diário comparado ao base). Sucesso: 100% dos equipamentos com diff vazio contra o base, zero chamados atribuíveis à migração.

21.5 Resumo

- Segurança é **camadas independentes** (Figura 21.1): superfície → input → forward → RAW — cada uma segura quando as outras falham.
- As peças finais: **bogons** no RAW, **cascata anti-bruta** (ordem invertida!), **FastTrack** (desempenho ao preço da visibilidade — QoS exigirá exceções) e **logs-sentinela** com prefixo.
- O produto do volume é um **firewall-base.rsc** comentado e versionado — aplicado a todo equipamento novo, auditado contra todo equipamento antigo (Tabela 21.1).
- Auditoria é ver cada item do checklist na saída real do equipamento — e a bateria de fumaça funcional fecha toda mudança.

Fecha-se a Fase 1 do trivium do firewall... e abre-se o Volume 4: até aqui, roteamos entre redes; agora vamos **segmentar dentro** delas — bridge, VLANs e o switching em hardware dos CRS.

Bibliografia do capítulo

As páginas *Securing your router*, *Building Advanced Firewall* e *DDoS Protection* da documentação oficial (MikroTik, 2026a) são as fontes dos padrões consolidados aqui — vale lê-las na íntegra agora que você tem o volume todo como contexto. O CERT.br publica

recomendações de segurança para provedores brasileiros (cert.br/docs/). As referências completas, em ABNT, estão no apêndice [Referências](#).

Parte IV

Volume 4 — Bridge, switching e VLANs

22 Bridge e hardware offload

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Diferenciar **rotear** (camada 3) de **comutar** (camada 2) — e dizer qual dos dois cada problema pede;
- Explicar o que uma **bridge** faz: aprendizado de MACs, encaminhamento por tabela e inundação do desconhecido;
- Criar bridges no RouterOS, adicionar portas e ler a *host table*;
- Explicar o **hardware offload** (a flag H): quando o switch chip comuta sozinho, quando o trabalho cai na CPU — e quais recursos matam o offload;
- Reconhecer os papéis da bridge na rede do provedor: LAN de CPE, switch de POP e o motor por trás das VLANs dos próximos capítulos.

22.1 Camada 2: o andar de baixo

Três volumes inteiros olhando endereços IP — e no entanto, quando dois computadores da mesma LAN conversam, o IP quase não trabalha: quem entrega os quadros é a **camada 2**, endereçando por **MAC**. O RouterOS, que até aqui usamos como roteador, também é um **switch** — e a peça que faz isso é a **bridge**.

A distinção que organiza o volume:

- **Rotear** (camada 3): conectar redes **diferentes** — decisão por tabela de rotas, TTL decrementado, firewall na travessia. É o que fizemos até aqui;
- **Comutar** (camada 2): conectar equipamentos da **mesma** rede — decisão por tabela de MACs, transparente para o IP. É o que um switch faz, e o que a bridge do RouterOS faz.

Uma bridge é um **switch de software**: uma interface virtual que agrupa portas físicas e comuta quadros entre elas. O algoritmo cabe em três verbos:

1. **Aprende**: cada quadro que chega ensina “o MAC de origem X mora na porta Y” — anotado na *host table*;
2. **Encaminha**: quadro para MAC conhecido sai **só** pela porta onde ele mora;
3. **Inunda**: quadro para MAC desconhecido (ou broadcast) sai por **todas** as portas, menos a de origem — e a resposta ensinará o caminho.

💡 Analogia para guardar

A bridge é um **carteiro de condomínio** que começa sem conhecer ninguém: a primeira carta para um morador desconhecido é anunciada no pátio (inundação); quando o morador responde, o carteiro anota o bloco (aprendizado) e as próximas cartas vão direto (encaminhamento). O roteador, por contraste, é o **correio entre cidades** — outro andar do sistema.

22.2 Bridge no RouterOS

Criar o switch de software e povoá-lo:

```
/interface bridge add name=bridge-lan comment="LAN do escritorio"  
/interface bridge port add bridge=bridge-lan interface=ether2  
/interface bridge port add bridge=bridge-lan interface=ether3  
/interface bridge port add bridge=bridge-lan interface=ether4
```

Pronto: **ether2–ether4** agora são portas do mesmo “switch”, e dispositivos ligados a elas conversam em camada 2. Três consequências imediatas que mudam como você configura:

- **A bridge é uma interface** — e é *nela* que a configuração de camada 3 mora a partir de agora: o IP da LAN vai em **interface=bridge-lan**, o DHCP server escuta na bridge, as regras de firewall referenciam a bridge. As portas físicas viram “buracos de tomada”: nenhuma configuração IP nelas;
- **A host table conta a verdade** (**/interface bridge host print**): que MAC está em que porta, aprendido ao vivo — a primeira parada de todo diagnóstico “o dispositivo não aparece”;
- **Quem entra na bridge deixa de ser individual**: um endereço IP esquecido numa porta-membro é receita de comportamento bizarro.

Na rede do provedor, a bridge está em toda parte: é a LAN do CPE na casa do cliente (portas + WiFi na mesma bridge), o switch de gerência de uma torre (rádios + roteador no mesmo segmento) e — no CRS — o switch do POP inteiro. E aqui entra a pergunta de desempenho.

22.3 Hardware offload: quem carrega o piano?

Uma bridge de software comuta na **CPU**: cada quadro sobe, é decidido e desce. Num CHR ou num roteador pequeno servindo uma casa, tudo bem. Num CRS com 24 portas a 1 Gbps, seria suicídio — e é por isso que os CRS (2) carregam um **switch chip**: um circuito que comuta sozinho, em velocidade de fio, sem acordar a CPU.

O elo entre os dois mundos é o **hardware offload**: quando as condições permitem, o RouterOS *delega* a comutação da bridge ao switch chip. O sinal está na flag **H** de cada porta:

```
/interface bridge port print
```

```
[admin@crs-pop] > /interface bridge port print
Flags: I - INACTIVE; H - HW-OFFLOAD
Columns: INTERFACE, BRIDGE, HW
#    INTERFACE  BRIDGE      HW
0 H  ether1      bridge-pop  yes
1 H  ether2      bridge-pop  yes
2 H  ether3      bridge-pop  yes
```

H presente = aquele tráfego não gasta CPU. E a regra de ouro do capítulo: **recursos de software matam o offload**. A bridge só delega o que o chip sabe fazer; peça algo que exija a CPU e o RouterOS silenciosamente traz o tráfego de volta para o software — o switch de 200 Gbps vira um roteador de 500 Mbps sem avisar. Os assassinos clássicos do H:

Tabela 22.1: O que mata (e o que preserva) o hardware offload

Recurso na bridge	Efeito no offload
<code>use-ip-firewall=yes</code> (firewall vendo tráfego da bridge)	mata (tudo passa pela CPU)
VLAN filtering em chips antigos (CRS1xx/2xx)	mata; nos CRS3xx+, preservado
STP/RSTP	preservado (protocolos vão à CPU; dados não)
Bonding de portas	depende do chip
Mesma bridge misturando portas de chips diferentes	mata nas portas fora do chip

A tabela tem uma linha que anuncia o futuro: **VLAN filtering com offload nos CRS3xx** é exatamente o assunto dos próximos capítulos — segmentar POPs inteiros em velocidade de fio. E tem uma pegadinha de diagnóstico que vale sublinhar: a perda do offload **não gera erro** — só CPU alta e throughput baixo. `/interface bridge port print` procurando H ausente é o primeiro reflexo do técnico de POP.

! O conceito mais importante do capítulo

A bridge é o switch de software do RouterOS — IP, DHCP e firewall passam a morar **nela**, não nas portas. E desempenho de switching é uma pergunta só: **a flag H está lá?** Com offload, o switch chip comuta em velocidade de fio; sem ele, cada quadro custa CPU — e o RouterOS troca um pelo outro em silêncio quando você pede recursos que

o chip não sabe fazer (Tabela 22.1).

22.4 Laboratório

22.4.1 Ambiente

Os três CHRs (Capítulo 5), a partir dos snapshots **hardening-completo** (r1, r2) e **base-limpa** (r3). Ajuste **para este volume** (vale para os próximos capítulos): no VirtualBox, mova o Adaptador 2 de **lab-r3** para a rede interna **lab-a** — r2 e r3 ficam ambos “espetados” no r1, que fará papel de switch. (No CHR não há switch chip — a flag H não aparecerá; o offload é conceitual aqui e real no CRS do seu POP. Tudo o mais é idêntico.)

22.4.2 Comandos passo a passo

Passo 1 — desmontar a LAN roteada. No lab-r1, a LAN atual mora na **ether2** física. Vamos migrá-la para uma bridge — a operação clássica de quem “ganha portas”. Primeiro, crie a bridge e mova o IP (Safe Mode, claro — mexer onde mora o DHCP da LAN merece cinto):

```
/interface bridge add name=bridge-lan comment="LAN em bridge"  
/ip address set [find address="192.168.10.1"] interface=bridge-lan  
/ip dhcp-server set dhcp-lan interface=bridge-lan
```

Passo 2 — portas na bridge. **ether2** (onde vive o r2/r3 via lab-a) vira porta:

```
/interface bridge port add bridge=bridge-lan interface=ether2
```

E atualize a interface list (o firewall do Volume 3 referencia LAN — eis a recompensa da indireção do Capítulo 17):

```
/interface list member set [find interface=ether2] interface=bridge-lan
```

Passo 3 — dois vizinhos de bridge. Em lab-r2 e lab-r3 (console do r3), garanta DHCP client na **ether2** de cada:

```
/ip dhcp-client add interface=ether2
```

Ambos devem receber 192.168.10.1xx do r1. E o teste de camada 2: de lab-r3, pingue o r2 (IP dele na LAN — veja em `/ip dhcp-server lease print` no r1):

```
/ping 192.168.10.105 count=3
```

Funciona — e repare: esse tráfego r3 r2 **não é roteado** pelo r1; ele é **comutado** pela bridge (mesma rede, camada 2). O firewall **forward** do r1 nem o vê.

Passo 4 — a host table. Em lab-r1, o caderno do carteiro:

```
/interface bridge host print
```

```
[vitor@lab-r1] > /interface bridge host print
Flags: L - LOCAL, D - DYNAMIC
Columns: MAC-ADDRESS, ON-INTERFACE, BRIDGE, AGE
#    MAC-ADDRESS      ON-INTERFACE  BRIDGE      AGE
0 D  08:00:27:3B:71:9E  ether2        bridge-lan  12s
1 D  08:00:27:C4:88:2A  ether2        bridge-lan   4s
2 L  08:00:27:6A:2B:1C  bridge-lan    bridge-lan
```

Os MACs de r2 e r3 aprendidos (flag D, com idade), ambos na **ether2** — no laboratório as duas VMs chegam pela mesma “tomada” (a rede interna lab-a é o switch do VirtualBox); num CRS físico, cada um apareceria na sua porta. A flag L marca o MAC da própria bridge.

Passo 5 — assistir ao aprendizado. Limpe um host e veja-o voltar:

```
/interface bridge host print where !local
```

Anote um MAC, gere silêncio (pare pings), aguarde o AGE crescer — e dispare um ping do r3: o AGE zera (quadro novo reensina). É o ciclo aprende-esquece que mantém a tabela enxuta.

Passo 6 — a bateria de sempre. O firewall sobreviveu à migração? Bateria de fumaça do Capítulo 21, versão rápida: r2 navega (/ping google.com), DHCP renova, SSH da LAN ao r1 morre, gerência viva. Se algo quebrou, o suspeito é a interface list do Passo 2.

22.4.3 Verificação

1. O IP da LAN, o DHCP server e a interface list apontam para **bridge-lan** (/ip address print, /ip dhcp-server print, /interface list member print);
2. r2 e r3 têm leases e se pingam **diretamente** (comutados, não roteados);
3. A host table mostra os MACs dinâmicos com AGE vivo;
4. A bateria de fumaça do firewall passou intacta.

22.4.4 Desafios

1. No Passo 3, o ping $r3 \rightarrow r2$ não passa pelo firewall **forward** do $r1$. Isso é um problema de segurança? Em que cenário de provedor a resposta é “sim” — e qual ferramenta (deste volume) resolve?
2. Um técnico deixou um IP configurado direto na **ether2 além** do IP da bridge. Que classe de defeitos isso causa? (Pense no que o RouterOS responde por ARP e por onde ele roteia.)
3. Num CRS326 com 24 portas na bridge, você precisa que o firewall inspecione o tráfego **entre** duas portas específicas. Quais são suas opções, e qual o preço de cada uma? (Tabela 22.1 ajuda.)
4. A host table de um switch de POP mostra o mesmo MAC **alternando** entre duas portas, com **AGE** sempre zerando. O que está acontecendo, e por que é urgente? (Palavra-chave para investigar: loop — e o próximo capítulo do livro que trata disso é o de STP... que não existe; o assunto aparece no 7. Por ora, raciocine.)

Solução dos desafios

1. Para uma LAN de **um** cliente (casa, escritório), não: os dispositivos são do mesmo dono e confiança. Vira problema quando a mesma bridge carrega **clientes diferentes** — vizinhos de condomínio num switch, assinantes numa torre: um infectado ataca os outros em camada 2, invisível para o firewall do roteador. As respostas do volume: separar clientes em **VLANs** (Capítulo 23 em diante) — e, em bridges que precisam mesmo filtrar, o firewall de bridge/**use-ip-firewall** (pagando o offload, Tabela 22.1).
2. O RouterOS passa a ter **dois caminhos** para a mesma rede: a interface física (com o IP esquecido) e a bridge. ARP pode ser respondido pelos dois lados, rotas conectadas duplicam (uma por interface) e o tráfego ora sai por uma, ora por outra — sintomas: conexões que “às vezes funcionam”, ARP incompleto em vizinhos, e a host table confusa. Regra do capítulo: porta-membro não tem configuração própria; camada 3 mora **só** na bridge.
3. Opções: (a) **use-ip-firewall=yes** — o firewall IP passa a ver o tráfego em bridge; preço: **todo** o tráfego da bridge desce para a CPU (mata o offload geral — inaceitável num POP carregado); (b) **bridge filter** (regras de camada 2) — mais barato, mas igualmente processado em CPU para o que casar; (c) a resposta arquitetural: **tirar as duas portas da bridge comum** — VLANs separadas + roteamento entre elas no próprio CRS ou num roteador acima: o firewall vê o tráfego **na travessia de camada 3**, onde ele é natural, e o resto do POP continua em hardware. A opção (c) é o desenho do Capítulo 26.
4. MAC alternando entre portas = o mesmo quadro chegando por dois caminhos: ou há um **loop físico** (dois cabos fechando um anel entre switches — broadcasts circulam e multiplicam até saturar tudo: é uma *broadcast storm* em formação, urgência máxima), ou alguém está **forjando o MAC** (ataque/equipamento defeituoso). Ação imediata: identificar as duas portas e derrubar uma; ação estrutural: STP/RSTP ligado nas bridges de POP (o RouterOS traz RSTP por padrão) e desenho sem anéis não intencionais.

22.5 Resumo

- **Comutar** (camada 2, por MAC, dentro da rede) **rotear** (camada 3, por IP, entre redes) — a bridge é o switch de software do RouterOS: aprende, encaminha, inunda.
- Configuração de camada 3 (IP, DHCP, firewall, listas) mora **na bridge**; portas-membro são tomadas burras. A **host table** é o caderno do carteiro — primeira parada do diagnóstico.
- Tráfego entre portas da mesma bridge **não passa** pelo firewall **forward** — consciência disso é segurança.
- **Hardware offload** (flag H): o switch chip comuta em velocidade de fio; recursos de software o matam **em silêncio** (Tabela 22.1) — CPU alta + throughput baixo num CRS = procurar o H perdido.
- Nos CRS3xx, o VLAN filtering preserva o offload — a porta de entrada dos próximos capítulos.

No próximo capítulo, o divisor de águas da rede de provedor: **VLANs e 802.1Q** — como um cabo só carrega dezenas de redes separadas.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/interface bridge` (portas, host table, STP) e as tabelas de compatibilidade de hardware offload por switch chip — consulte a do seu modelo antes de projetar um POP. Tanenbaum; Feamster; Wetherall (2021) fundamenta a comutação em camada 2 (aprendizado, inundação, spanning tree). As referências completas, em ABNT, estão no apêndice [Referências](#).

23 VLANs e 802.1Q

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o problema que a VLAN resolve: **redes separadas sem cabos separados**;
- Descrever a etiqueta **802.1Q**: onde ela entra no quadro, o que é o **VID** e o que significa um quadro *tagged* ou *untagged*;
- Usar o vocabulário sem tropeçar: **trunk**, **access**, VLAN nativa;
- Criar interfaces `/interface vlan` no RouterOS e montar o *router-on-a-stick* — várias redes roteadas por um cabo só;
- Reconhecer as VLANs na rede do provedor: serviços, clientes e gerência compartilhando a mesma fibra sem se ver.

23.1 O problema: um cabo, várias redes

O capítulo anterior terminou com um alerta: colocar clientes diferentes na mesma bridge é colocá-los na mesma sala. A solução ingênua — um cabo e uma porta para cada rede — morre na primeira conta: o POP atende a rede de clientes, a de gerência, a de servidores e a VLAN da operadora... por **uma** fibra até a torre. Precisamos de redes logicamente separadas sobre o mesmo meio físico.

A resposta tem trinta anos e está em todo equipamento de rede do planeta: a **VLAN** (*Virtual LAN*), padronizada como **IEEE 802.1Q**. A ideia cabe numa frase: **cada quadro Ethernet ganha uma etiqueta dizendo a que rede pertence** — e switches e roteadores tratam cada etiqueta como se fosse um cabo separado.

A etiqueta (*tag*) são 4 bytes inseridos dentro do cabeçalho Ethernet (6), dos quais importam dois campos: o **VID** (*VLAN ID*, 12 bits — redes de 1 a 4094) e a **prioridade** (3 bits — que o QoS do Volume 7 conhece como CoS).

Com a etiqueta, nasce o vocabulário que você usará todos os dias:

- Quadro **tagged**: carrega a etiqueta — viaja pelos enlaces entre equipamentos de rede;
- Quadro **untagged**: quadro comum, sem etiqueta — o que computadores, câmeras e ONUs falam (dispositivos finais em geral ignoram VLANs);
- Porta/enlace **trunk** (tronco): transporta **várias** VLANs, todas tagged — a fibra do POP à torre;

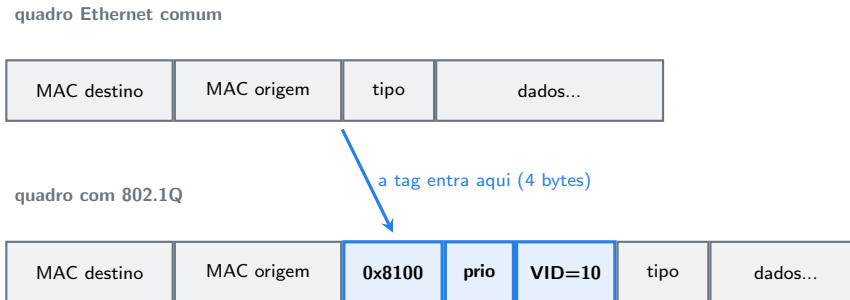


Figura 23.1: A etiqueta 802.1Q: 4 bytes entre o MAC de origem e o tipo — com o VID dizendo “este quadro pertence à VLAN 10”.

- Porta **access** (acesso): pertence a **uma** VLAN; recebe/entrega quadros untagged e o switch etiqueta/desetiqueta por ela — a porta do computador;
- **VLAN nativa**: a VLAN dos quadros untagged que chegam a um trunk (tradicionalmente a VID 1) — fonte clássica de surpresas de segurança; o 7 a domará com o conceito de **PVID**.

💡 Analogia para guardar

O 802.1Q transforma um cabo num **feixe de canos coloridos**: cada quadro ganha uma pulseira de cor (o VID) ao entrar, viaja pelo cano da sua cor e perde a pulseira ao sair para um dispositivo final. Trunk é o feixe inteiro; porta access é a boca de um cano só — e quem decide a cor é o switch, nunca o visitante.

23.2 VLANs no RouterOS: a interface vlan

No RouterOS, cada VLAN que o roteador deve **enxergar em camada 3** vira uma interface virtual sobre a interface física (ou sobre uma bridge):

```
/interface vlan add name=vlan10-clientes vlan-id=10 interface=ether2
/interface vlan add name=vlan20-gerencia vlan-id=20 interface=ether2
```

Cada **vlan** é uma interface completa: recebe IP, DHCP server, entra em interface lists, aparece no firewall — tudo que uma **ether** faz. Os quadros que saem por ela nascem tagged com o VID configurado; os que chegam tagged com aquele VID são entregues a ela.

O desenho que isso habilita é o **router-on-a-stick** (“roteador no palito”): um roteador pendurado por **um** cabo trunk num switch, roteando entre todas as VLANs:

```
/ip address add address=10.10.0.1/24 interface=vlan10-clientes
/ip address add address=10.20.0.1/24 interface=vlan20-gerencia
```

Duas redes, um cabo. O tráfego **dentro** de cada VLAN é comutado pelo switch (camada 2, sem tocar o roteador); o tráfego **entre** VLANs sobe o palito, é **roteado** — e aqui está o prêmio de segurança: a travessia passa pelo firewall **forward**, exatamente o que faltava no desafio do Capítulo 22. Segmentar em VLANs devolve ao firewall a visão que a bridge única roubava.

Três notas de campo antes do laboratório:

1. **Os dois lados do enlace devem concordar:** VID 10 aqui é VID 10 lá. Uma ponta tagged e outra untagged = silêncio absoluto (o quadro chega, a etiqueta não casa, ninguém reclama). É o defeito número 1 de VLAN — e o motivo de documentar VIDs como se documenta endereçamento;
2. **MTU:** a tag rouba 4 bytes. Equipamentos modernos aceitam quadros de 1522 bytes sem drama; em rádios antigos e caminhos com túneis, VLANs somam-se ao problema de MTU que você já conhece do `change-mss` (4);
3. **/interface vlan não filtra** — ela apenas dá ao roteador presença nas VLANs. Impor “a porta X só fala VLAN 10” é papel do **bridge VLAN filtering**, o próximo capítulo. Aqui as duas pontas cooperam; lá, o switch policia.

! O conceito mais importante do capítulo

VLAN = **um cabo, várias redes**: a tag 802.1Q (VID) faz cada quadro pertencer a uma rede lógica, trunk transporta muitas (tagged), access entrega uma (untagged). No RouterOS, `/interface vlan` dá ao roteador um “pé” em cada VLAN — e rotear **entre** VLANs devolve o tráfego ao firewall. Regra de ouro do campo: as duas pontas do enlace precisam contar a mesma história de VIDs.

23.3 Laboratório

23.3.1 Ambiente

Topologia do volume (Capítulo 22): `lab-r1` com a `bridge-lan`, e `lab-r2/lab-r3` na rede `lab-a`. Neste capítulo, `r1` e `r2` conversarão por VLANs **sobre o mesmo enlace** que já os liga; o `r3` fará o papel de “intruso sem VLAN” na prova de isolamento.

23.3.2 Comandos passo a passo

Passo 1 — as VLANs no r1. Sobre a bridge da LAN (é o caso realista — o “palito” costuma ser uma bridge/trunk), crie duas VLANs com suas redes:

```
/interface vlan add name=vlan10-svc vlan-id=10 interface=bridge-lan comment="servicos"
/interface vlan add name=vlan20-ger vlan-id=20 interface=bridge-lan comment="gerencia de e
/ip address add address=10.10.0.1/24 interface=vlan10-svc
/ip address add address=10.20.0.1/24 interface=vlan20-ger
```

Passo 2 — o outro lado, no r2. O r2 precisa contar a mesma história de VLANs sobre a ether2:

```
/interface vlan add name=vlan10 vlan-id=10 interface=ether2
/interface vlan add name=vlan20 vlan-id=20 interface=ether2
/ip address add address=10.10.0.2/24 interface=vlan10
/ip address add address=10.20.0.2/24 interface=vlan20
```

Passo 3 — provar os canos. De lab-r2, um ping por cano:

```
/ping 10.10.0.1 count=3
/ping 10.20.0.1 count=3
```

Ambos limpos. Repare no que acabou de acontecer: pelo **mesmo cabo** (lab-a) onde já viviam a LAN untagged (192.168.10.0/24) e o DHCP, agora trafegam **mais duas redes**, cada uma no seu cano tagged — três redes, zero cabos novos.

Passo 4 — a prova do isolamento. O r3 está no mesmo cabo, **sem VLANs** configuradas. De lab-r3:

```
/ping 10.10.0.1 count=3
```

no route to host — para o r3, a VLAN 10 **não existe**: os quadros tagged passam pela placa dele e são descartados (etiqueta desconhecida). Mesmo que ele tivesse uma rota, não tem interface naquela rede. Agora dê a ele o “crachá” e veja a porta abrir:

```
/interface vlan add name=vlan10 vlan-id=10 interface=ether2
/ip address add address=10.10.0.3/24 interface=vlan10
/ping 10.10.0.1 count=3
```

Funciona. E é exatamente essa a limitação a entender: **quem controla a própria configuração pode “entrar” na VLAN** — num cabo compartilhado, a tag separa, mas não tranca. A tranca (a porta do switch decidindo quais VLANs aceita) é o bridge VLAN filtering do próximo capítulo. Remova o crachá do r3 ao terminar:

```
/ip address remove [find interface=vlan10]
/interface vlan remove [find name=vlan10]
```

Passo 5 — o roteador vendo os canos. Em lab-r1, o torch mostra os quadros por interface — observe a VLAN 10 enquanto o r2 pinga 10.10.0.1:

```
/tool torch interface=vlan10-svc
```

(Interrompa com q.) E as interfaces contam sua natureza:

```
/interface vlan print
/interface print where type=vlan
```

23.3.3 Verificação

1. r1 e r2 se pingam pelas **duas** VLANs, sobre o mesmo enlace da LAN untagged — três redes num cabo;
2. O r3 sem configuração de VLAN não alcança 10.10.0.1 (e você viu o erro certo: sem interface na rede, **no route to host**);
3. Com o “crachá” (interface vlan + IP), o r3 entrou — e você sabe explicar por que isso é uma limitação, não um recurso;
4. `/interface vlan print` nos dois roteadores conta a **mesma** história de VIDs (10 e 20).

23.3.4 Desafios

1. No r1 as VLANs foram criadas sobre a **bridge-lan**; no r2, sobre a **ether2**. Por que essa assimetria funciona — e quando criar a VLAN sobre a bridge é **obrigatório**?
2. Um provedor entrega ao cliente corporativo “internet na VLAN 100 e telefonia na VLAN 200” por uma fibra. Desenhe (comandos) o lado do cliente num RB5009: as duas VLANs sobre a **sfp-sfpplus1**, com DHCP client na 100 e um IP fixo 10.200.0.2/30 na 200.
3. O ping na VLAN 20 entre r1 e r2 funciona — mas deveria? A VLAN 20 é “gerência de equipamentos” e o r2 faz papel de CPE de cliente. Critique o desenho do laboratório com os olhos do Capítulo 21 e proponha a correção (que o Capítulo 26 consagrará).
4. Sobre o quadro da 6: um quadro já tagged com VID 10 pode receber **outra** tag por fora? Investigue o nome da técnica, o EtherType envolvido e um uso clássico de provedor.

Solução dos desafios

1. A tag viaja no quadro — não importa se quem a lê/escreve é uma interface física ou uma bridge: os VIDs casam e pronto. Criar sobre a **bridge é obrigatório** quando os quadros tagged podem chegar **por mais de uma porta** (o trunk pode migrar de porta, ou há redundância): a VLAN sobre a bridge enxerga todas as portas-membro; sobre uma **ether** específica, só aquele cabo. Regra prática moderna: se existe bridge, crie a VLAN sobre ela (com o filtering do próximo capítulo mandando no resto).

2.

```
/interface vlan add name=vlan100-inet vlan-id=100 interface=sfp-sfpplus1
/interface vlan add name=vlan200-voz vlan-id=200 interface=sfp-sfpplus1
/ip dhcp-client add interface=vlan100-inet
/ip address add address=10.200.0.2/30 interface=vlan200-voz
```

(E, sendo o RB5009 a borda do cliente: NAT com `out-interface=vlan100-inet` e o firewall do Volume 3 — as VLANs entram nas interface lists como qualquer WAN.)

3. O defeito: o CPE do cliente **tem um pé na VLAN de gerência** — qualquer cliente curioso (ou comprometido) com acesso ao próprio CPE alcança a rede que administra o parque. Ferido o princípio “gerência intocável” (Capítulo 18). Correção: gerência em VLAN que **só existe** entre equipamentos do provedor (o switch não a entrega em portas de cliente — filtering, próximo capítulo), CPEs gerenciados por outra via (a própria sessão PPPoE/túnel dedicado), e firewall barrando a VLAN 20 de qualquer origem não-provedor. No laboratório seguimos didáticos; no POP real, a VLAN 20 jamais desceria ao cliente.

4. Pode — é o **QinQ** (802.1ad): uma tag de serviço (S-TAG, EtherType 0x88a8) envelopando a tag do cliente (C-TAG, 0x8100). Uso clássico: o provedor transporta as VLANs **do cliente** (quaisquer VIDs que ele use) através do backbone dentro de uma única VLAN de serviço — “VLAN do cliente 55” viaja intacta por dentro da “VLAN de transporte 1055”. No RouterOS: `/interface vlan` sobre `/interface vlan`, ou `ethertype=0x88a8` no bridge VLAN filtering.

23.4 Resumo

- VLAN = redes lógicas sobre o mesmo físico: a tag **802.1Q** (4 bytes, 6) com o **VID** (1–4094) diz a que rede o quadro pertence.
- Vocabulário: **tagged** (com etiqueta, entre equipamentos), **untagged** (dispositivos finais), **trunk** (muitas VLANs), **access** (uma, untagged), VLAN nativa (untagged no trunk — cuidado).
- `/interface vlan` sobre ether/bridge dá ao roteador um pé em cada VLAN — IP, DHCP, firewall, tudo normal; **router-on-a-stick** roteia entre VLANs por um cabo, devolvendo a travessia ao firewall.
- As duas pontas devem contar a **mesma história de VIDs** — ponta errada não gera erro, gera silêncio.
- A tag separa, mas **não tranca** num cabo compartilhado: quem configura a própria ponta entra na VLAN — a tranca é o bridge VLAN filtering.

No próximo capítulo, exatamente ela: **bridge VLAN filtering** — o switch (de software e de hardware) decidindo quais VIDs entram e saem em cada porta: trunk e access de verdade, com PVID e tudo.

Bibliografia do capítulo

A documentação oficial (MikroTik, 2026a) cobre `/interface vlan` e os cenários de combinação com bridge (incluindo os casos a evitar). O padrão IEEE 802.1Q define tag, VIDs

e prioridades; Tanenbaum; Feamster; Wetherall (2021) o apresenta didaticamente. As referências completas, em ABNT, estão no apêndice [Referências](#).

24 Bridge VLAN filtering

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Ativar o **VLAN filtering** numa bridge e explicar o que muda: o switch passa a **policar** etiquetas em vez de ignorá-las;
- Configurar portas **trunk** e **access** de verdade, com a tabela `/interface bridge vlan`, o **PVID** e os **frame-types**;
- Explicar o caminho de um quadro pelo switch filtrado: o que acontece na entrada (ingress) e na saída (egress) de cada porta;
- Manter o acesso de **gerência** ao próprio switch numa VLAN — incluindo o detalhe que derruba iniciantes: a bridge como membro tagged;
- Aplicar a ordem de configuração que **não** tranca você para fora.

24.1 Do consentimento à lei

O laboratório do Capítulo 23 terminou com uma demonstração incômoda: o r3 entrou na VLAN 10 sozinho, bastando configurar a própria placa. Num cabo compartilhado, a tag 802.1Q é um **acordo entre pontas** — e acordos não protegem contra quem não os assina.

O **bridge VLAN filtering** transforma o acordo em lei: com ele ativo, a bridge (o switch) passa a **decidir, porta a porta**, quais VIDs entram, quais saem, e com ou sem etiqueta. O cliente pode etiquetar o que quiser; se a porta dele só admite VLAN 10 untagged, é isso que ele terá.

Liga-se com um parâmetro:

```
/interface bridge set bridge-sw vlan-filtering=yes
```

— mas **não rode isso ainda**. A ordem importa (é a pegadinha número 1 do capítulo, tratada adiante). Antes, entenda as três peças que passam a mandar.

24.1.1 Peça 1: a tabela de VLANs da bridge

/interface bridge vlan é o mapa: para cada VLAN, quem participa e como:

```
/interface bridge vlan add bridge=bridge-sw vlan-ids=10 tagged=bridge-sw,ether2 untagged=e
```

Traduzindo: “a VLAN 10 existe nesta bridge; sai **tagged** pelo trunk **ether2** (e para a própria bridge — já explico); sai **untagged** pela porta de acesso **ether3**”. Na **saída** (egress) de cada porta, essa tabela é lei: quadro da VLAN 10 saindo pela ether3 perde a etiqueta; pela ether2, mantém; por uma porta que não está na linha, **não sai**.

24.1.2 Peça 2: o PVID — a etiqueta de entrada

E o quadro **untagged** que *chega* a uma porta — de que VLAN ele é? A resposta é o **PVID** (*Port VLAN ID*) da porta:

```
/interface bridge port set [find interface=ether3] pvid=10
```

Todo quadro sem etiqueta que entrar pela **ether3** é internamente tratado como VLAN 10. PVID é o “carimbo de entrada” — e o par tabela-de-saída + PVID-de-entrada define os dois papéis clássicos:

Tabela 24.1: As receitas de porta access e trunk

Papel	Na tabela vlan	PVID	frame-types
access (VLAN 10)	untagged=porta na VLAN 10	10	admit-only-untagged-and-priority-tagged
trunk	tagged=porta em cada VLAN transportada	(irrelevante se só aceita tagged)	admit-only-vlan-tagged

24.1.3 Peça 3: frame-types — o que a porta admite

O terceiro controle é o filtro de **entrada** (ingress): que tipos de quadro a porta aceita?

- **admit-all** (padrão): tagged e untagged;
- **admit-only-untagged-and-priority-tagged**: só untagged — a porta access ortodoxa: se o cliente mandar quadro etiquetado (como o r3 “esperto” do capítulo passado), **descarte**;
- **admit-only-vlan-tagged**: só tagged — o trunk ortodoxo: quadro sem etiqueta num tronco é acidente ou ataque à VLAN nativa; descarte.

Com as três peças, o caminho do quadro fecha: **ingress** (frame-types admite? untagged ganha o PVID) → comutação **dentro da VLAN** (host table por VLAN) → **egress** (a tabela diz se sai e se sai tagged/untagged).

Analogia para guardar

O switch filtrado é uma **portaria de eventos com pulseiras**: na entrada, o segurança da porta confere o traje (frame-types) e coloca a pulseira da casa em quem chega sem (PVID); lá dentro, cada salão é uma VLAN; na saída, a porta VIP mantém a pulseira (tagged, trunk) e a porta do salão único a corta (untagged, access). Pulseira trazida de casa não vale: quem manda é a portaria.

24.2 A gerência e a pegadinha do tagged=bridge

Releia a linha da tabela: `tagged=bridge-sw,ether2`. Por que a **própria bridge** aparece como membro tagged?

Porque o switch também é um equipamento com IP de gerência — e se a gerência mora numa VLAN (como deve, Capítulo 26), os quadros dela precisam chegar **à CPU do próprio switch**. Na arquitetura do RouterOS, a CPU é alcançada “saindo” pela interface bridge: incluir a bridge como **tagged** na VLAN é abrir o cano entre aquela VLAN e o cérebro do equipamento — e é sobre esse cano que a `/interface vlan` de gerência (com o IP) se pendura:

```
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-sw
/ip address add address=10.99.0.2/24 interface=vlan99-ger
```

Esquecer o **bridge** na lista tagged da VLAN 99 = gerência morta no momento em que o filtering ligar. O que nos leva à pegadinha-mor:

Aviso

vlan-filtering=yes é a última linha, nunca a primeira. No momento em que o filtering liga, a tabela de VLANs vira lei — e tudo que não está nela **para de passar**, incluindo o seu acesso. A ordem segura: (1) montar a tabela `/interface bridge vlan` completa (com a bridge nos tagged das VLANs de gerência); (2) configurar PVIDs e frame-types das portas; (3) conferir tudo com **print**; (4) **só então** ligar o filtering — de preferência com acesso por um caminho fora da bridge (nossa `ether1` de gerência do laboratório, o console serial no CRS real) e Safe Mode.

Uma nota de desempenho para fechar a teoria: nos **CRS3xx**, tudo isso — tabela, PVID, frame-types — é programado **no switch chip**: filtering com flag H viva, em velocidade de fio (Capítulo 22). Nos CRS1xx/2xx antigos, ligar o filtering derruba o tráfego para a CPU — lá, usa-se o menu legado do chip (Capítulo 25). Saber **qual** dos dois mundos é o seu equipamento é o primeiro passo de qualquer projeto de POP.

! O conceito mais importante do capítulo

Com VLAN filtering, quem manda é o **switch**, não as pontas: ingress (frame-types + PVID) decide o que entra e com que VLAN; a tabela `/interface bridge vlan` decide por onde sai e se sai tagged. Porta access e trunk são só combinações dessas peças (Tabela 24.1). E a liturgia de ativação — tabela primeiro, bridge tagged para gerência, filtering por último — é o que separa a segmentação do POP de uma viagem até ele.

24.3 Laboratório

24.3.1 Ambiente

Os três CHRs com papéis novos: **lab-r2 será o switch**. No VirtualBox, devolva o Adaptador 2 de lab-r3 para a rede interna **lab-b** (desfazendo o ajuste do Capítulo 22). Topologia:

- lab-r1 (roteador): ether2 → lab-a — já tem a `vlan10-svc` (10.10.0.1/24) sobre a `bridge-lan` (Capítulo 23);
- lab-r2 (switch): ether2 → lab-a (**trunk** para o r1), ether3 → lab-b (**access VLAN 10** para o r3). Gerência intacta pela `ether1` (fora da bridge — nosso “console serial”);
- lab-r3 (dispositivo final): ether2 → lab-b, **sem nenhuma configuração de VLAN** — o ponto do exercício.

Limpe no r2 as interfaces vlan do capítulo anterior (`/interface vlan remove [find]` após remover os IPs correspondentes: `/ip address remove [find interface~"vlan"]`).

24.3.2 Comandos passo a passo

Passo 1 — a bridge do switch. Em lab-r2:

```
/interface bridge add name=bridge-sw comment="switch do lab"
/interface bridge port add bridge=bridge-sw interface=ether2
/interface bridge port add bridge=bridge-sw interface=ether3
```

Passo 2 — a tabela ANTES do filtering. A VLAN 10 sai tagged pelo trunk e untagged pela porta do r3; e já deixamos a bridge como membro tagged (gerência futura na VLAN — desafio 3 usa):

```
/interface bridge vlan add bridge=bridge-sw vlan-ids=10 tagged=bridge-sw,ether2 untagged=ether3
```

Passo 3 — os papéis das portas. Trunk ortodoxo na ether2, access ortodoxo na ether3:

```
/interface bridge port set [find interface=ether2] frame-types=admit-only-vlan-tagged
/interface bridge port set [find interface=ether3] pvid=10 frame-types=admit-only-untagged
```

Passo 4 — conferir, e só então ligar. A liturgia:

```
/interface bridge vlan print
/interface bridge port print
/interface bridge set bridge-sw vlan-filtering=yes
```

(Seu SSH ao r2 chega pela **ether1**, fora da bridge — o filtering não o toca. Num CRS real, este seria o momento do Safe Mode.)

Passo 5 — o milagre do access. Em lab-r3 — que não sabe o que é VLAN — apenas um endereço na rede da VLAN 10:

```
/ip address add address=10.10.0.30/24 interface=ether2
/ping 10.10.0.1 count=3
```

Funciona. Acompanhe o quadro: sai do r3 untagged → entra na ether3 do switch (frame-types admite; PVID carimba VLAN 10) → comutado na VLAN 10 → sai pelo trunk ether2 **tagged** → chega ao r1, cuja **vlan10-svc** sobre a bridge o reconhece. O r3 nunca soube da etiqueta — o switch cuidou de tudo. É exatamente assim que a ONU do cliente recebe “a VLAN de PPPoE” sem saber.

Passo 6 — a lei nas portas. Duas tentativas de violação:

- (a) Em lab-r3, banque o esperto do capítulo passado — tente entrar tagged pela porta access:

```
/interface vlan add name=vlan10 vlan-id=10 interface=ether2
/ip address add address=10.10.0.31/24 interface=vlan10
/ping 10.10.0.1 count=3
```

Silêncio total — o **frame-types** da access descarta quadros etiquetados na entrada. A tranca que faltava no Capítulo 23 existe. Desfaça (`/ip address remove [find interface=vlan10]`, `/interface vlan remove vlan10`).

- (b) Em lab-r1, tente uma VLAN que **não está na tabela** do switch:

```
/interface vlan add name=vlan30-teste vlan-id=30 interface=bridge-lan
/ip address add address=10.30.0.1/24 interface=vlan30-teste
```

De lab-r3... nada a testar: a VLAN 30 morre no trunk do r2 (egress: não está na tabela, não sai por porta nenhuma). Confirme no r2 que só a 10 (e a 1 padrão) existem: `/interface bridge vlan print`. Limpe o teste no r1.

24.3.3 Verificação

1. `/interface bridge vlan print` no r2 mostra a VLAN 10 com `tagged=bridge-sw,ether2` e `untagged=ether3` (a flag D da VLAN 1 padrão é normal);
2. O r3, **sem configuração de VLAN**, pinga 10.10.0.1 através do switch — access + PVID funcionando;
3. As duas violações do Passo 6 falharam em silêncio (tagged na access; VID fora da tabela no trunk);
4. Sua gerência ao r2 continuou viva o tempo todo (ela está fora da bridge — e você sabe por que isso importou).

24.3.4 Desafios

1. No Passo 6a, o quadro do r3 morreu na **entrada** da porta access. Se o `frame-types` fosse o padrão `admit-all`, o ataque funcionaria? Percorra ingress → comutação → egress e responda com precisão.
2. Migre a gerência do **r2** para dentro da VLAN 99 pelo trunk: acrescente a VLAN 99 à tabela (quem é tagged?), crie a `/interface vlan` sobre a bridge com IP 10.99.0.2/24 e a contraparte no r1 (10.99.0.1/24 sobre a bridge-lan). Liste os comandos nos dois lados e o teste. (Não desative a ether1 — rede de segurança do laboratório.)
3. Um trunk com `pvid=1` e `frame-types=admit-all` transporta as VLANs 10 e 20. Explique o ataque clássico de **VLAN hopping** por dupla etiqueta (802.1Q-in-802.1Q) que essa configuração permite — e como as receitas da Tabela 24.1 o previnem.
4. No CRS326 do POP, você ligou `vlan-filtering=yes` e o tráfego continuou em velocidade de fio; no CRS125 antigo do armário, a CPU foi a 100%. Explique a diferença — e onde a configuração de VLAN do CRS125 deveria viver (dica: próximo capítulo).

Solução dos desafios

1. Com `admit-all` no ingress, o quadro tagged VID 10 **entraria**. Comutação: ele pertence à VLAN 10, que existe na bridge — segue. Egress pelo trunk: a tabela permite a 10 tagged — **saí**. Ou seja: sim, o ataque voltaria a funcionar; a única defesa da porta access contra etiquetas trazidas de casa é o `frame-types` restritivo. (O PVID não ajuda aqui — ele só age sobre quadros *sem* etiqueta.)

2. No r2:

```
/interface bridge vlan add bridge=bridge-sw vlan-ids=99 tagged=bridge-sw,ether2
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-sw
/ip address add address=10.99.0.2/24 interface=vlan99-ger
```

No r1:

```
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-lan
/ip address add address=10.99.0.1/24 interface=vlan99-ger
```

Teste: do r1, `/ping 10.99.0.2`. Os pontos didáticos: a VLAN 99 é tagged **no trunk e na própria bridge** (sem `bridge-sw` na lista, o IP de gerência nunca veria os quadros), e **não** aparece na ether3 — o r3 não tem como alcançá-la.

3. O atacante numa porta com PVID 1 envia quadro com **duas** tags: externa VID 1, interna VID 20. O switch remove a externa (casa com o PVID/nativa do trunk em alguns cenários de egress untagged) e reenvia — o quadro chega ao próximo switch **com a tag interna exposta** (VID 20): o atacante injetou tráfego na VLAN 20 sem pertencer a ela. As receitas previnem por três lados: trunk `admit-only-vlan-tagged` sem VLAN nativa untagged (nada sai untagged no trunk), PVID de access em VLAN sem privilégio, e a VLAN 1 fora do uso (nem na tabela).

4. O CRS326 (série 3xx) programa o filtering **no switch chip** — a CPU só vê o que é para ela (gerência); dados correm em hardware com a flag H. O CRS125 (série 1xx) não sabe fazer 802.1Q da bridge no chip: `vlan-filtering=yes` derruba a comutação para a CPU — 100% e vazão de roteadorzinho. No CRS125, a VLAN se configura no **menu legado do switch chip** (`/interface ethernet switch`), assunto do Capítulo 25 — o mesmo resultado, programado na linguagem daquele hardware.

24.4 Resumo

- VLAN filtering transforma a bridge em **switch que polícia**: ingress (`frame-types + PVID` carimbando untagged), comutação por VLAN, egress pela tabela `/interface bridge vlan` (sai/não sai; tagged/untagged).
- **Access** = untagged na tabela + PVID + só-untagged; **trunk** = tagged na tabela + só-tagged (Tabela 24.1) — e etiqueta trazida pelo cliente morre na porta.
- Gerência em VLAN exige a **bridge como membro tagged** e a `/interface vlan` sobre a bridge — esquecer é gerência morta.
- Liturgia de ativação: **tabela → portas → conferir → filtering por último**, com acesso alternativo garantido (console/ether fora da bridge) — `vlan-filtering=yes` prematuro é a viagem clássica ao POP.
- CRS3xx: filtering em hardware (H vivo); CRS1xx/2xx: filtering na CPU — lá, o caminho é o switch chip legado (próximo capítulo).

No próximo capítulo, descemos ao porão: o **switch chip** dos CRS — o menu legado, as diferenças entre gerações e como extrair velocidade de fio de cada uma.

Bibliografia do capítulo

A página *Bridge VLAN Filtering* da documentação oficial (MikroTik, 2026a) é a referência canônica — inclusive a tabela de quais modelos fazem filtering em hardware. Os guias *Basic VLAN switching* da mesma documentação cobrem cenários por família de produto. As referências completas, em ABNT, estão no apêndice [Referências](#).

25 Switch chip nos CRS

i Objetivos de aprendizagem

Ao final deste capítulo, você será capaz de:

- Explicar o que um **switch chip** faz e como o RouterOS o expõe em `/interface ethernet switch`;
- Escolher o caminho de configuração certo por família — **CRS3xx/5xx** (tudo pela bridge, offload automático) *versus* **CRS1xx/2xx** e **RouterBOARDS** (menu legado do chip);
- Usar os recursos que moram no chip além de VLAN: **rule table** (ACLs em velocidade de fio), **port mirroring** e a host table de hardware;
- Reconhecer os limites de cada chip (tamanho de tabelas, recursos) na ficha técnica antes de projetar o POP;
- Diagnosticar o clássico “CRS lento”: descobrir para onde o tráfego está indo — chip ou CPU.

25.1 O porão do switch

Dois capítulos falamos de bridge como se tudo fosse software — e o Capítulo 22 avisou: nos CRS, quem trabalha de verdade é o **switch chip**, um circuito dedicado (Marvell nos CRS3xx/5xx, Atheros/ Qualcomm nos antigos e nas RouterBOARDS) com suas próprias tabelas de MACs, de VLANs e de regras, comutando em velocidade de fio enquanto a CPU dorme.

O RouterOS expõe o porão em `/interface ethernet switch`:

```
/interface ethernet switch print
```

```
[admin@crs326] > /interface ethernet switch print
Columns: NAME, TYPE, MIRROR-SOURCE, MIRROR-TARGET
# NAME      TYPE          MIRROR-SOURCE  MIRROR-TARGET
0 switch1   Marvell-98DX3236  none           none
```

Ali você vê o chip pelo nome (pesquise-o: a página *Switch Chip Features* da documentação diz exatamente o que o **seu** sabe fazer) e os menus dele: portas, host table de hardware, rule table, espelhamento. Todo equipamento MikroTik com mais de uma porta comutável tem um — até o hEX da casa do cliente.

25.2 Dois mundos, uma decisão

A pergunta prática que define qualquer projeto de switching MikroTik: **por onde configurar as VLANs?** A resposta depende da geração (Figura 25.1):

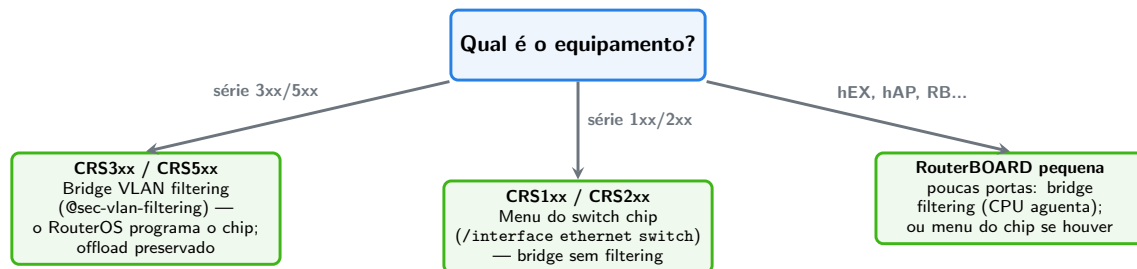


Figura 25.1: A decisão de projeto: em qual “idioma” configurar o switching, conforme a geração do hardware.

- **CRS3xx/5xx — o mundo unificado.** Você configura **só a bridge** (7) e o RouterOS traduz para o chip: tabela de VLANs, PVIDs, tudo vira programação de hardware, com a flag **H** intacta. É o presente e o futuro — e o motivo de este livro ter ensinado a bridge primeiro;
- **CRS1xx/2xx — o mundo legado.** O chip (QCA8511 e afins) é capaz — mas não fala o idioma da bridge. As VLANs se configuram **no menu do chip**: `/interface ethernet switch vlan` (a tabela), ... `port` (modo de cada porta: `vlan-mode`, `vlan-header`, `default-vlan-id` — o PVID daquele mundo). A bridge existe só para agrupar, **sem** `vlan-filtering`. Os conceitos são os mesmos do 7 — muda a sintaxe;
- **RouterBOARDS pequenas** — o hEX e irmãos têm chips básicos: para meia dúzia de portas de escritório, o bridge filtering na CPU resolve; onde o chip suportar (`switch1` com menu de VLAN), o legado serve. Consulte sempre a página do chip.

⚠ Aviso

Não misture os dois idiomas. Bridge com `vlan-filtering=yes` e VLANs no menu do switch chip ao mesmo tempo é receita de comportamento indefinido — cada geração tem seu caminho, e a documentação do modelo diz qual. O sintoma de mistura (ou do idioma errado): o “CRS lento” — CPU alta, throughput de software. Diagnóstico em dois comandos: `/interface bridge port print` procurando **H** ausente, e `/system resource print` com CPU suspeita sob tráfego que “deveria ser de graça”.

25.3 O que mais mora no chip

VLAN é o prato principal, mas o porão tem mais ferramentas — todas em velocidade de fio:

Rule table — ACLs de hardware. Uma tabela de regras estilo firewall (matchers de MAC, IP, porta; ações de drop/redirect/mirror) processada **pelo chip**:

```
/interface ethernet switch rule add switch=switch1 ports=ether5 src-mac-address=!AA:BB:CC:
```

(`new-dst-ports=""` = descartar.) É o jeito de impor “nesta porta, só este equipamento” num POP — sem gastar um ciclo de CPU. As regras variam por chip; as dos CRS3xx são particularmente ricas (é o embrião do *Layer 3 hardware offloading* dos CRS3xx/5xx mais novos — roteamento no chip, fora do nosso escopo, mas bom saber que existe).

Port mirroring — o grampo legítimo. Copiar o tráfego de uma porta para outra, onde um analisador escuta:

```
/interface ethernet switch set switch1 mirror-source=ether5 mirror-target=ether10
```

No **@sec-torch-sniffer** você verá o sniffer do próprio RouterOS; o mirror é a versão “de verdade” para depurar a porta de um cliente sem entrar no caminho.

Host table de hardware. A prima da `/interface bridge host`: `/interface ethernet switch host print` mostra os MACs que o **chip** conhece — e aceita entradas estáticas (prender um MAC a uma porta). Nos CRS3xx integrada à da bridge; nos legados, é a que vale.

E os **limites**: cada chip tem tabelas finitas — tantos mil MACs, tantas VLANs ativas, tantas regras de ACL. A ficha técnica (2) e a página *Switch Chip Features* trazem os números; um POP com mais MACs que a tabela do chip degrada para inundação (o switch vira hub para os excedentes). Projetar é conferir os números **antes**.

! O conceito mais importante do capítulo

O switch chip é um **computador de comutação com tabelas próprias** — VLANs, MACs, ACLs — e o seu trabalho é programá-lo no idioma certo da geração: **bridge filtering nos CRS3xx+** (o RouterOS traduz), **menu do chip nos legados**. Misturar idiomas ou exceder as tabelas derruba o tráfego para a CPU — e o “switch de 10 Gbps lento” quase sempre é isso: hardware esperando alguém falar a língua dele.

25.4 Laboratório

25.4.1 Ambiente

O CHR **não tem switch chip** — este laboratório é em parte “seco” (leitura de especificações e projeto), em parte executável em qualquer RouterBOARD física que você tenha (um hEX serve: chip básico, menus presentes). Sem hardware, faça os passos de projeto e os desafios — o raciocínio é o produto.

25.4.2 Comandos passo a passo

Passo 1 — conhecer o chip (hardware físico). Numa RouterBOARD:

```
/interface ethernet switch print
/interface ethernet switch port print
```

Anote o modelo do chip. Abra a página *Switch Chip Features* da documentação e localize a linha dele: quantas entradas de host table? Rule table existe? VLAN table de que tamanho?

Passo 2 — a host table de hardware. Com dispositivos ligados às portas:

```
/interface ethernet switch host print
```

Compare com `/interface bridge host print` — no hEX, as duas contam a mesma história por caminhos diferentes (chip × software).

Passo 3 — projeto no papel: o POP de 24 portas. Sem equipamento, este passo vale por dois. Dado um **CRS326-24G-2S+** (chip Marvell, série 3xx) servindo um POP: uplink SFP+ 1 = trunk para o roteador de borda; SFP+ 2 = trunk para a torre; ether1–20 = clientes dedicados (cada um numa VLAN própria, 101–120); ether21–24 = equipamentos do provedor (VLAN 99 de gerência). Escreva, em `.rsc`, a configuração completa no idioma **certo** para a série 3xx — bridge, tabela de VLANs, PVIDs, frame-types, interface de gerência. (Solução no desafio 1 — tente antes de olhar.)

Passo 4 — o mesmo POP num CRS125. Reescreva o miolo (as VLANs de duas portas de cliente e a gerência) no idioma legado: `/interface ethernet switch vlan + ... port` com `vlan-mode=secure`, `vlan-header` e `default-vlan-id`. Compare o esforço — e entenda por que a série 3xx venceu.

25.4.3 Verificação

1. (Com hardware) Você identificou o chip do seu equipamento e localizou seus limites na documentação;
2. O `.rsc` do Passo 3 segue a liturgia do 7 (tabela → portas → filtering por último) e a gerência mora na VLAN 99 com a bridge como tagged;
3. Você sabe apontar, nos dois idiomas, onde vive cada conceito: tabela de VLANs, PVID, admissão de quadros;
4. Dado um “CRS lento”, você lista os dois comandos de diagnóstico e os três suspeitos (idioma errado, mistura, tabela estourada).

25.4.4 Desafios

1. Escreva o `.rsc` do Passo 3 (CRS326). Estruture: bridge com todas as portas; VLANs 101–120 (cada uma tagged no uplink SFP+1 e untagged na porta do cliente); VLAN 99 tagged nos dois SFP+ e untagged nas ether21–24; PVIDs e frame-types; `vlan-filtering=yes` ao final. (Abrevie as 20 VLANs de cliente mostrando 2 e indicando o padrão.)
2. No desenho do desafio 1, cada cliente está **sozinho** na própria VLAN — clientes não se enxergam em camada 2. Que problema clássico de condomínio/POP isso resolve? E o que o roteador de borda precisa ter para que os clientes continuem chegando à internet?
3. Um POP com CRS326 tem 6 mil dispositivos atrás de uma porta (uma torre PtMP inteira). A host table do chip suporta 16 mil MACs. Outro integrador propõe ligar **três** torres iguais no mesmo switch. Faça a conta e diga o risco — e duas mitigações de projeto.
4. A rule table permite “nesta porta, só este MAC”. Por que essa regra é ao mesmo tempo uma ótima defesa num POP e uma péssima ideia numa porta de torre PtMP? O que usar em cada caso?

Solução dos desafios

1. Esqueleto (padrão indicado, 2 de 20 clientes mostrados):

```
/interface bridge add name=bridge-pop
/interface bridge port
add bridge=bridge-pop interface=sfp-sfpplus1 frame-types=admit-only-vlan-tagged
add bridge=bridge-pop interface=sfp-sfpplus2 frame-types=admit-only-vlan-tagged
add bridge=bridge-pop interface=ether1 pvid=101 frame-types=admit-only-untagged-and-prio
add bridge=bridge-pop interface=ether2 pvid=102 frame-types=admit-only-untagged-and-prio
# ... ether3-20: pvid=103..120, mesmo frame-types
add bridge=bridge-pop interface=ether21 pvid=99 frame-types=admit-only-untagged-and-prio
# ... ether22-24 idem
/interface bridge vlan
add bridge=bridge-pop vlan-ids=101 tagged=bridge-pop,sfp-sfpplus1 untagged=ether1
add bridge=bridge-pop vlan-ids=102 tagged=bridge-pop,sfp-sfpplus1 untagged=ether2
# ... 103-120 no padrao
add bridge=bridge-pop vlan-ids=99 tagged=bridge-pop,sfp-sfpplus1,sfp-sfpplus2 untagged=e
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-pop
/ip address add address=10.99.1.2/24 interface=vlan99-ger
/interface bridge set bridge-pop vlan-filtering=yes
```

(Refinamento pró: `tagged=bridge-pop` só é necessário nas VLANs que a CPU precisa ver — a 99; nas de cliente, pode sair.)

2. Resolve o **vizinho bisbilhoteiro/atacante**: sem camada 2 comum, não há ARP spoofing entre clientes, não há vírus varrendo o condomínio, não há briga de DHCP

— cada cliente enxerga só o provedor. O roteador de borda precisa **ter presença em cada VLAN** (`/interface vlan 101–120`, tipicamente agregadas por PPPoE ou DHCP por VLAN) e rotear/NAT-ear — o router-on-a-stick do Capítulo 23 em escala. É o desenho que o Capítulo 26 consolida.

3. $3 \times 6\,000 = 18\,000$ MACs $> 16\,000$ da tabela: sob carga plena, ~2 mil MACs não cabem — o chip **inunda** o tráfego deles por todas as portas da VLAN: banda desperdiçada, privacidade vazada, CPU do que escuta castigada. Mitigações: (a) **quebrar o domínio L2**: rotear na torre (cada torre com sua sub-rede — os MACs de clientes morrem lá e o switch vê só o MAC do roteador da torre); (b) distribuir torres em switches distintos / limitar aprendizagem por porta. A lição: host table é recurso de projeto, não detalhe.

4. Na porta de um **cliente dedicado/servidor do POP**, o MAC é um e conhecido: a regra elimina troca de equipamento não autorizada e spoofing — ótima. Numa porta de **torre PtMP**, chegam **milhares** de MACs legítimos (todos os clientes atrás do rádio): travar em um MAC derruba todo mundo. Lá, o controle certo é por VLAN + os mecanismos da camada wireless (access-list do Volume 5) e, para contenção de L2, isolamento entre clientes no próprio rádio.

25.5 Resumo

- O **switch chip** é o comutador de verdade dos CRS e RouterBOARDS — tabelas próprias de MACs, VLANs e regras, expostas em `/interface ethernet switch`.
- A decisão de idioma (Figura 25.1): **CRS3xx/5xx = bridge VLAN filtering** (RouterOS programa o chip, H vivo); **CRS1xx/2xx = menu legado do chip**; nunca os dois juntos.
- Além de VLAN: **rule table** (ACLs em fio), **port mirroring** (grampo de diagnóstico), host table de hardware com entradas estáticas.
- Tabelas são **finitas**: MACs, VLANs e regras têm teto por chip — estourar = inundação silenciosa; projetar é conferir a página *Switch Chip Features* antes.
- “CRS lento” = tráfego na CPU: procure o H perdido, o idioma errado ou a tabela cheia.

No próximo capítulo, o volume fecha com o projeto integrador: a **segmentação completa de um POP** — gerência, clientes e serviços, do switch ao firewall.

Bibliografia do capítulo

As páginas *Switch Chip Features* (tabela por modelo) e *CRS3xx, CRS5xx switch chip features* da documentação oficial (MikroTik, 2026a) são leitura obrigatória de projeto; os manuais de cada CRS indicam o idioma de configuração suportado. As referências completas, em ABNT, estão no apêndice [Referências](#).

26 Segmentação da rede do provedor

i Objetivos de aprendizagem

Este capítulo integra o volume. Ao final, você será capaz de:

- Projetar o **plano de VLANs** de um POP: gerência, clientes e serviços, com numeração e endereçamento documentados;
- Aplicar os três princípios da segmentação de provedor: gerência isolada, cliente não vê cliente, e **toda travessia entre segmentos passa pelo firewall**;
- Montar o desenho completo — roteador de borda + switch filtrado + segmentos — combinando os quatro capítulos do volume;
- Policiar a fronteira entre segmentos com o firewall do Volume 3 (interface lists por VLAN);
- Reconhecer o mesmo desenho nos cenários reais: POP de fibra (VLANs até a OLT), torre de rádio e o condomínio corporativo.

26.1 A ordem de serviço

O provedor cresceu: o POP central, que era um roteador e um switch burro, vai ser reorganizado. A ordem de serviço:

POP Centro — reorganização de rede Segmentos: **gerência** (equipamentos do provedor), **clientes dedicados** (cada um isolado) e **serviços** (servidores do provedor: monitoramento, cache). Requisitos: cliente não enxerga cliente nem gerência; gerência alcançável só pela equipe; toda travessia entre segmentos passa pelo firewall da borda; switch comutando em hardware. Entregáveis: plano de VLANs, configuração de switch e borda, testes.

O primeiro entregável não é comando — é **tabela**. Todo POP bem operado tem um plano de VLANs escrito, e ele é o contrato entre quem configura o switch, quem configura a borda e quem chega de madrugada para consertar:

Tabela 26.1: O plano de VLANs do POP (extrato)

VLAN	Nome	Rede	Quem participa
99	gerência	10.99.0.0/24	equipamentos do provedor + borda
101	cliente-A	100.64.101.0/24	porta do cliente A + borda

VLAN	Nome	Rede	Quem participa
102	cliente-B	100.64.102.0/24	porta do cliente B + borda
200	serviços	10.200.0.0/24	servidores do POP + borda

Repare nas decisões embutidas: numeração com **significado** (99 = infra, 1xx = clientes, 2xx = serviços — o padrão se estende sem reunião); rede casada com a VLAN (o terceiro octeto repete o número — diagnóstico agradece); e **a borda participa de todas** — ela é o único ponto de travessia.

26.2 Os três princípios

Tudo que este volume ensinou converge em três frases:

1. **Gerência isolada.** A VLAN 99 existe apenas entre equipamentos do provedor: o switch não a entrega em porta de cliente (tabela de VLANs, 7), e o firewall da borda só a aceita da equipe (interface list **gerencia**, Capítulo 17). Quem compromete um CPE não ganha um pé na sua infraestrutura;
2. **Cliente não vê cliente.** Cada cliente na própria VLAN (Capítulo 25, desafio 2): sem camada 2 comum, morrem o ARP spoofing entre vizinhos, o vírus que varre o condomínio e a briga de DHCPs. O único caminho entre dois clientes é **subir à borda** — onde há firewall;
3. **Travessia só pelo firewall.** É a soma dos outros dois com o router-on-a-stick (Capítulo 23): o tráfego intra-VLAN é comutado em hardware (rápido, invisível — e tudo bem, é tráfego do mesmo dono); o tráfego **entre** VLANs é roteado na borda, atravessando o **forward** do Volume 3. Segmentar é, no fundo, **decidir o que o firewall vê**.

A Figura 26.1 mostra o desenho completo:

26.3 O firewall enxergando segmentos

O toque final vem do Volume 3, com as interface lists absorvendo as VLANs. Na borda, cada `/interface vlan` entra na lista do seu papel:

```
/interface list member add list=gerencia interface=vlan99-ger
/interface list member add list=LAN interface=vlan101-cliA
/interface list member add list=LAN interface=vlan102-cliB
/interface list add name=SERVICOS
/interface list member add list=SERVICOS interface=vlan200-srv
```

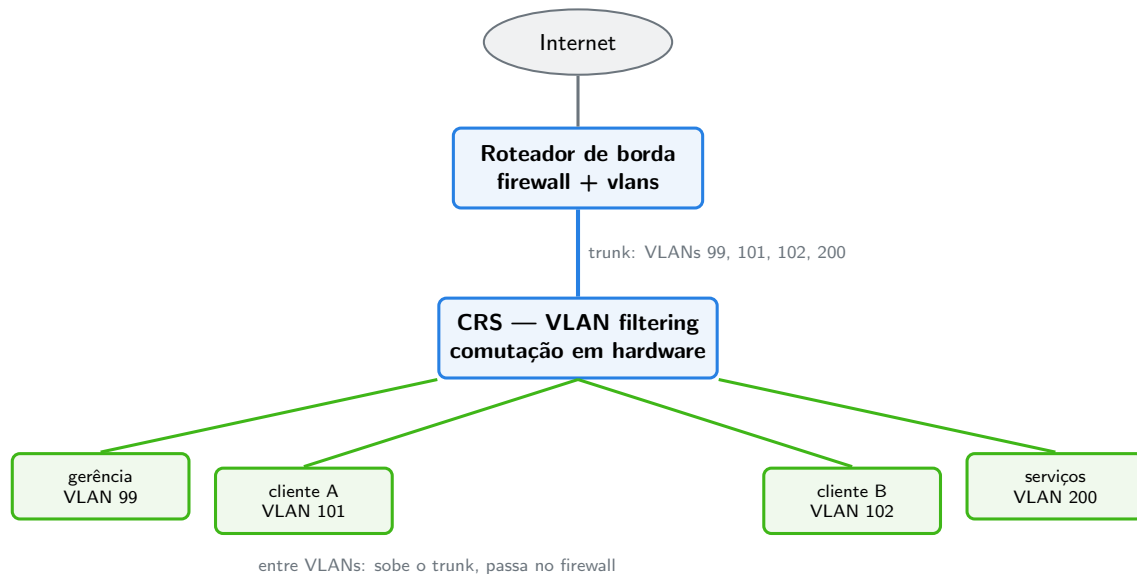


Figura 26.1: O POP segmentado: um trunk entre borda e switch; segmentos em portas access; travessias só pela borda.

E o firewall-base (Capítulo 21) já sabe o que fazer: gerência com acesso total ao input, clientes como LAN (DNS/DHCP servidos, gerência intocável), e as políticas novas entre segmentos viram poucas regras de forward — por exemplo, “clientes alcançam os serviços do POP só no que é público”:

```
/ip firewall filter add chain=forward in-interface-list=LAN out-interface-list=SERVICOS pr
/ip firewall filter add chain=forward in-interface-list=LAN out-interface-list=SERVICOS ac
```

Perceba o que aconteceu no desenho como um todo: o switch garante **quem está em que segmento** (camada 2, em hardware); o firewall garante **o que atravessa entre segmentos** (camada 3, com estado). Cada camada faz o que sabe — é a defesa em camadas do Capítulo 21 ganhando geografia.

! O conceito mais importante do capítulo

Segmentar é **desenhar o que o firewall vê**: VLANs (no switch, em hardware) definem os territórios; a borda, única ponte entre eles, aplica a política. Os três princípios — gerência isolada, cliente não vê cliente, travessia só pelo firewall — cabem em qualquer escala: do condomínio ao POP regional, muda o tamanho da 8, não o desenho.

26.4 Laboratório

26.4.1 Ambiente

O POP em miniatura, com os papéis do capítulo anterior: **lab-r1** = borda, **lab-r2** = switch (bridge **bridge-sw** com filtering do 7), **lab-r3** = cliente A. A VLAN 10 do capítulo anterior será renumerada para o plano oficial (8): 101 = cliente A, 99 = gerência.

26.4.2 Comandos passo a passo

Passo 1 — o plano no switch. Em **lab-r2**, ajuste a tabela ao plano (a VLAN 10 didática vira 101; a 99 entra para a gerência via trunk):

```
/interface bridge vlan set [find vlan-ids=10] vlan-ids=101
/interface bridge port set [find interface=ether3] pvid=101
/interface bridge vlan add bridge=bridge-sw vlan-ids=99 tagged=bridge-sw,ether2
```

Passo 2 — a borda fala o plano. Em **lab-r1**, renomeie o mundo antigo e crie os pés da borda em cada segmento:

```
/interface vlan set [find name=vlan10-svc] name=vlan101-cliA vlan-id=101
/ip address set [find interface=vlan101-cliA] address=100.64.101.1/24
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-lan
/ip address add address=10.99.0.1/24 interface=vlan99-ger
```

E a gerência do switch dentro da VLAN 99 (em **lab-r2**):

```
/interface vlan add name=vlan99-ger vlan-id=99 interface=bridge-sw
/ip address add address=10.99.0.2/24 interface=vlan99-ger
```

Teste da espinha: de **lab-r1**, `/ping 10.99.0.2 count=3` — a gerência do POP vive na VLAN 99, pelo trunk.

Passo 3 — o cliente no plano. Em **lab-r3** (que continua ignorante de VLANs — porta access cuida):

```
/ip address set [find address="10.10.0"] address=100.64.101.30/24
/ip route add dst-address=0.0.0.0/0 gateway=100.64.101.1
/ping 100.64.101.1 count=3
```

Passo 4 — as listas e a política. Em **lab-r1**, as VLANs entram nas listas do firewall:


```
/interface list member add list=gerencia interface=vlan99-ger
/interface list member add list=LAN interface=vlan101-clia
```

E o teste dos três princípios, direto do lab-r3 (cliente):

```
/ping 10.99.0.2 count=3
/ping 10.99.0.1 count=3
```

Ambos mortos — e repare o *porquê* de cada um: o 10.99.0.2 (switch) morre porque a regra “gerência intocável” do forward (Capítulo 18) barra a travessia LAN→gerência; o 10.99.0.1 (a própria borda) morre no **input...** a menos que o ICMP-com-teto o aceite — verifique! Se o ping na borda responde, é o accept de ICMP do Capítulo 17 fazendo seu papel; o que importa é que **nenhum serviço** da VLAN 99 se alcança (teste `ssh vitor@10.99.0.1` do r3: morto no drop all).

Passo 5 — a prova do hardware (conceitual) e a bateria. No CHR não há chip, mas confira a *intenção*: em lab-r2, `/interface bridge vlan print` e `port print` devem contar exatamente a 8 — num CRS326, essa mesma configuração estaria no chip, flag H viva. Feche com a bateria: r3 navega (`/ping 8.8.8.8` via borda + NAT), gerência do switch acessível **do r1** e inacessível **do r3**, e o export do plano:

```
/interface bridge vlan export file=pop-vlans
```

Crie o snapshot **pop-segmentado** nas três VMs — a Fase 1 do livro termina aqui, e a Fase 2 (wireless, PPPoE) partirá deste estado.

26.4.3 Verificação

1. O plano vive igual nos três lugares: 8 tabela do switch interfaces da borda (mesmos VIDs, mesmas redes);
2. O cliente navega (VLAN 101 → borda → NAT) sem enxergar gerência (VLAN 99 inalcançável de dentro, nos dois testes do Passo 4);
3. A gerência do POP (borda switch) conversa pela VLAN 99 sobre o trunk;
4. Exports arquivados e snapshot **pop-segmentado** criado nas três VMs.

26.4.4 Desafios

1. Chega o cliente B. Liste **todos** os toques necessários (switch, borda, firewall, tabela) — e conte-os. O que esse número diz sobre o desenho?
2. No Passo 4, o ping do r3 à borda (10.99.0.1) pode responder pelo accept de ICMP do input. Isso é um vazamento? Argumente os dois lados e proponha o refinamento se sua resposta for “sim”.

3. A torre de rádio rural entra no POP pelo segundo SFP+ do switch, e atrás dela vêm 300 clientes PtMP. Adapte o plano: a torre é trunk ou access? Que VLANs descem até ela — e qual princípio do capítulo é o mais difícil de manter no rádio (dica: os 300 compartilham o meio)?
4. O desenho do capítulo tem um ponto único de falha evidente. Qual é, o que acontece com cada tipo de tráfego quando ele cai — e quais volumes deste livro atacam esse problema?

Solução dos desafios

1. (a) Switch: 1 linha na tabela de VLANs (102 tagged no trunk, untagged na porta) + PVID/frame-types da porta; (b) borda: `/interface vlan 102` + IP 100.64.102.1/24 + membro da lista LAN; (c) firewall: **zero** (as políticas usam listas); (d) documentação: 1 linha na 8. Meia dúzia de toques, nenhum em regra de firewall — o desenho escala porque política e dados estão separados (Capítulo 18) e as listas absorvem o crescimento.
2. Lados: *não é vazamento* — ICMP com teto é diagnóstico universal e não expõe serviço (o drop all segura SSH/WinBox); *é vazamento* — o cliente consegue **enumerar** a infraestrutura (descobre que 10.99.0.1 existe e responde), primeiro passo de reconhecimento. Refinamento comum de provedor: restringir o accept de ICMP do input por origem (`in-interface-list=!LAN` para os IPs de infra, ou aceitar ICMP da LAN apenas para os IPs que a LAN deve ver — o gateway dela). Não há resposta única; há decisão documentada.
3. A torre é **trunk**: por ela descem a VLAN de gerência (99 — os rádios são equipamentos do provedor) e a(s) VLAN(s) de clientes daquela torre (ex.: 150). O princípio difícil é o “**cliente não vê cliente**”: na VLAN 150, os 300 compartilham o mesmo domínio L2 pelo ar — a segmentação por VLAN individual não escala para PtMP. As respostas moram na camada wireless (isolamento de clientes no AP, *default-forward* off — Volume 5) e no PPPoE (cada cliente numa sessão ponto a ponto — Volume 6), que devolvem o isolamento sem 300 VLANs.
4. O **roteador de borda**: toda travessia e toda saída passam por ele. Caindo: tráfego **intra-VLAN** continua (comutação no switch independe), mas internet, travessias entre segmentos e serviços da borda (DNS, DHCP) morrem — o POP viram ilhas isoladas funcionais. Ataques ao problema: redundância de gateway (VRRP — Volume 14), múltiplos uplinks (ECMP/failover — Volume 14) e desenho de rede com mais de um caminho (OSPF — Volume 8).

26.5 Resumo

- Segmentação começa por **documento**: o plano de VLANs
 - (8) com numeração significativa e redes casadas — o contrato entre switch, borda e madrugada.
- Os três princípios: **gerência isolada** (VLAN que cliente não recebe

- firewall), **cliente não vê cliente** (VLAN por cliente; PtMP resolve na camada wireless/PPPoE), **travessia só pelo firewall** (borda é a única ponte).
- O desenho (Figura 26.1): trunk único borda switch, access nos segmentos, comutação em hardware dentro, roteamento com estado entre.
- As VLANs entram nas **interface lists** e o firewall-base do Volume 3 absorve os segmentos quase sem regras novas — escala é separar política de dados.
- O mesmo desenho serve do condomínio ao POP regional — muda o tamanho da tabela, não a arquitetura.

Fecha-se o Volume 4 — e a **Fase 1** inteira: você opera o RouterOS, constrói a borda, veste o firewall e segmenta o POP. A Fase 2 leva tudo isso para a rua: o Volume 5 sobe na torre — rádio, regulamentação e os enlaces que levam internet aonde a fibra não chega.

Bibliografia do capítulo

Este capítulo integra os anteriores; além das referências deles, os guias *Basic VLAN switching* e os exemplos de segmentação da documentação oficial (MikroTik, 2026a) mostram variações do mesmo desenho por família de hardware. As referências completas, em ABNT, estão no apêndice [Referências](#).

Referências

Obras e documentos citados ao longo do manual, em formato ABNT.

BURGESS, Dennis. **Learn RouterOS**. 2. ed. Raleigh: Lulu Press, 2011.

DISCHER, Stephen R. W. **RouterOS by Example**. 2. ed. College Station: ISP Services, Inc., 2016.

MIKROTIK. **RouterOS Documentation**. <https://help.mikrotik.com/docs/>, a2026.

MIKROTIK. **Cloud Hosted Router (CHR) Documentation**. <https://help.mikrotik.com/docs/display/ROS/Cloud+Hosted+Router,+CHR>, b2026.

MIKROTIK. **MikroTik Product Catalog**. <https://mikrotik.com/products>, c2026.

REKHTER, Yakov *et al.* **Address Allocation for Private Internets**. RFC 1918, Internet Engineering Task Force, 1996.

TANENBAUM, Andrew S.; FEAMSTER, Nick; WETHERALL, David J. **Redes de Computadores**. 6. ed. Porto Alegre: Bookman, 2021.